Chance Constrained Optimization with Kernel Density Estimation

Master's Thesis

for the degree of

Master of Science (M.Sc.)

Data Science

in the Faculty of Sciences at

Friedrich-Alexander-Universität Erlangen-Nürnberg

submitted on November 4, 2026

by **Syed Ansab Waqar Gillani**

Supervisor: Prof. Dr. Michael Schuster

Friedrich-Alexander-Universität
Naturwissenschaftliche Fakultät

Contents

1 The Problem of Optimization Under Uncertainty **1**

1.1 Where Deterministic Optimization Falls Short 1

1.1.1 The mean-value approach and the flaw of averages 1

1.1.2 A worked dispatch example 2

1.1.3 Application contexts 2

1.2 Two Extremes and the Gap Between Them 3

1.2.1 Ignoring uncertainty and worst-case robustness 3

1.2.2 Chance constraints as a reliability specification 4

1.3 What Chance Constraints Are and Why They Are Hard 4

1.3.1 Individual and joint chance constraints 4

1.3.2 Three structural sources of difficulty 5

1.4 Distributional Uncertainty and the Residual-Space Shift 6

1.4.1 Why a parametric law can fail 7

1.4.2 Estimating residuals rather than uncertainty coordinates . . . 8

1.4.3 Bandwidth, one-sided bias, and joint events 9

1.5 Scope and Positioning 10

1.5.1 Positioning relative to anchor work 10

1.5.2 Research questions and chapter map 10

2 A Survey on Stochastic Optimization and Chance Constraints **12**

2.1 Optimization Paradigms Under Uncertainty 12

2.1.1 Deterministic, robust, stochastic, and chance-constrained models 12

2.1.2 Model uncertainty and distributional robustness 13

2.1.3 Individual and joint chance constraints 14

2.2 Classical Analytical Reformulations 14

2.2.1 Canonical affine case and origins 15

2.2.2 Log-concavity and its boundary 15

2.2.3 General nonlinear case 15

2.3 Approximation Methods and Their Trade-Offs 16

2.3.1 Parametric and Gaussian reformulations 16

2.3.2 Scenario approach 17

2.3.3 Sample average approximation (SAA) 18

2.3.4 Conditional value-at-risk (CVaR) and convex restrictions . . . 18

2.3.5 Smooth quantiles and black-box optimization 19

2.3.6 Comparison in one view 19

3 Chance Constraints and Kernel Density Estimation 21
3.1 Kernel Density Estimation for Residual Probabilities 21
3.1.1 Parzen–Rosenblatt estimator and residual CDF 21
3.1.2 Kernel choices and one-sided bias 22
3.1.3 Bandwidth and AMISE 23
3.1.4 Multivariate KDE and residual scalarization 25
3.2 KDE in the Chance-Constrained Literature 26
3.2.1 Application streams 26
3.2.2 Safety-oriented kernels and convergence theory 26

4 Smooth Approximation with Ordinary Symmetric KDE 28
4.1 The Residual-Space Transformation 28
4.1.1 Safe-event identity 28
4.1.2 Dimension reduction and its scope 29
4.2 Residual Samples 29
4.3 KDE of the Residual Distribution 30
4.4 The Integrated Kernel Reformulation 30
4.5 Gradient Computation 31
4.6 Stochastic Optimization Reformulated via Unbiased KDE 32
4.7 Properties of the Unbiased KDE Approximation 33

5 The Biased KDE Approach and Conservative Reformulation 36
5.1 Why Smoothing Alone Is Not Safe 36
5.1.1 Directional failure of a symmetric surrogate 36
5.1.2 Fixed-decision envelope versus optimizer selection 37
5.2 The Biasing Construction 37
5.3 Specific Biased Kernels 38
5.4 Stochastic Optimization Reformulated via Biased KDE 40
5.5 Joint Chance Constraints: Three Approaches 40
5.6 Log-Sum-Exp Smoothing of the Scalar Violation Statistic 42
5.7 Bandwidth Continuation and Kernel Switching 43

6 Convergence Theory 46
6.1 The Gap the Theory Fills 46
6.1.1 From estimator consistency to optimizer consistency 46
6.1.2 The three-theorem chain 47

6.2 Setup and Assumptions 47
6.3 The Passive Constraint Case 49
6.4 Limits of Approximate Solutions Are Optimal 50
6.5 Existence of a Convergent Solution Sequence 51
6.6 What the Theory Does and Does Not Cover 52
6.7 The Gap Between the Two Anchor Papers 54

7 Stochastic Optimal Control: Formulation and Audit 56
7.1 Problem Formulation and Residual Constraints 56
7.2 Discretization and Continuation 57
7.3 Local Lunar Benchmark 58
7.4 Conservation and Quadrature Audit 59
7.5 Interpretation and Transfer Limits 60
7.5.1 Stage feasibility gates 60
7.5.2 Keil and Schuster transfer hypotheses 60
7.5.3 Scope of valid claims 62

8 Implementation and Validation Protocol 63
8.1 Pipeline and Claim Contract 63
8.2 Data Generation and Residual Interfaces 64
8.3 Validation Layers and Reproducibility Limits 64

9 Experimental Evaluation 66
9.1 Design, Estimands, and Provenance 66
9.2 Static Nonlinear Benchmark 67
9.3 Paper-Aligned Hyperparameter Experiment 69
9.4 Energy Dispatch and Exact Distributional Check 71
9.5 Bandwidth, Sample Size, and Joint Statistics 72
9.6 Lunar Continuation Record 74
9.7 Evidence Summary 75

10 Discussion: Evidence and Method Selection 76
10.1 Answers to the Research Questions 76
10.2 Relation to the Anchor Literature 77
10.3 Mechanistic Synthesis 77
10.4 Method-Selection Boundaries 78
10.5 Future Work 78

11 Conclusion **81**
11.1 Contributions and Answers to the Research Questions 81
11.2 Limitations 82
11.3 Future Work 83

References **84**

A Core Implementation **89**
A.1 KDE Estimators 89
A.2 Benchmark Models 93
A.3 Optimization 95

1 The Problem of Optimization Under Uncertainty

Engineering decisions must satisfy constraints under conditions that are only partly known when the decision is made. A dispatch schedule must cover uncertain net demand, and a descent trajectory must satisfy terminal conditions despite disturbances. Optimizing against a nominal forecast does not quantify the probability of failure, while protection against every admissible disturbance can demand substantial resources. Chance constraints express the required reliability directly. This thesis studies how kernel density estimation (KDE) of constraint residuals turns that probabilistic requirement into a differentiable nonlinear constraint, and what statistical and numerical qualifications accompany the resulting decisions.

1.1 Where Deterministic Optimization Falls Short

Deterministic optimization replaces the uncertain input by a single representative value. This section first shows why that substitution provides no reliability level, then quantifies the consequence on a seeded dispatch example, and finally outlines the application areas in which the same mechanism appears.

1.1.1 The mean-value approach and the flaw of averages

Let $x \in X \subseteq \mathbb{R}^n$ denote the decision, $f : X \to \mathbb{R}$ its objective, and $\xi \in \mathbb{R}^d$ a random input. The scalar residual $g(x, \xi)$ is defined so that $g(x, \xi) \leq 0$ means the requirement is satisfied. A nominal formulation replaces ξ by a fixed value $\bar{\xi}$, often its mean when that mean exists:

$$\underset{x \in X}{\text{minimize}}\, f(x) \quad \text{subject to } g(x, \bar{\xi}) \leq 0. \tag{1}$$

This formulation is useful when variability is small relative to available margins, or when its consequences can be incorporated adequately into ordinary operating costs. It provides no reliability level by itself. If the nominal optimum has an active constraint, disturbances on the violating side of that boundary cause failure even though the nominal residual is zero. Moreover, for nonlinear g, evaluating the constraint at the mean input need not equal its expected value. These mechanisms underlie the flaw of averages (Savage, 2009); their practical severity depends on the residual distribution and the margin at the selected decision.

1.1.2 A worked dispatch example

Consider a single-bus system in which capacity x must cover uncertain net demand ξ. With a positive unit capacity cost $c > 0$, the nominal decision problem is

$$\underset{x \geq 0}{\text{minimize}}\, cx \quad \text{subject to } \xi - x \leq 0, \qquad g(x, \xi) = \xi - x. \tag{2}$$

For illustration, let $\log \xi \sim \mathcal{N}(\log 100, 0.18^2)$, with demand measured in MW. The repository figure generator draws 200,000 samples with random seed 7. Its sample mean is approximately 101.65 MW; choosing that capacity leaves approximately 46.34% of the generated demands uncovered. The empirical 95% quantile is approximately 134.41 MW. These are reproducible values from a seeded synthetic illustration, rather than measurements from an operating power system or a separate experimental dataset. Figure 1(a) marks both capacities against the demand distribution, and panel (b) traces the quantile capacity as the accepted shortfall varies.

For this monotone residual and positive cost, the chance-constrained optimum is the demand quantile $x^* = F_\xi^{-1}(1-\varepsilon)$. Thus reliability has an explicit capacity cost. The mean and median differ for the skewed lognormal law: the mean-value plan lies at its own empirical shortfall probability, not at $\varepsilon = 0.5$. As ε approaches zero, the population quantile and required capacity diverge because this distribution has unbounded upper support.

1.1.3 Application contexts

The residual formulation connects several application areas. In lunar landing, uncertain dynamics must be reconciled with terminal and path requirements (Keil et al., 2021). In renewable dispatch, uncertain demand and generation enter balance and network constraints (Bienstock et al., 2014). Process planning couples uncertain inputs to production and operating limits (Calfa et al., 2015). Portfolio allocation regulates losses whose distribution can exhibit heavy tails (Cont, 2001; Rockafellar and Uryasev, 2000). Each setting requires a decision, a definition of violation, and a distribution against which reliability is assessed. Their disturbance laws and consequences of failure remain application-specific. Chapters 7 and 9 use synthetic static, dispatch, and lunar models to examine the computational mechanisms; process and portfolio applications supply broader motivation, not additional validated case studies.

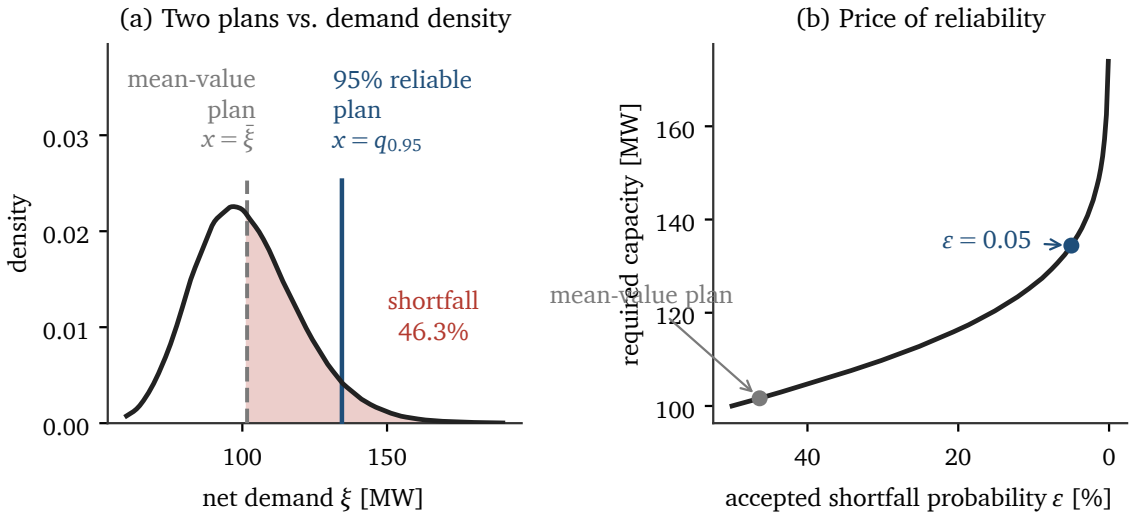


Figure 1: Seeded synthetic dispatch illustration. (a) The empirical mean-value capacity and 95% quantile against a KDE of the generated demand samples; shading marks shortfalls for the mean-value plan. (b) Empirical quantile capacity as a function of the accepted shortfall probability, with the mean-value plan located at its measured violation frequency. The curve shows capacity; multiplying by c gives cost.

1.2 Two Extremes and the Gap Between Them

Between ignoring uncertainty and protecting against every admissible disturbance lies a range of modeling choices. This section first characterizes worst-case robust optimization and the role of its uncertainty set, and then introduces chance constraints as a direct specification of the required reliability.

1.2.1 Ignoring uncertainty and worst-case robustness

Worst-case robust optimization enforces the constraint throughout a prescribed uncertainty set Ξ:

$$g(x, \xi) \leq 0 \quad \text{for all } \xi \in \Xi. \tag{3}$$

This is appropriate when the set captures the disturbances that must be tolerated and failure within it is unacceptable. Its protection is relative to the chosen set: a bounded set can yield a tractable robust problem even when the underlying probability law has unbounded support (Ben-Tal et al., 2009). Enforcing the full support may instead be infeasible, as in finite-capacity dispatch with unbounded demand. A very broad set can also allocate resources to extreme configurations that have little probability. The central modeling decision is therefore which uncertainty and consequences the specification must cover, rather than whether robustness is intrinsically too conservative.

1.2.2 Chance constraints as a reliability specification

A chance constraint requires

$$\mathbb{P}(g(x,\xi) \leq 0) \geq 1-\varepsilon, \qquad \varepsilon \in (0,1), \tag{4}$$

where ε is the risk level, the allowed violation probability. Smaller values yield nested, more restrictive feasible sets and cannot improve the optimal objective in a minimization problem with otherwise unchanged data. Unlike an expected penalty, the constraint controls failure frequency directly; it does not control the magnitude of a violation once one occurs. That distinction matters when choosing between probability constraints and tail-risk measures (Shapiro et al., 2014; Rockafellar and Uryasev, 2000).

The limiting zero-risk condition is almost-sure feasibility. It implies protection at every support point when the residual is continuous in the uncertainty; bounded support and further feasibility and regularity conditions are needed to obtain a finite robust limit of the decisions. For the unbounded dispatch example, no finite capacity achieves zero risk. Figure 2 is therefore a schematic comparison, with its high-reliability endpoint representing a selected bounded uncertainty set. The target setting of this thesis is a specified, nonzero risk budget under data-described uncertainty, potentially outside a convenient parametric family.

1.3 What Chance Constraints Are and Why They Are Hard

This section makes the chance constraint precise and explains why it is difficult to handle numerically. It first defines individual and joint chance constraints and the corresponding optimization problem, and then identifies three structural sources of difficulty.

1.3.1 Individual and joint chance constraints

For a measurable residual g, an individual chance constraint has the form

$$\mathbb{P}(g(x,\xi) \leq 0) \geq 1-\varepsilon. \tag{5}$$

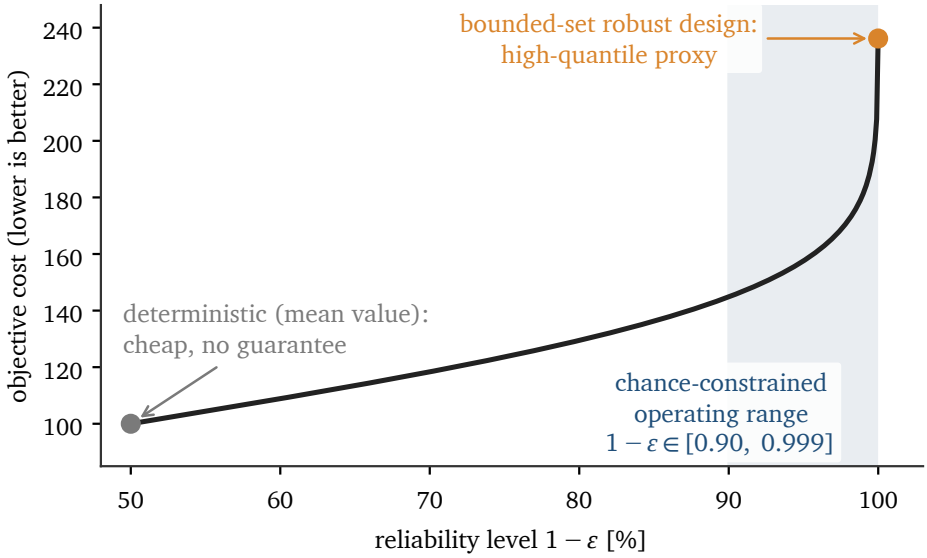


Figure 2: Schematic cost–reliability comparison. The curve is generated from Gaussian quantiles; its marked high-reliability endpoint is a finite-quantile proxy for protection over a selected bounded uncertainty set, not protection over the full Gaussian support. The shaded band illustrates possible chance-constraint targets.

For m simultaneous requirements, the joint form is

$$\mathbb{P}(g_r(x,\xi) \le 0,\ r = 1,\dots,m) \ge 1-\varepsilon. \tag{6}$$

Separate componentwise constraints with budget ε do not generally deliver the same joint reliability. A sufficient construction assigns budgets ε_r satisfying $\sum_{r=1}^{m} \varepsilon_r \le \varepsilon$ and enforces each component at level $1-\varepsilon_r$. Boole's inequality establishes this allocation without an independence assumption, but overlapping violation events can make it conservative (Prékopa, 1995). The individual chance-constrained program is

$$\underset{x\in X}{\text{minimize}}\, f(x) \quad \text{subject to } \mathbb{P}(g(x,\xi) \le 0) \ge 1-\varepsilon. \tag{7}$$

1.3.2 Three structural sources of difficulty

Three properties of chance-constrained programs complicate their numerical treatment; they are discussed in turn.

Nonconvex feasible geometry. Even affine scenario constraints can produce a nonconvex probabilistic feasible set. With two equally likely scenarios and $\varepsilon = 0.5$, satisfying either scenario is sufficient, so the feasible set is the union of their half-spaces.

Such a union need not be convex. Convexity is available under additional structure, including suitable joint convexity of the deterministic feasible set and a log-concave uncertainty law (Prékopa, 1995). A general nonlinear residual need not satisfy these conditions.

Probability evaluation over a moving region. Define $S(x) = \{\xi : g(x, \xi) \leq 0\}$ and the safe probability $p(x) = \mathbb{P}(\xi \in S(x))$. If the uncertainty admits a density ρ_ξ, then

$$p(x) = \int_{S(x)} \rho_\xi(u)\, du. \tag{8}$$

The region changes with the decision, and the density may itself be unknown. Evaluating this integral repeatedly can be demanding even when the density is available, especially in high dimension. When only samples are available, integration and distribution estimation become coupled parts of the optimization problem (Shapiro et al., 2014).

Discontinuous empirical indicators. Given N independent, identically distributed (IID) samples $\xi_1, \ldots, \xi_N$, the direct sample estimate is

$$\bar{p}_N(x) = \frac{1}{N} \sum_{j=1}^{N} \mathbf{1}\{g(x, \xi_j) \leq 0\}. \tag{9}$$

For a fixed sample, this estimate is constant wherever no residual crosses zero and jumps at crossings. The underlying probability can be smooth even though its empirical approximation is discontinuous. Differentiating the indicator average therefore supplies no useful local information between crossings to a gradient-based solver. Smoothing addresses this computational obstacle, but it also changes the estimated feasible boundary. Figure 3 illustrates the first and third difficulties; the residual transformation below addresses the evaluation structure of the second.

1.4 Distributional Uncertainty and the Residual-Space Shift

Even when the chance constraint is well defined, the probability law against which it is evaluated is often known only through data. This section first explains why a fitted parametric law can misrepresent the relevant tail, then introduces the residual-space

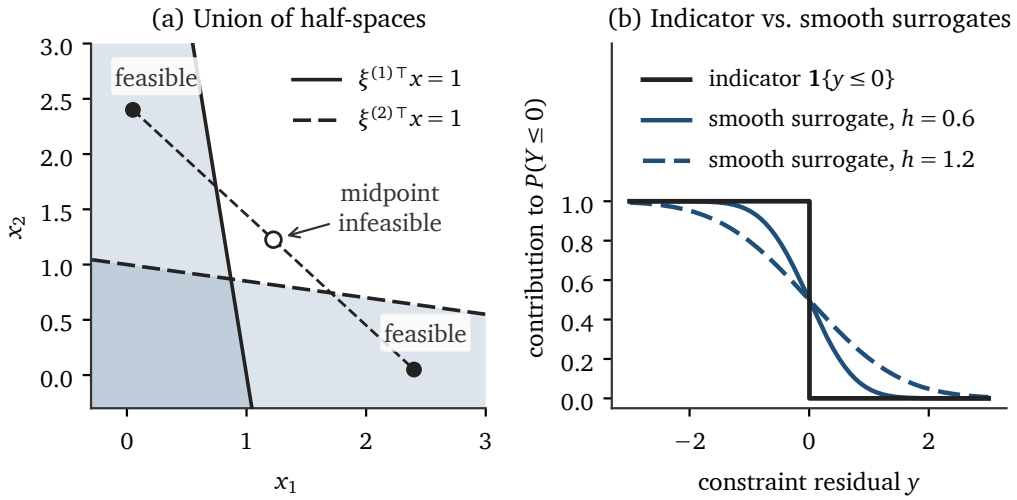


Figure 3: Two structural obstacles. (a) A union of scenario half-spaces gives a nonconvex feasible set. (b) A sample indicator and smooth approximations illustrate the transition introduced by smoothing. Smoothing an indicator does not generally convexify the optimization problem.

estimate on which the thesis builds, and finally names the open questions concerning bandwidth, one-sided bias, and joint events that the later chapters address.

1.4.1 Why a parametric law can fail

A parametric law can provide an efficient and interpretable probability model when its assumptions are supported. Reliability calculations, however, depend on the probability mass near and beyond the active threshold. Matching a mean and variance does not establish an accurate tail: skewness, heavy tails, and multiple modes can shift that mass substantially. Misspecification can either understate risk or impose unnecessary conservatism.

Figure 4 isolates this mechanism using synthetic laws. In its centered Gamma example, the generator's fitted Gaussian assigns approximately 0.23% probability above the marked threshold, while approximately 1.73% of the generated sample lies there. These values demonstrate a particular mismatch, not a universal error factor or an empirical claim about forecast data. KDE reduces commitment to a named distributional family, but finite samples and bandwidth still govern the quality of its tail estimate (Silverman, 1986).

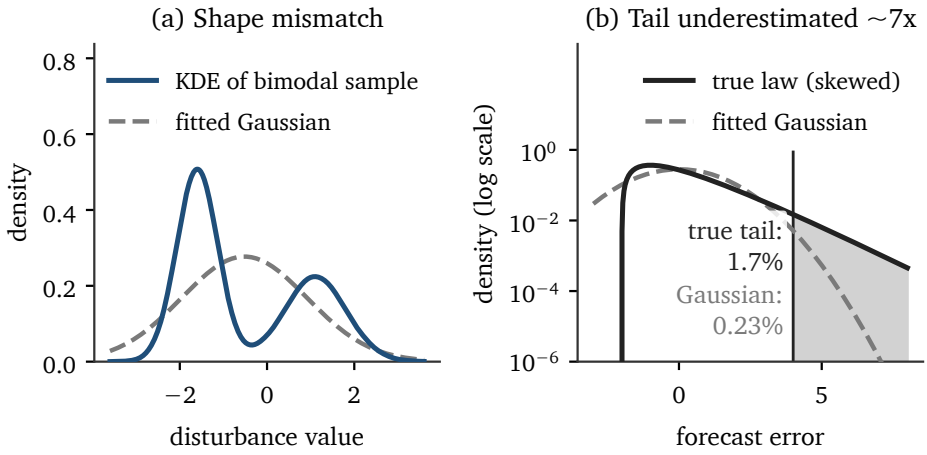


Figure 4: Synthetic illustrations of distributional misspecification. (a) A KDE of a generated bimodal sample and its moment-matched Gaussian fit. (b) A centered Gamma density and a Gaussian fit to 400,000 generated samples; the annotations compare empirical Gamma and fitted-Gaussian tail probabilities above the marked threshold. Panel (b) uses random seed 3.

1.4.2 Estimating residuals rather than uncertainty coordinates

For a fixed decision x, define $Y(x) = g(x, \xi)$ and its cumulative distribution function $F_{Y(x)}$. The identity

$$F_{Y(x)}(0) = \mathbb{P}(Y(x) \leq 0) = \mathbb{P}(g(x, \xi) \leq 0) = p(x) \tag{10}$$

expresses the original safety event without approximation. Suppose $Y(x)$ admits a density. For residual samples $y_j(x) = g(x, \xi_j)$, bandwidth $h > 0$, and a nonnegative kernel K integrating to one, its KDE and integrated safe probability are

$$\hat{\rho}_h(y; x) = \frac{1}{Nh} \sum_{j=1}^{N} K\left(\frac{y - y_j(x)}{h}\right), \qquad \hat{p}_h(x) = \int_{-\infty}^{0} \hat{\rho}_h(y; x)\, dy. \tag{11}$$

This is the residual-based construction developed in kernel-smoothed chance-constrained optimization (Calfa et al., 2015; Schuster, 2025). Figure 5 illustrates it at a fixed decision.

Three benefits follow. First, the estimated variable is scalar for an individual constraint, regardless of the dimension of ξ. Second, the integration region is the fixed half-line $(-\infty, 0]$; decision dependence enters through the residual samples. Third, integration yields a finite sum of kernel CDF evaluations that is differentiable in x when the integrated kernel and residual maps have the required smoothness. Chapter 4 derives that sum and its gradient. These benefits concern probability evaluation and solver access;

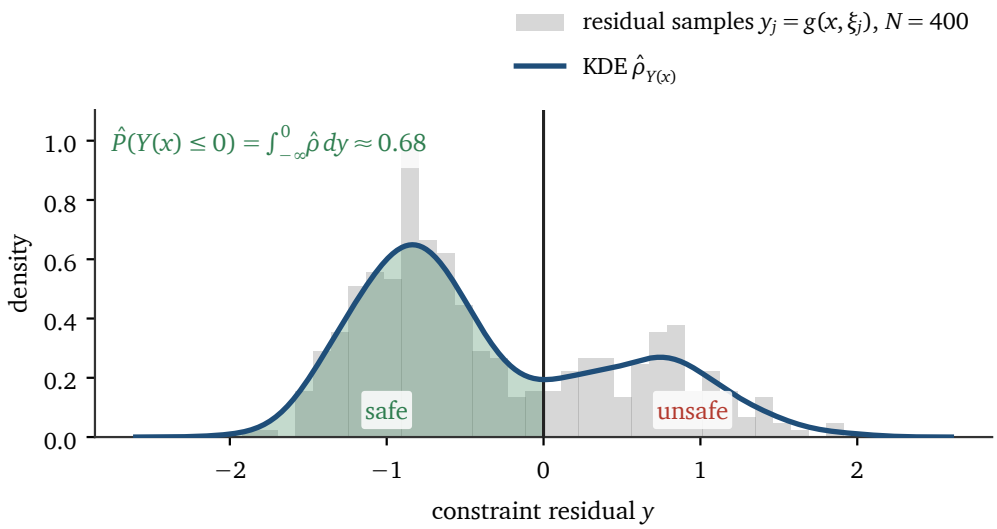


Figure 5: Residual-space estimation at a fixed decision, illustrated with 400 synthetic residual samples. A KDE smooths the histogram, and its mass on the shaded half-line estimates the safe probability. The residual samples change when the decision changes.

they neither eliminate rare-event sampling requirements nor ensure global convexity.

1.4.3 Bandwidth, one-sided bias, and joint events

Bandwidth determines both statistical smoothing and the constraint geometry presented to the optimizer. An ordinary symmetric KDE can overestimate the safe probability at finite sample size. A suitably one-sided integrated kernel instead bounds the empirical indicator in the conservative direction. This is a pointwise statement at a fixed decision; transferring it to the unknown probability after selecting a decision from the same data requires additional statistical control or independent validation.

For a joint event, $\max_r g_r(x, \xi) \le 0$ is exactly equivalent to satisfying every component. The maximum preserves a scalar statistic but can be nonsmooth in the decision. Chapter 5 studies log-sum-exp smoothing alongside alternative joint constructions, making the additional approximation explicit.

1.5 Scope and Positioning

The final section of this chapter places the thesis relative to its anchor literature and fixes its scope. It then states the four research questions and the chapter map that organize the remainder of the work.

1.5.1 Positioning relative to anchor work

The thesis connects smooth residual reformulation (Calfa et al., 2015), safety-oriented biased-kernel constructions (Keil et al., 2021), and optimizer-level convergence under explicit assumptions (Schuster, 2025). It develops the formulation in Chapter 4, analyzes the local one-sided and joint constructions in Chapter 5, and interprets the convergence conditions in Chapter 6. Chapters 7–9 connect these statements to an implementation and a reproducible validation protocol.

The contribution is an applied optimization analysis and implementation, not a new theorem in KDE theory. The local biased KDE, a shifted Epanechnikov construction, is distinguished from the Split–Bernstein kernel of Keil et al. and optimal-control workflow. No closed-loop stability or recursive-feasibility result is established, and the synthetic lunar transcription does not by itself certify continuous-time path safety. Statistical statements concern the sampling law; transfer to a changed deployment distribution is outside scope. In particular, asymptotic convergence and empirical test frequencies are not distribution-free finite-sample guarantees for a decision optimized on the training sample.

1.5.2 Research questions and chapter map

Four research questions organize the analysis:

1. **RQ1: Formulation.** How does residual-space KDE produce a differentiable chance-constraint approximation, and which assumptions justify its estimator, conservative bounds, and optimization limits? Chapters 4–6 provide the mathematical analysis.

2. **RQ2: Numerical comparison.** How do nominal, scenario, parametric-reference, and KDE decisions compare in violation probability, objective value, conservatism, and solver status where the benchmark supports those measurements? Chapters 8–9 define and report the comparisons.

3. **RQ3: Bandwidth and smoothing.** How do bandwidth, one-sided bias, and sample size affect estimated feasibility, held-out violations, and solver behavior? Chapters 3–6 establish the mechanisms, and Chapter 9 reports the diagnostics.

4. **RQ4: Joint extension.** How do scalar joint-event statistics and log-sum-exp smoothing compare with alternative constructions, and what conservatism or numerical limitations do they introduce? Chapter 5 develops the extension, and Chapter 9 reports a statistic-level diagnostic of it.

Chapter 2 supplies the stochastic-optimization survey; Chapter 3 supplies the KDE foundations. Chapter 7 specifies the optimal-control transcription and its audit; Chapters 10–11 interpret the evidence, identify unresolved limitations, and state the supported conclusions.

2 A Survey on Stochastic Optimization and Chance Constraints

Chapter 1 establishes the reliability problem and the residual-space shift. This chapter surveys the optimization paradigms and chance-constraint formulations needed to place the thesis method in context. It first distinguishes deterministic, robust, stochastic, and chance-constrained models, then reviews classical analytical reformulations and the principal sample-based and risk-based approximations. Kernel density estimation and its application literature are developed in Chapter 3, where the survey becomes the residual-space construction used by this thesis.

2.1 Optimization Paradigms Under Uncertainty

Every model under uncertainty makes two choices: what information about the random input it uses and what protection its solution provides. These choices distinguish four related paradigms.

2.1.1 Deterministic, robust, stochastic, and chance-constrained models

Deterministic optimization replaces the random input by a nominal value or forecast. It is computationally attractive but does not state a probability of failure. Robust optimization instead specifies an uncertainty set Ξ and requires

$$g(x, \xi) \leq 0 \quad \text{for every } \xi \in \Xi, \tag{12}$$

which gives set-wise protection and may admit tractable dual reformulations (Soyster, 1973; Ben-Tal et al., 2009). The protection depends entirely on the chosen set; a broad set can pay for physically implausible points, while an unbounded set can make the problem infeasible.

Stochastic programming consumes a probability law and optimizes expected cost. In a two-stage model, a here-and-now decision x is followed by a recourse action $y(\xi)$ after the uncertainty is observed, giving an objective of the form $f(x)+\mathbb{E}[Q(x, \xi)]$ (Dantzig, 1955; Birge and Louveaux, 2011). This is appropriate when an adverse outcome is a cost that can be repaired. It does not, by itself, bound the frequency of an individual failure.

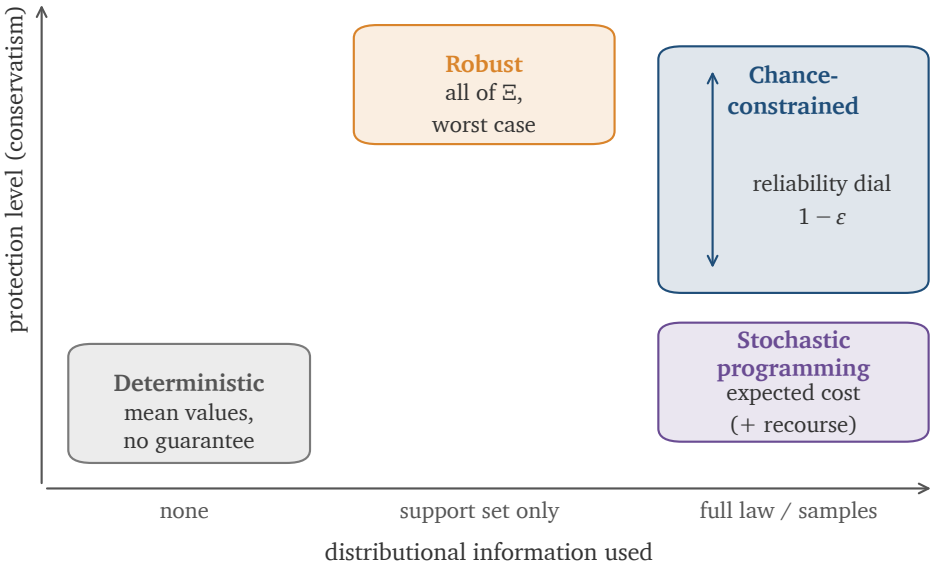


Figure 6: Four paradigms of optimization under uncertainty arranged by the distributional information used and the protection supplied. Deterministic optimization uses a point estimate; robust optimization protects a prescribed set; stochastic programming minimizes expected cost, possibly with recourse; chance-constrained programming provides a continuously chosen reliability level $1-\varepsilon$.

Chance-constrained programming uses the distribution, or samples from it, to regulate the frequency of a failure event:

$$\mathbb{P}(g(x,\xi)\leq 0)\geq 1-\varepsilon. \tag{13}$$

The risk level ε is chosen by the decision maker. Chance constraints are therefore appropriate when violations are discrete events whose frequency is regulated, whereas stochastic programming is appropriate when their magnitude is priced in the objective. The paradigms are complementary formulations of the same application rather than a universal ranking. Figure 6 summarizes their positions according to distributional information and protection level.

2.1.2 Model uncertainty and distributional robustness

The preceding distinction concerns what a model requires under a given law or uncertainty set. A separate question is whether the law itself is known accurately. Distributionally robust chance constraints require the probability bound to hold for every distribution in an ambiguity set. Depending on how this set is constructed, the formulation can incorporate moment information, a statistical distance from an empirical

law, or another data-derived ambiguity description (Calfa et al., 2015; Chen et al., 2024). The additional protection concerns uncertainty about the distribution, rather than a different definition of the failure event.

This distinction is relevant to the interpretation of KDE. Estimating a flexible density removes a fixed parametric shape restriction, but it does not eliminate estimation error or distribution shift. Throughout this chapter, a method described as nonparametric does not require a named distributional family; any reliability guarantee still depends on its sampling and regularity assumptions. The thesis studies estimators and decisions under declared sampling laws. It does not construct a distributional ambiguity set for deployment uncertainty.

2.1.3 Individual and joint chance constraints

Let $x \in X \subseteq \mathbb{R}^n$, let $\xi \in \mathbb{R}^d$ have law $\mathbb{P}$, and let g_r be measurable residual functions. An individual chance constraint controls one requirement. A joint chance constraint controls all m requirements simultaneously:

$$\mathbb{P}(g_r(x,\xi) \leq 0,\ r = 1,\dots,m) \geq 1-\varepsilon. \tag{14}$$

The violation form is equivalent for an individual constraint, $\mathbb{P}(g(x,\xi) > 0) \leq \varepsilon$. As noted in Chapter 1, enforcing componentwise constraints with budgets summing to ε implies (14) by Boole's inequality but can be conservative when violation events overlap (Prékopa, 1995). The maximum residual $\max_r g_r(x,\xi)$ gives an exact scalar representation of the joint event; its nonsmoothness motivates the extension in Chapter 5.

2.2 Classical Analytical Reformulations

Before turning to approximations, this section reviews the settings in which a chance constraint admits an exact deterministic treatment. It traces the formulation from its origins to the canonical affine case, identifies log-concavity as the main source of convexity together with its limits, and closes with the general nonlinear case that motivates the approximation methods of Section 2.3.

2.2.1 Canonical affine case and origins

Chance-constrained programming originated in uncertain production planning (Charnes et al., 1958; Charnes and Cooper, 1959). Early deterministic equivalents and the extension from individual to joint requirements established the basic formulation (Charnes and Cooper, 1963; Miller and Wagner, 1965). For an affine residual $g(x,\xi) = a(x)^\top \xi - c(x)$, the constraint is a quantile condition:

$$\mathbb{P}(a(x)^\top \xi \le c(x)) \ge \alpha, \qquad \alpha = 1-\varepsilon, \tag{15}$$

with violation probability $v(x) = \mathbb{P}(a(x)^\top \xi > c(x))$. The remaining question is whether this probability can be evaluated or reformulated without imposing an unrealistic law.

2.2.2 Log-concavity and its boundary

Prékopa's theorem identifies an important region of convexity. If the law of ξ is log-concave and the set $\{(x,\xi) : g(x,\xi) \le 0\}$ is convex, then the safe probability is log-concave in x; its upper level sets are consequently convex (Prékopa, 1971, 1995, 2003). Gaussian laws and uniforms on convex sets are examples. Outside these assumptions, mixtures, multimodal disturbances, heavy-tailed data, or nonlinear dynamics can destroy convexity. Figure 7 compares two probability level sets for the bilinear constraint $\xi^\top x \le 1$. The Gaussian panel is convex by the Gaussian second-order-cone reformulation discussed below. It is not a direct application of the jointly convex-set version of Prékopa's theorem, because the residual is bilinear in (x,ξ). The mixture panel illustrates how changing the law can produce nonconvex geometry with the same residual function.

2.2.3 General nonlinear case

In the general case, evaluating $v(x)$ requires integrating an unknown density over a region that changes with x. Even when the density is known, the region may be curved, disconnected, and high-dimensional. The approximation methods below recover tractability by conceding distributional generality, exactness, smoothness, or computational size.

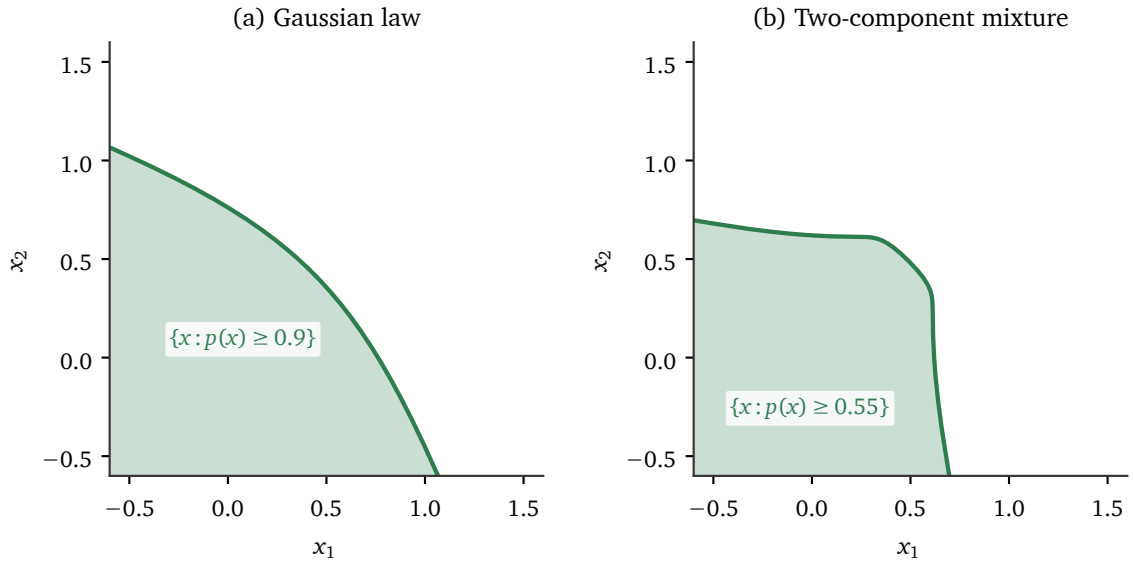


Figure 7: Convexity boundary for the bilinear constraint $\xi^\top x \leq 1$. (a) Under a Gaussian law the 0.9 probability level set is convex, as the Gaussian second-order-cone representation discussed below implies. (b) Changing only the law to a two-component Gaussian mixture makes the 0.55 level set nonconvex.

2.3 Approximation Methods and Their Trade-Offs

This section reviews the principal approximation methods in turn: parametric and Gaussian reformulations, scenario approach, sample-average approximation, CVaR restrictions, and smooth-quantile and black-box methods. A final comparison collects their assumptions, smoothness, conservatism, and guarantee types in one table.

2.3.1 Parametric and Gaussian reformulations

For $\xi \sim \mathcal{N}(\mu, \Sigma)$ and an affine constraint, the chance constraint has the deterministic form

$$c(x) - \mu^\top a(x) \geq \Phi^{-1}(1-\varepsilon)\,\|\Sigma^{1/2} a(x)\|_2. \tag{16}$$

When $a(x)$ and $c(x)$ are affine in x, Σ is positive semidefinite, and $\varepsilon \leq 1/2$, this is a convex second-order cone constraint (Charnes and Cooper, 1963; Shapiro et al., 2014). The probability reformulation is exact when the law is correct, and its conic representation is computationally efficient. The norm expression need not be differentiable at zero variance; conic solvers handle this structure directly. Its limitation is that the certificate inherits every distributional assumption: matching moments does not establish tail accuracy, and the formulation gives no internal warning when the true

law is skewed, heavy-tailed, or multimodal. Spherical-radial decomposition supplies related analytic gradients for Gaussian-like laws (van Ackooij and Henrion, 2014), but it relocates rather than removes the modeling assumption.

2.3.2 Scenario approach

The scenario approach draws independent samples $\xi_1, \ldots, \xi_N$ and enforces

$$g(x, \xi_i) \leq 0, \qquad i = 1, \ldots, N. \tag{17}$$

For convex programs, scenario theory gives a distribution-free finite-sample confidence statement over IID samples from the underlying law. It does not require a parametric law, but it does require convexity and support-rank conditions. Like robust optimization, scenario enforcement protects every sampled realization; its certificate is probabilistic over the sampling process rather than a claim that the uncertainty law is absent. A standard binomial bound has the form

$$\sum_{i=0}^{k-1} \binom{N}{i} \varepsilon^i (1-\varepsilon)^{N-i} \leq \delta, \tag{18}$$

where k is an appropriate support rank and δ is the allowed confidence failure probability (Calafiore and Campi, 2006; Campi and Garatti, 2008). The program contains one hard constraint per sample and therefore grows with N; it also protects every sampled point, which can be substantially more conservative than the permitted violation budget. The guarantee does not transfer automatically to the nonconvex nonlinear programs studied here.

Scenario enforcement and probability estimation should also be distinguished computationally. If each residual is smooth, each scenario constraint is smooth; the method's cost comes from the number of rows and the structure of the resulting program, rather than an indicator discontinuity. In the scalar dispatch example, enforcing every scenario gives $x_N = \max_i \xi_i$, whereas the empirical chance constraint permits a prescribed number of exceedances. This example explains possible scenario conservatism without asserting that every scenario formulation becomes more conservative in the same way or excludes sampling-and-discarding variants.

2.3.3 Sample average approximation (SAA)

SAA estimates the violation probability directly:

$$\hat{v}_N(x) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{g(x,\xi_i) > 0\} \leq \hat{\varepsilon}. \tag{19}$$

The estimator is transparent and, under standard conditions, SAA solutions have asymptotic consistency (Luedtke and Ahmed, 2008; Pagnoncelli et al., 2009). The indicator average is piecewise constant wherever no residual crosses zero, so direct differentiation gives no useful gradient between crossings. Binary variables can encode the count exactly, but this creates a mixed-integer formulation whose difficulty grows with sample size. SAA therefore trades the scenario approach's conservatism for a discontinuous or combinatorial optimization problem.

The surrogate budget $\hat{\varepsilon}$ need not equal the target ε. A smaller budget can compensate for sampling uncertainty, provided the adjustment is justified by a statistical argument. At a fixed decision, the count is an unbiased estimate of the violation probability. At the optimizer, the same count has also influenced selection of the decision, so fixed-decision unbiasedness is insufficient to certify the selected point. Independent validation, uniform bounds over the decision class, or an applicable scenario argument address different parts of this selection problem. This distinction remains necessary when smoothing replaces the indicator.

2.3.4 Conditional value-at-risk (CVaR) and convex restrictions

For loss $L(x,\xi) = g(x,\xi)$ and confidence level $\alpha \in (0,1)$, conditional value-at-risk (CVaR) has the representation

$$\mathrm{CVaR}_\alpha(L) = \min_{t\in\mathbb{R}} \left[t + \frac{1}{1-\alpha}\mathbb{E}(L-t)_+ \right]. \tag{20}$$

Because CVaR dominates the corresponding value-at-risk, imposing $\mathrm{CVaR}_{1-\varepsilon}(L) \leq 0$ is a sufficient, conservative restriction of the chance constraint (Rockafellar and Uryasev, 2000; Nemirovski and Shapiro, 2006). Convexity is preserved when L is convex in x and the expectation is replaced by a sample average. The positive-part function is piecewise linear in a scalar loss. For a finite sample, auxiliary variables replace its kinks by inequality constraints that inherit the regularity of the residual map. The population expectation can also be differentiable under appropriate continuity and

integrability conditions; convexity does not imply either differentiability or linearity in a nonlinear decision variable. CVaR also controls the magnitude of tail losses, so it can reject a decision with rare but large violations even when its violation frequency satisfies the chance constraint.

2.3.5 Smooth quantiles and black-box optimization

The chance constraint is equivalent, under the usual quantile convention, to

$$q_{1-\varepsilon}(g(x,\xi)) \leq 0. \tag{21}$$

The empirical quantile is an order statistic and is nonsmooth in x. Peña-Ordieres et al. (2020) construct a differentiable sample-based quantile approximation and a trust-region sequential quadratic programming (SQP) method. This is a close methodological comparator because it constructs a smooth constraint from the same residual samples. Its smoothed CDF and inverse-quantile construction are closely related to integrated-kernel probability estimation. The useful distinction is which constraint representation and numerical method are used, rather than a claim that KDE uniquely provides smooth data-driven chance constraints. The present implementation evaluates the residual CDF at the fixed threshold zero; it does not benchmark the full trust-region algorithm of that paper.

A black-box method evaluates the simulator, counts failures, and optimizes using only function values, for example with pattern search or other derivative-free optimization (DFO) methods (Nelder and Mead, 1965; Audet and Hare, 2017). Its universality is useful for discontinuous or legacy simulators, but evaluations grow rapidly with decision dimension. In a discretized trajectory problem, the hundreds of decision variables make this route difficult to use as a primary method. Figure 8 shows the underlying obstacle: the empirical violation estimate is a piecewise-constant empirical estimator, with flat regions and jumps at sampled residual crossings.

2.3.6 Comparison in one view

The principal alternatives partition four properties: distributional assumptions, smoothness, conservatism, and guarantee type. Here “smooth” means that the reformulated constraint supplied to the solver is differentiable; a statistical statement

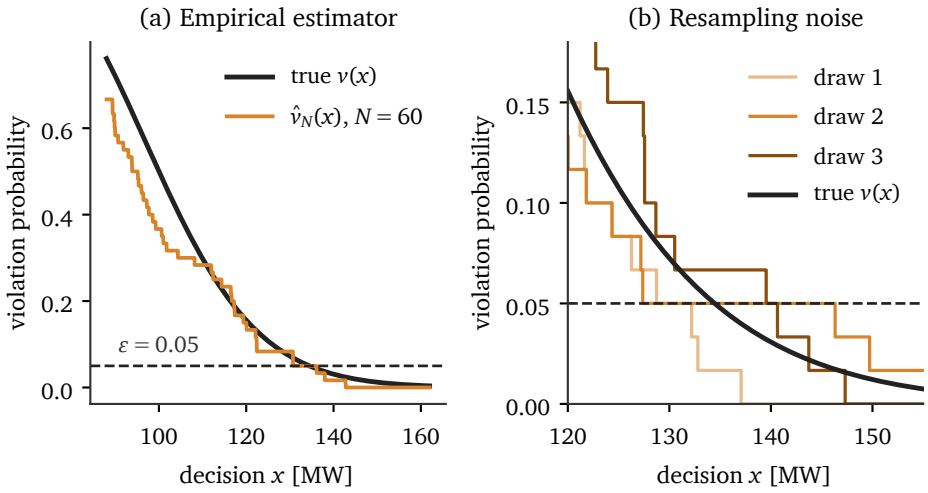


Figure 8: Empirical violation probability for the lognormal dispatch illustration. With $N = 60$ samples the estimator is piecewise constant, and independent draws disagree near the $\varepsilon = 0.05$ boundary. The figure is a seeded synthetic diagnostic of the indicator and resampling limitations.

Table 1: Approximation methods for chance constraints. Guarantee labels distinguish finite-sample statements, deterministic inner bounds, statistical estimator statements, and asymptotic results.

Method	Distribution-free	Smooth	Conservative	Guarantee
Gaussian reformulation	No	Yes	No if correct law	Model-based
Scenario approach	Yes	Yes if g smooth	Often	Finite-sample*
SAA indicator	Yes	No	Tunable	Asymptotic
CVaR restriction	Partial	Piecewise	Structural	Deterministic bound
Smooth quantile	Yes	Yes	Tunable	Statistical
Black-box DFO	Yes	No	No	None automatic
Unbiased KDE	Yes	Yes	No	Asymptotic
Biased KDE	Yes	Yes	By construction	Conditional

*For convex programs under the assumptions of scenario theory (Calafiore and Campi, 2006; Campi and Garatti, 2008).

is not automatically a safety certificate. Table 1 compares the methods along these properties.

KDE is therefore not a universal winner. It is attractive when the residual map is smooth, the law is empirical or non-Gaussian, and a single smooth probability constraint is more useful than many sampled constraints. Its price is bandwidth selection, finite-sample tail error, and the need to distinguish fixed-decision envelopes from guarantees for a data-selected optimizer.

3 Chance Constraints and Kernel Density Estimation

Chapter 2 surveys the optimization paradigms and approximation methods that motivate a nonparametric treatment of chance constraints. This chapter narrows that survey to kernel density estimation (KDE): it defines the residual-space estimator, explains bandwidth and dimensionality trade-offs, and maps the relevant chance-constraint literature to the gap addressed by the thesis. Chapter 4 then derives the smooth residual-KDE reformulation used in the optimization experiments.

3.1 Kernel Density Estimation for Residual Probabilities

This section introduces the kernel density estimator in the residual-space form used throughout the thesis. It defines the Parzen–Rosenblatt estimator and its integrated safe probability, compares symmetric and one-sided kernels, discusses bandwidth selection, and explains why scalar residuals avoid the curse of dimensionality that affects multivariate estimation.

3.1.1 Parzen–Rosenblatt estimator and residual CDF

Let $y_1(x), \dots, y_N(x)$ be IID residual samples and let ρ_x denote the density of $Y(x) = g(x, \xi)$, assumed to exist for the decisions under study. For a nonnegative kernel K with $\int K(u)\,du = 1$ and bandwidth $h > 0$, the KDE is

$$\hat{\rho}_h(y; x) = \frac{1}{Nh} \sum_{j=1}^{N} K\left(\frac{y - y_j(x)}{h}\right), \qquad \hat{p}_h(x) = \int_{-\infty}^{0} \hat{\rho}_h(y; x)\,dy. \tag{22}$$

This is the Parzen–Rosenblatt estimator (Rosenblatt, 1956; Parzen, 1962). If $F_K(t) = \int_{-\infty}^{t} K(u)\,du$, then $\hat{p}_h(x) = N^{-1} \sum_j F_K(-y_j(x)/h)$. Thus the probability is a finite sum of integrated-kernel evaluations rather than a numerical integral over uncertainty space. For IID samples and a regular residual density (for example, the usual smoothness and integrability conditions on ρ_x), pointwise consistency uses a bandwidth sequence h_N satisfying $h_N \to 0$ and $Nh_N \to \infty$ (Parzen, 1962; Devroye and Györfi, 1985). The sequence links smoothing to sample size: the normal-reference rule introduced below chooses $h_N = O(N^{-1/5})$. This is pointwise/statistical consistency for a fixed decision and does not by itself imply convergence of a data-selected optimizer;

optimizer convergence requires stronger uniform convergence and optimization regularity, as discussed in Section 3.2.2.

The uncertainty sample is drawn once and kept fixed during optimization, while $y_j(x) = g(x, \xi_j)$ changes at every iterate. This preserves a deterministic sample problem and permits differentiation through the residual map. With fixed h and continuous K, the integrated estimate is continuously differentiable wherever g is continuously differentiable; a continuously differentiable kernel supports a second derivative when the residual also has the required regularity. Thus a kernel need not itself be infinitely smooth to provide a differentiable probability constraint.

The statistical and computational roles of the estimator are different. One scalar chance constraint becomes one aggregated inequality, but its evaluation and gradient still require N residual contributions. Residual scalarization reduces the dimension of the density estimate; it does not make simulation cost independent of sample size. Chapter 4 makes these costs explicit when deriving the gradient and assembling the nonlinear program.

3.1.2 Kernel choices and one-sided bias

The Gaussian kernel is infinitely differentiable and has $F_K = \Phi$. The compactly supported Epanechnikov kernel, $K(u) = \frac{3}{4}(1-u^2)_+$, is efficient for classical density estimation (Epanechnikov, 1969). Both are symmetric: their integrated functions smooth the indicator from both sides and can overestimate the safe probability. The Split–Bernstein constructions of Keil et al. (2020, 2021) are designed for a one-sided safety property. The local implementation in this thesis uses a shifted Epanechnikov analogue and does not reproduce the published workflow of Split–Bernstein kernels, Markov chain Monte Carlo (MCMC) sampling, and optimal control. A pointwise kernel inequality for a fixed decision is therefore distinct from a finite-sample guarantee for the optimized decision.

For the safe-probability convention used here, a fully shifted compact kernel gives a simple illustration. With the Epanechnikov CDF F_K and shift $b(h) = h$, the per-sample contribution is

$$\psi_h(y) = F_K\left(\frac{-y-h}{h}\right) \leq \mathbf{1}\{y \leq 0\}. \tag{23}$$

Consequently, the smoothed safe fraction is bounded by the empirical safe fraction for every fixed sample. Taking expectations at a fixed decision gives a lower bound on the population safe probability. Neither statement removes random sampling error at a

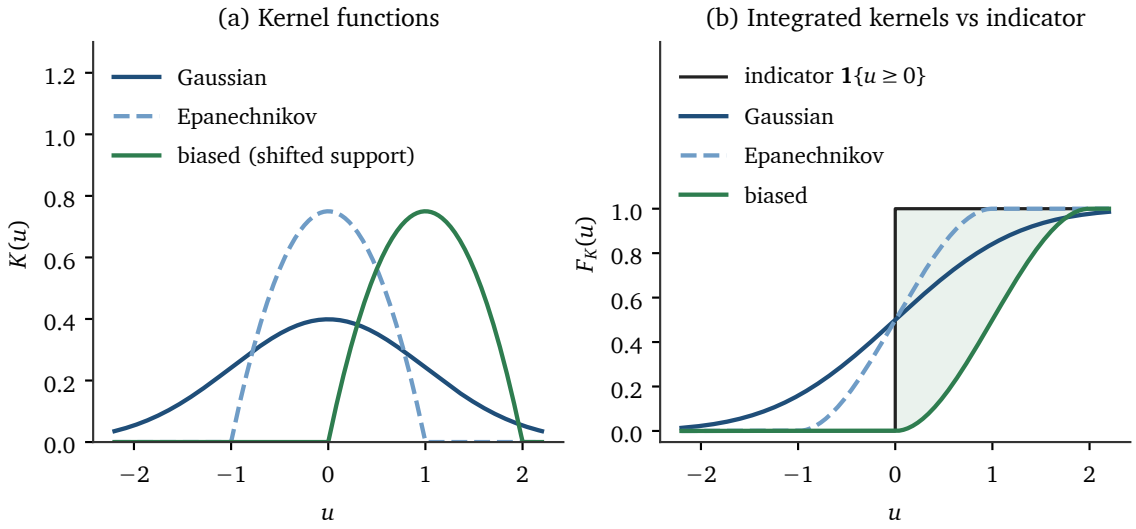


Figure 9: Kernel shapes and integrated kernels. The shifted Epanechnikov kernel has support $[0, 2]$ in the standardized argument, so its CDF is bounded above by $\mathbf{1}\{u \geq 0\}$. This is a per-sample smoothing inequality. It does not by itself give a finite-sample population guarantee for a data-selected decision.

decision selected using that sample. Figure 9 displays the underlying integrated-kernel inequality using a standardized argument.

3.1.3 Bandwidth and AMISE

Bandwidth controls the bias–variance trade-off. For a twice-differentiable density ρ_x, define $R(\phi) = \int \phi(u)^2\,du$ and $\mu_2(K) = \int u^2 K(u)\,du$. The standard one-dimensional expansion of the asymptotic mean integrated squared error (AMISE) is

$$\mathrm{AMISE}(h) = \frac{R(K)}{Nh} + \frac{h^4 \mu_2(K)^2}{4} R(\rho_x''), \tag{24}$$

with optimal order $h = O(N^{-1/5})$ and error $O(N^{-4/5})$ (Silverman, 1986). Silverman's and Scott's rules, plug-in estimates, and cross-validation estimate the unknown scale or curvature term (Silverman, 1986; Scott, 1992; Wand and Jones, 1995). They minimize a density-estimation criterion, however. In this thesis the KDE is thresholded at zero, differentiated through $g(x, \xi_j)$, and embedded in an optimizer; the bandwidth that minimizes AMISE need not minimize objective error, constraint error, or tail error. Figure 10 illustrates the density-level bias–variance trade-off used to motivate the optimization sensitivity study.

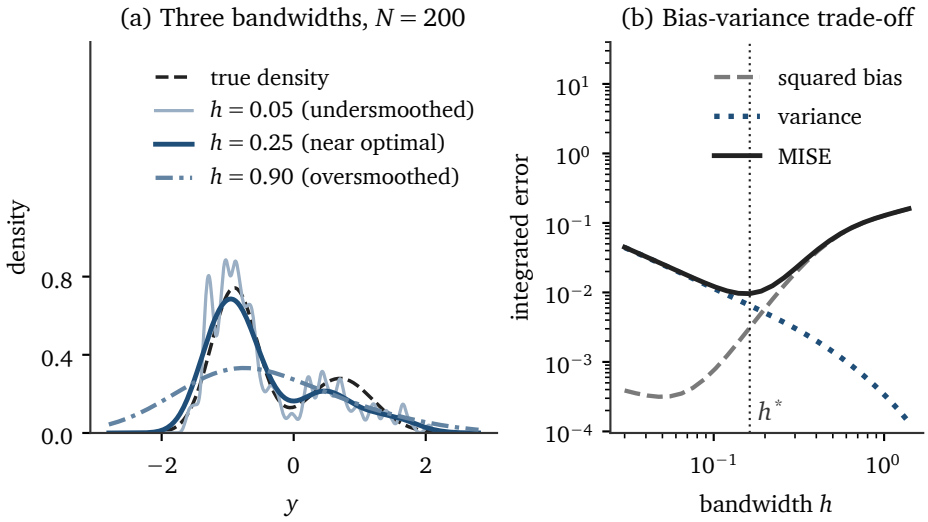


Figure 10: Bandwidth trade-off for a seeded bimodal density reconstruction. (a) Small bandwidth follows sample noise, an intermediate bandwidth recovers the two modes, and large bandwidth blurs them. (b) Across 120 replications, variance decreases and squared bias increases with bandwidth; their sum has an interior minimum. The plot concerns density error, so its minimizer need not be optimal for a chance-constrained decision.

In practice, the bandwidth is selected jointly with the sample size rather than held fixed across the asymptotic sequence. For example, the robust normal-reference rule commonly attributed to Silverman is

$$h_N = 0.9 \min\{\hat{\sigma}, \mathrm{IQR}/1.34\} N^{-1/5}, \tag{25}$$

where $\hat{\sigma}$ and the interquartile range measure sample scale (Silverman, 1986). A normal-reference constant is an initialization choice, not evidence that a multimodal or heavy-tailed residual is Gaussian. Cross-validation and plug-in procedures use different criteria and can select different bandwidths. If the bandwidth is recomputed as the decision changes, its dependence must also be accounted for in the derivatives or explicitly frozen; the formula in Chapter 4 assumes a fixed bandwidth during that differentiation.

Tail constraints require additional care even in one dimension. A sample contains on average $N\varepsilon$ observations in an event of probability ε, so a small violation budget can be weakly resolved despite a visually smooth density. Density-integrated error also weights regions differently from a constraint evaluated at one threshold. The experiments therefore report bandwidth, sample size, and held-out event frequency together instead of interpreting a favorable density plot as a reliability certificate.

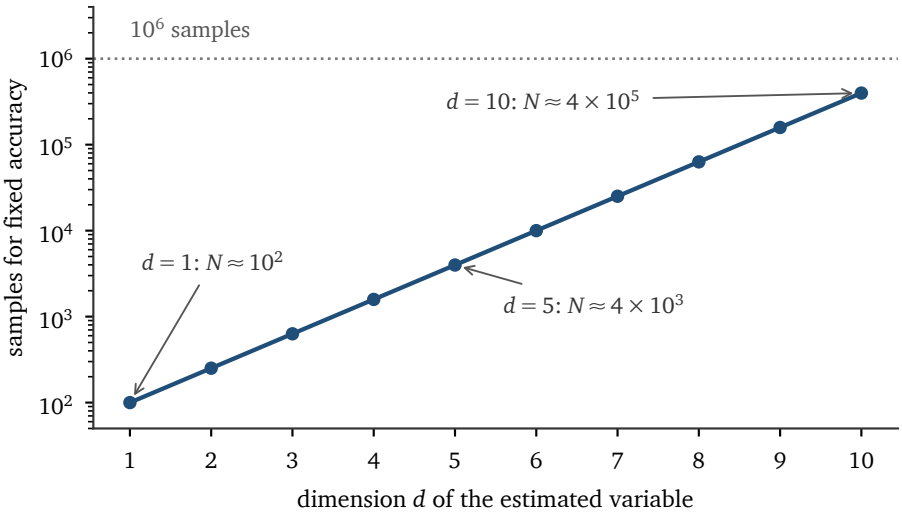


Figure 11: Rate-based illustration of the curse of dimensionality. The plotted sample count is the order required to retain the accuracy of $N = 100$ samples in one dimension under the AMISE rate $O(N^{-4/(d+4)})$. The annotations are approximate scaling values, not empirical requirements for a particular application.

3.1.4 Multivariate KDE and residual scalarization

For a d-dimensional variable, a bandwidth matrix H gives

$$\hat{f}_H(y) = \frac{1}{N} \sum_{j=1}^{N} |H|^{-1/2} K\big(H^{-1/2}(y - y_j)\big).$$

The optimal error rate becomes $O(N^{-4/(d+4)})$, with bandwidth order $O(N^{-1/(d+4)})$ (Silverman, 1986; Scott, 1992). Holding accuracy fixed therefore requires rapidly increasing sample size. Figure 11 plots the rate-based sample requirement normalized to $N = 100$ in one dimension; its values are illustrations from the rate, not measured minimum sample sizes.

The residual-space construction avoids estimating the full d-dimensional uncertainty law for an individual constraint. It estimates the scalar $Y(x) = g(x, \xi)$ and integrates over the fixed half-line $(-\infty, 0]$. This removes the uncertainty dimension from the KDE itself, but not from sampling or simulation. A joint constraint has a residual vector of dimension m; a scalar maximum preserves one-dimensional estimation but introduces a nonsmooth maximum, while multivariate residual KDE reintroduces the dimensionality cost. Chapter 5 compares these alternatives.

3.2 KDE in the Chance-Constrained Literature

The second part of the chapter maps how kernel-smoothed chance constraints have been used in the literature. It first reviews the application streams in process systems, aerospace, and energy, and then turns to the safety-oriented kernel constructions and optimizer-level convergence results that anchor this thesis.

3.2.1 Application streams

An early practical use of kernel-smoothed chance constraints replaced a specified process-planning distribution with historical data (Calfa et al., 2015). The same residual-probability construction was subsequently used in aerospace optimal control for launcher ascent and the Goddard problem (Caillau et al., 2018). Energy-system work developed adaptive KDE formulations for microgrids and extended them to dispatch and power-system scheduling with multivariate or joint constraints (Ciftci, 2017; Ciftci et al., 2020; Wu et al., 2021; Wu and Kargarian, 2023). These streams establish practical breadth, but their guarantees are primarily estimator-level.

The application papers do not all estimate the same object or provide the same guarantee. Calfa et al. treat individual and joint constraints with right-hand-side uncertainty using kernel-smoothed univariate quantiles and multivariate distribution functions, together with a divergence-based treatment of distributional uncertainty (Calfa et al., 2015). The optimal-control line uses kernel probability approximations inside trajectory optimization, where derivative availability and transcription accuracy become central (Caillau et al., 2018; Keil et al., 2021). The energy and network papers demonstrate settings in which empirical disturbance distributions and coupled constraints are relevant. Their results motivate the model adapters in this thesis, but are not reproduced merely by applying a scalar estimator to a synthetic residual.

3.2.2 Safety-oriented kernels and convergence theory

Keil et al. introduced biased kernels whose integrated functions are constructed to lie on the conservative side of the violation indicator and applied them to chance-constrained soft lunar landing (Keil et al., 2020, 2021). A warm-start continuation strategy reduces the numerical cost of changing bandwidth and kernel bias (Keil and Rao, 2021). Network applications and analytic-gradient treatments further demonstrate that the construction is not limited to one domain (Gugat and Schuster, 2018;

Schuster et al., 2022). Adjacent work studies nonparametric prediction intervals (Wan et al., 2020), chance-constrained gas–power flow (Wang et al., 2021), distributionally robust load shedding (Qi et al., 2026), portfolio tail risk (Liu et al., 2022), and kernel embeddings in robotics (Gopalakrishnan et al., 2018).

KDE consistency alone concerns the estimated probability for a fixed decision. The optimizer-level results of Schuster (2025) establish convergence of optimal values and solution sets under explicit assumptions on the residual density, kernel, bandwidth sequence, uniform convergence, and the optimization problem. They are asymptotic results, not finite-sample confidence bounds. The unresolved practical boundary addressed here is the connection between these assumptions and nonlinear numerical programs: how bandwidth choices behave inside an optimizer, how a one-sided local surrogate affects cost and held-out violations, and how joint-event reductions change the measured risk. Chapters 4–6 develop the formulation and its qualifications; Chapters 7–9 test the resulting evidence chain on declared synthetic models. Figure 12 arranges these streams on a timeline.

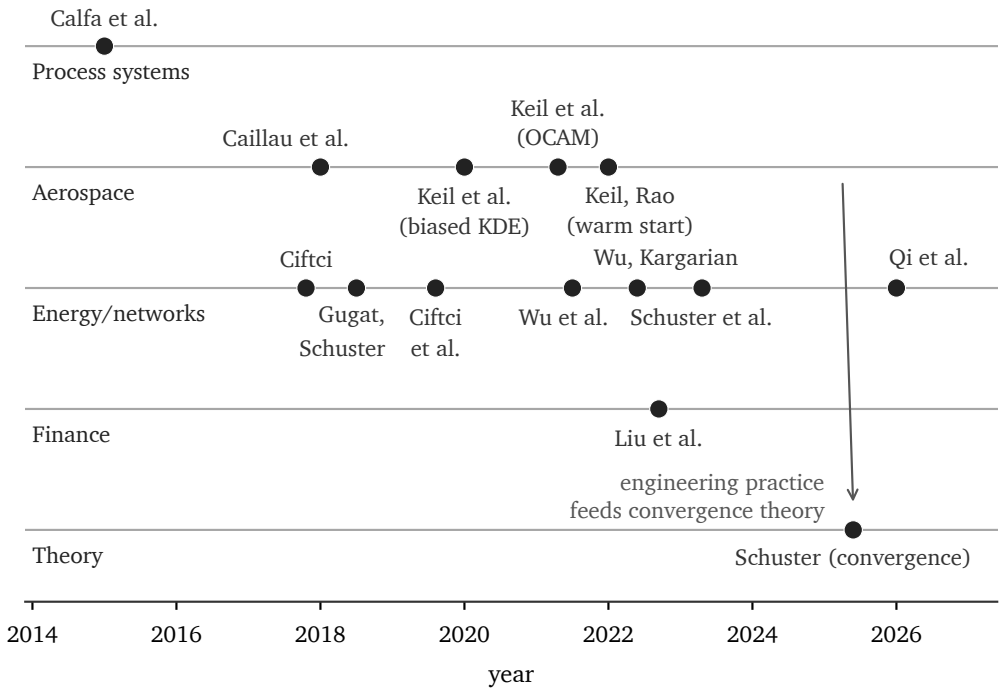


Figure 12: Timeline of kernel-smoothed chance-constrained optimization, grouped by stream: process systems, aerospace, energy and networks, finance, and convergence theory. The thesis studies the connection between these streams under explicit finite-sample and implementation qualifications.

4 Smooth Approximation with Ordinary Symmetric KDE

Chapters 2 and 3 identify what a useful approximation must provide: a single deterministic constraint that approximates the safe probability, depends smoothly on the decision, and carries an analytic gradient. This chapter constructs that object. The construction has five steps, and the order matters because each step is what makes the next one possible: the change of variables that moves estimation into residual space (Section 4.1), the residual samples that the change of variables produces (Section 4.2), the kernel density estimate built on them (Section 4.3), the integrated-kernel identity that collapses the probability into a finite smooth sum (Section 4.4), and the analytic gradient that the identity makes available (Section 4.5). With the object built, Section 4.6 assembles the full reformulated nonlinear program, and Section 4.7 states its properties and limitations, including the one property it lacks, which Chapter 5 repairs. Throughout the thesis, this symmetric, unshifted estimator is called the *unbiased* KDE, in contrast to the biased kernels of Chapter 5. The name refers to the absence of a deliberate safety shift ($b(h) = 0$), not to statistical unbiasedness: the estimator retains the smoothing bias discussed in Chapter 3.

4.1 The Residual-Space Transformation

The first step of the construction moves probability estimation from the uncertainty space into the space of constraint residuals. This section states the identity that makes this move exact and then explains the dimension reduction it achieves.

4.1.1 Safe-event identity

The fundamental insight behind the KDE approach, previewed in Chapter 1 and developed here, is a change of the space in which probabilities are estimated. The question the chance constraint literally poses is geometric: what is the probability that ξ falls in the safe region $S(x) = \{\xi : g(x, \xi) \leq 0\}$, a subset of $\mathbb{R}^d$ whose shape deforms with every candidate decision? The question this thesis poses instead is scalar: what is the probability that the constraint residual

$$Y(x) \;=\; g(x, \xi), \tag{26}$$

is nonpositive? The two questions have the same answer, identically and without approximation,

$$\mathbb{P}\big(Y(x) \le 0\big) \;=\; \mathbb{P}\big(g(x,\xi) \le 0\big) \;\ge\; 1-\varepsilon, \tag{27}$$

because the event $\{Y(x) \le 0\}$ *is* the event $\{\xi \in S(x)\}$, written in the variable the constraint actually speaks about.

4.1.2 Dimension reduction and its scope

What changes is not the answer but the tractability of computing it, and the change is substantial rather than cosmetic. The original question demands the integral of an unknown d-dimensional density over a moving region, the second of the three difficulties cataloged in Chapter 1, with the curse of dimensionality of Chapter 3 attached to any attempt to estimate that density nonparametrically. The transformed question asks for a one-dimensional cumulative distribution function evaluated at a fixed point, zero. The dimension of ξ no longer enters the estimation problem: whether the uncertainty space has two dimensions or two hundred, the residual of an individual constraint is a scalar, and the constraint map g itself has performed the dimension reduction, without information loss, because Eq. (27) is an identity. This is the construction Schuster (2025) places at the foundation of the convergence theory, and it is the reason the KDE machinery of Chapter 3 can be deployed in exactly the one-dimensional regime where it is strongest.

4.2 Residual Samples

For a fixed decision x and the samples $\xi_1, \ldots, \xi_N$ of Chapter 3, define the residual samples

$$y_j(x) \;=\; g(x,\xi_j), \qquad j = 1,\ldots,N. \tag{28}$$

These are realizations of the random variable $Y(x)$, obtained by pushing each uncertainty sample through the constraint map at the current decision, and they are the raw material of everything that follows.

One property of Eq. (28) must be understood precisely, because it is the property that distinguishes this construction from ordinary density estimation and shapes the analysis of the entire thesis: the residual samples are functions of the decision variable. The uncertainty samples ξ_j are drawn once and frozen; the residual cloud $\{y_j(x)\}$ moves whenever the optimizer moves x. Estimation and optimization are therefore

not sequential stages but a single coupled process. The estimator is rebuilt, implicitly, at every iterate, and the constraint the solver sees is not a fixed statistical object but a family of them indexed by x. This coupling is what makes the smoothness of the next two sections consequential rather than decorative: a smooth estimator of a fixed density would be a statistical convenience, while a smooth estimator of a moving one is a differentiable constraint.

4.3 KDE of the Residual Distribution

Assume that the pushforward variable $Y(x) = g(x, \xi)$ admits a density for the decisions under study. The Parzen–Rosenblatt estimator of Chapter 3, applied to the residual samples, estimates that density as

$$\hat{\rho}_h(y; x) \;=\; \frac{1}{Nh} \sum_{j=1}^{N} K\!\left(\frac{y - y_j(x)}{h} \right), \tag{29}$$

with kernel K and bandwidth $h > 0$ carrying the meanings and the selection theory established there. The estimated safe probability is the mass this density places on the safe half-line,

$$\hat{p}_h(x) \;=\; \int_{-\infty}^{0} \hat{\rho}_h(y; x)\, dy. \tag{30}$$

Two features of Eq. (30) are the direct payoff of the residual-space transformation. The integration region is fixed: it is $(-\infty, 0]$ for every decision, every problem, and every distribution, because all dependence on x has migrated into the samples. Moreover, the integrand is a known finite sum of kernel bumps, not an unknown density, so the integral is not a numerical task but an algebraic one, which the next section carries out.

4.4 The Integrated Kernel Reformulation

Substituting Eq. (29) into Eq. (30) and integrating term by term against the integrated kernel $F_K(t) = \int_{-\infty}^{t} K(u)\, du$ yields the closed form

$$\hat{p}_h(x) \;=\; \frac{1}{N} \sum_{j=1}^{N} F_K\!\left(\frac{-y_j(x)}{h} \right) \;=\; \frac{1}{N} \sum_{j=1}^{N} F_K\!\left(\frac{-g(x, \xi_j)}{h} \right), \tag{31}$$

and, for the Gaussian kernel, $F_K = \Phi$, the standard normal CDF, so

$$\hat{p}_h(x) \;=\; \frac{1}{N}\sum_{j=1}^{N}\Phi\!\left(\frac{-g(x,\xi_j)}{h}\right). \tag{32}$$

The deterministic approximation of the chance constraint is then the single smooth inequality

$$\hat{p}_h(x) \;\geq\; 1-\varepsilon. \tag{33}$$

Eq. (31) is a central equation in this thesis, and the following comparison justifies the claim. Set Eq. (31) beside the indicator estimator of Chapter 1, Eq. (9): the two expressions are term-for-term identical except that the discontinuous indicator $\mathbf{1}\{g(x,\xi_j) \leq 0\}$ has been replaced by the smooth sigmoid $F_K(-g(x,\xi_j)/h)$, exactly the substitution drawn in Chapter 3's kernel figure. Every sample still votes on feasibility, but the vote is no longer binary; a sample near the constraint boundary contributes a graded value that responds continuously as the decision moves. The piecewise-constant empirical estimator has been replaced by a ramp, and the entire discontinuous probabilistic feasibility requirement of Chapter 2 by a smooth, differentiable, deterministic nonlinear constraint. Figure 13(a) shows the replacement in the standalone scalar diagnostic: the piecewise-constant empirical estimator, the true probability, and the KDE constraint at three bandwidths, with the bandwidth visibly controlling how tightly the ramp hugs the step.

4.5 Gradient Computation

Differentiating Eq. (31) by the chain rule, using $F_K' = K$, gives the gradient in closed form,

$$\nabla_x \hat{p}_h(x) \;=\; -\frac{1}{Nh}\sum_{j=1}^{N}K\!\left(\frac{-g(x,\xi_j)}{h}\right)\nabla_x g(x,\xi_j). \tag{34}$$

The structure of this formula is central to the mechanism of the method. The gradient is a kernel-weighted average of the constraint gradients $\nabla_x g(x,\xi_j)$, and the weights $K(-g(x,\xi_j)/h)$ are largest for the samples whose residuals lie within a bandwidth of the boundary. The estimator automatically concentrates the solver's attention on the scenarios that are about to change status, which is exactly the directional information Chapter 2 showed the indicator withholds. For the Gaussian kernel every quantity in Eq. (34) is elementary. Second derivatives can be obtained by the same chain rule

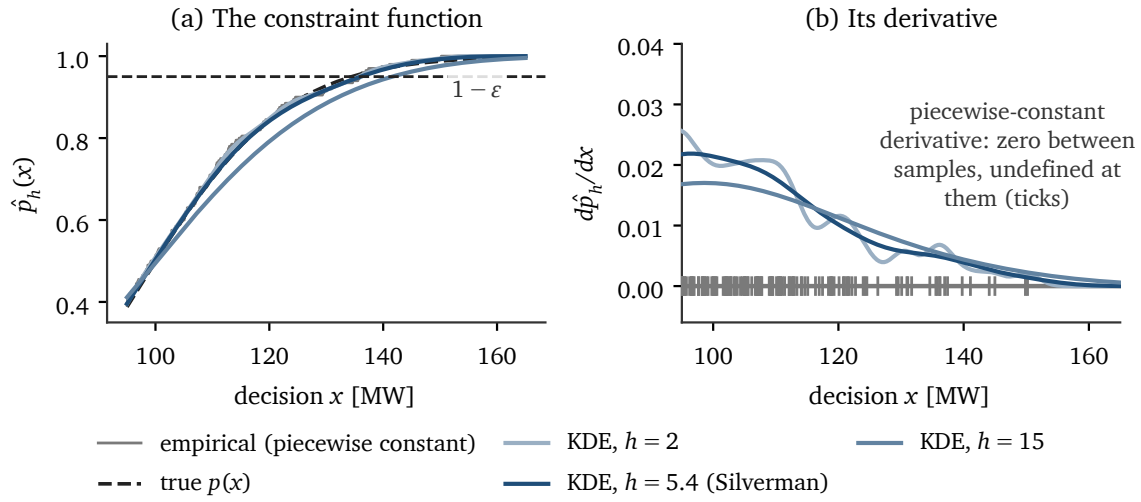


Figure 13: The reformulation as seen by the solver, for a standalone lognormal scalar diagnostic ($N = 200$ samples). (a) The piecewise-constant empirical estimator, the true probability $p(x)$, and the KDE constraint $\hat{p}_h(x)$ at three bandwidths: small h tracks the empirical estimator including its noise, Silverman's $h = 5.4$ tracks the truth, and large h distorts the curve and shifts the feasibility boundary where it crosses $1-\varepsilon$. (b) The derivatives: that of the empirical estimator is zero between samples and undefined at crossings (ticks on the axis), whereas the KDE derivative is available analytically at every x.

when g is twice differentiable; the implementation used here supplies the analytic first gradient. Sequential quadratic programming and interior-point methods therefore avoid finite differences of a piecewise-constant empirical estimator (Wächter and Biegler, 2006). Figure 13(b) displays the comparison directly: the derivative of the piecewise-constant empirical estimator is zero except at isolated undefined points, while the KDE derivative is a smooth, analytically available function at every decision. This differentiability is the core computational advantage of the KDE reformulation over the indicator-based and black-box approaches of Chapter 2.

4.6 Stochastic Optimization Reformulated via Unbiased KDE

Assembling the pieces, the chance-constrained program of Chapters 1–3,

$$\underset{x \in X}{\text{minimize}}\ f(x) \quad \text{subject to} \quad \mathbb{P}\big(g(x,\xi) \le 0\big) \ge 1-\varepsilon, \tag{35}$$

becomes, after the KDE substitution,

$$\underset{x \in X}{\text{minimize}}\ f(x) \quad \text{subject to} \quad \frac{1}{N}\sum_{j=1}^{N} F_K\left(\frac{-g(x,\xi_j)}{h}\right) \ge 1-\varepsilon. \tag{36}$$

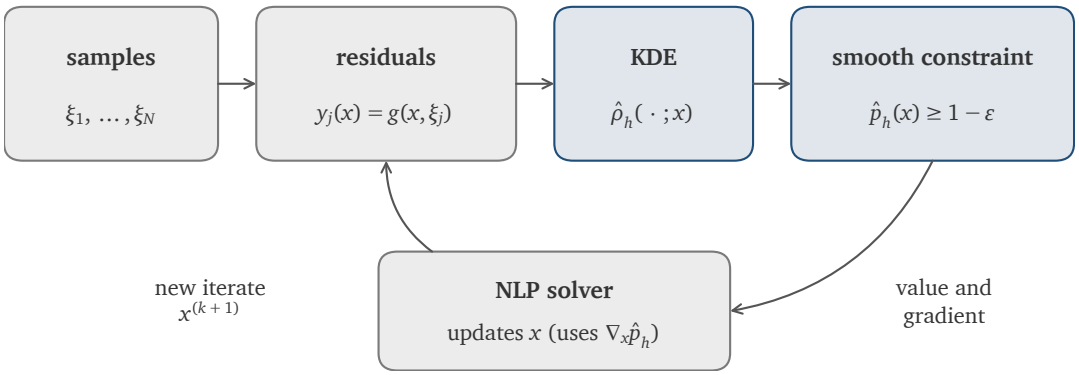


Figure 14: The reformulation as a computational loop. Uncertainty samples are drawn once and frozen; at each iterate the constraint map produces the residual cloud $y_j(x)$, the kernel estimator converts the cloud into the smooth constraint $\hat{p}_h(x) \geq 1 - \varepsilon$ with value and analytic gradient, and the nonlinear programming solver returns the next iterate to the residual map. One smooth inequality replaces one probabilistic constraint, and the program size is independent of N.

Problem (36) is a deterministic nonlinear program (NLP), and its structure should be read off explicitly because it answers, point by point, the requirements stated at the start of this chapter. One smooth nonlinear inequality replaces one probabilistic constraint; the number of constraint rows does not grow with the sample size, in deliberate contrast to the scenario approach. The samples ξ_j enter as fixed parameters inside a single sum, although evaluating the constraint and its gradient costs $O(N)$. The decision variable enters only through the residuals $g(x, \xi_j)$, so the constraint is exactly as smooth as the constraint map itself. Standard solvers, IPOPT, SNOPT, and SQP implementations, consume Eq. (36) directly, with the analytic derivatives of Section 4.5 supplied through the modeling layer (Wächter and Biegler, 2006).

Figure 14 draws the complete loop: the frozen samples and the moving decision meet in the residuals, the KDE turns the residuals into a smooth constraint value and gradient, and the solver feeds the next iterate back into the residual map. This loop is the computational object the remainder of the thesis studies, extends, and stress-tests.

4.7 Properties of the Unbiased KDE Approximation

The reformulation Eq. (36) has four relevant properties; the fourth is a genuine limitation. First, it is smooth: for continuous kernels K, the constraint is continuously differentiable in x wherever g is; a twice continuously differentiable constraint addi-

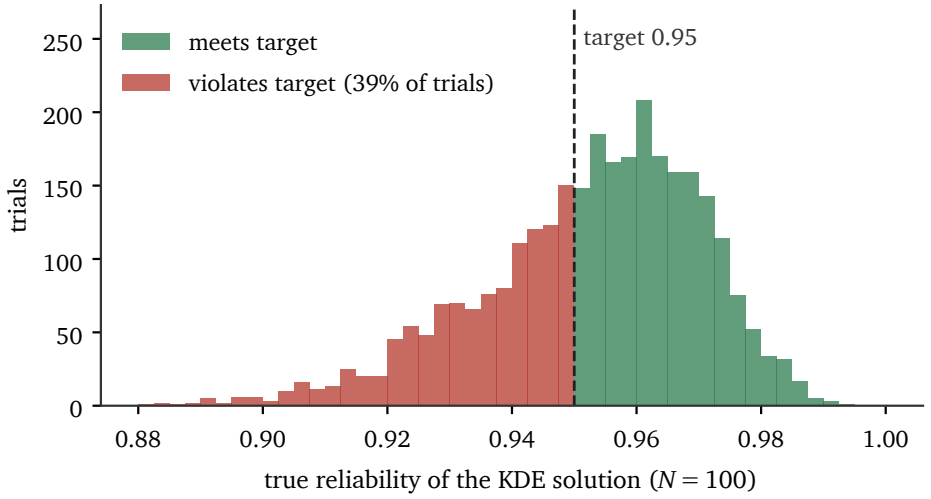


Figure 15: The unbiased reformulation is accurate but not safe in this standalone lognormal scalar diagnostic (3000 independent trials, $N = 100$). The decision certified by the unbiased KDE constraint at $1 - \varepsilon = 0.95$ is evaluated against the true demand law. The achieved reliability concentrates near the target, but falls below it in 39% of trials: the certificate $\hat{p}_h(x) \geq 1 - \varepsilon$ is optimistic. Chapter 5 addresses this defect.

tionally requires a C^1 kernel (as for the Gaussian), and the derivatives are the closed forms of Section 4.5. Second, it is consistent: as $N \to \infty$ with $h \to 0$ and $Nh \to \infty$, the estimate $\hat{p}_h(x)$ converges to the true probability $\mathbb{P}(g(x, \xi) \leq 0)$, inheriting the classical consistency of the kernel estimator (Parzen, 1962; Silverman, 1986), and under the conditions of Schuster (2025) the convergence lifts from the constraint to the optimization problem itself, optimal values and solution sets included. It is instructive to separate the two error sources at play, as anticipated in Chapter 3: a random sampling error of order $N^{-1/2}$, inherited from the finite sample, and a deterministic smoothing error controlled by the bandwidth, introduced by the kernel (Silverman, 1986; Wand and Jones, 1995). The conditions $h \to 0$ and $Nh \to \infty$ balance exactly these two contributions, and the biased construction of Chapter 5 acts on the second source alone, giving it a sign rather than removing it.

Third, the bandwidth controls a genuine trade-off: Figure 13(a) shows small h reproducing the piecewise-constant empirical estimator together with its sampling noise and large h distorting the feasible boundary, so h is not a nuisance parameter but a design variable, the optimization-versus-AMISE question flagged in Chapter 3 and studied experimentally in Chapter 9.

Fourth, and importantly for the structure of this thesis, the approximation is not conservative. A symmetric kernel smooths the indicator from both sides, so in finite samples the estimate $\hat{p}_h(x)$ can exceed the true probability, and the constraint $\hat{p}_h(x) \geq 1-\varepsilon$

can certify a decision the true chance constraint rejects. Figure 15 measures the consequence in the standalone lognormal scalar diagnostic: across three thousand independent trials at $N = 100$, the decision certified by the unbiased KDE violates the true reliability target in 39% of trials. The failures are individually small, the estimator is consistent and the misses concentrate near the target, but they are systematic, and a reformulation whose certificate fails in roughly four of ten trials functions as an accuracy method rather than a safety method. For applications where the risk level ε is a hard requirement, a lander, a grid code, a risk mandate, accuracy alone does not meet the requirement; a conservative construction is needed. Constructing a KDE reformulation that keeps the smoothness and closed-form gradient of this chapter while erring, by design, on the safe side at each fixed decision is the contribution of the biased kernels of Keil et al. (2021), and it is the subject of the next chapter.

5 The Biased KDE Approach and Conservative Reformulation

Chapter 4 closes on a measured verdict: the unbiased reformulation is an accuracy method, not a safety method, and its certificate fails in 39% of the standalone diagnostic trials. This chapter converts the verdict into a construction. The repair is not a patch applied after estimation but a redesign of the estimator itself, due to Keil et al. (2020, 2021): choose the kernel so that smoothing and conservatism become the same act, exactly the property previewed in Chapter 3's kernel figure and now developed in full. The chapter states the requirement the unbiased estimator fails (Section 5.1), formalizes the domination condition that defines a biased kernel (Section 5.2), examines the three kernels that matter in practice and the precise sense in which each satisfies or approximates the condition (Section 5.3), assembles the conservative reformulation pipeline and quantifies its conservatism on a known-truth problem (Section 5.4), and then extends the framework to joint chance constraints through three competing routes (Section 5.5), of which the scalar violation statistic with log-sum-exp smoothing (Section 5.6) is the extension this thesis recommends; it is proposed here and examined in Chapter 9 only at the level of the event statistic. The chapter closes with the bandwidth-continuation strategy that makes the whole construction solvable in practice (Section 5.7).

5.1 Why Smoothing Alone Is Not Safe

This section explains why the unbiased surrogate of Chapter 4 cannot serve as a safety method. It first shows that its failure is directional rather than statistical, and then states the requirement a safe surrogate must meet, distinguishing a bound for a fixed decision from a guarantee for the decision that the optimizer returns.

5.1.1 Directional failure of a symmetric surrogate

The limitation must be located precisely before it can be addressed, because it does not lie where one might first look. The unbiased estimator of Chapter 4 is consistent, its bandwidth theory is classical, and its certificate concentrates near the target; none of its statistical properties is at fault. The limitation is directional. A symmetric kernel's integrated form crosses the indicator at the constraint boundary, so each sample's con-

tribution to $\hat{p}_h(x)$ errs toward optimism for violating residuals and toward pessimism for safe ones, and nothing in the construction controls which effect dominates at the decision the optimizer actually returns. The optimizer, moreover, is not neutral: it systematically favors the cheapest certified decision, which tends to be a decision at which optimistic error contributed to the certificate being granted. Chapter 4 measures the result at 39% certificate failure.

5.1.2 Fixed-decision envelope versus optimizer selection

For engineering and safety-critical applications this behavior is problematic, and the requirement that replaces it can be stated in one line. What is needed is an estimator that never credits a violating sample: a surrogate $\hat{p}_h^{\,b}$ whose every per-sample contribution never exceeds the indicator it replaces. Section 5.2 makes this precise. Equivalently, in the violation form of Chapter 2, the per-sample surrogate dominates the empirical violation indicator; this does not make the population violation probability pointwise observable. Sampling and optimizer selection can still produce an unsafe population decision.

5.2 The Biasing Construction

The construction of Keil et al. (2020) unifies two half-solutions, each inadequate alone. The first replaces the violation indicator $\mathbf{1}\{t > 0\}$ with any function $\omega(t) \geq \mathbf{1}\{t > 0\}$: averaging ω over samples always upper-bounds the violation probability, but ω need not be bounded by one, so the average is not a valid probability and distorts the optimization. The second is the standard KDE surrogate of Chapter 4, a valid CDF bounded in $[0, 1]$ that violates the domination requirement. The biased kernel is the object that holds both properties at once. A kernel K with integrated form F_K and bias term $b(h) \geq 0$ satisfies the *domination condition* when its safe-side surrogate is bounded by the indicator,

$$F_K\left(\frac{-t - b(h)}{h}\right) \leq \mathbf{1}\{t \leq 0\} \qquad \text{for all } t \in \mathbb{R} \text{ and } h > 0, \tag{37}$$

equivalently, when the complementary violation surrogate dominates $\mathbf{1}\{t > 0\}$ from above for every residual. Condition (37) asks for exactly one thing beyond Chapter 4: the surrogate must vanish on the entire violating half-line $t > 0$, which the bias $b(h)$ accomplishes by shifting the kernel's mass to the safe side of the boundary.

The conservative estimator and constraint follow immediately. Averaging the dominated surrogate over the residual samples of Chapter 4 gives

$$\hat{p}_h^b(x) = \frac{1}{N}\sum_{j=1}^{N} F_K\left(\frac{-g(x,\xi_j)-b(h)}{h}\right), \qquad \hat{p}_h^b(x) \geq 1-\varepsilon. \tag{38}$$

For a fixed decision x independent of the sample, each term of Eq. (38) is bounded by the corresponding indicator and therefore $\mathbb{E}[\hat{p}_h^b(x)] \leq \mathbb{P}(g(x,\xi) \leq 0)$. The inequality is a fixed-decision expectation statement. Because an optimized decision depends on the same samples, it is not a finite-sample feasibility certificate without a selection-aware concentration argument or independent validation. Figure 16(a) draws the construction: the unbiased surrogate crosses the indicator and grants credit to violating samples (the optimism Chapter 4 measures), while the biased surrogate sits entirely beneath it, discounting boundary samples and crediting violations nothing at all. Smoothness has not been spent to buy safety; the biased surrogate keeps the closed-form constraint and analytic gradient of the unbiased one (with the Epanechnikov kernel it is continuously differentiable rather than infinitely smooth), and every formula of Chapter 4, the closed-form constraint, the analytic gradient, the constraint count independent of N (while evaluation cost is $O(N)$), survives with $-g(x,\xi_j)$ replaced by $-g(x,\xi_j)-b(h)$. In the language of the two error sources separated in Chapter 4, the bias acts on the smoothing error alone: the sampling error remains random and vanishes with N, while the smoothing error is now signed toward safety rather than merely made small.

5.3 Specific Biased Kernels

Condition (37) is a constraint on kernel tails, and the three kernels of Chapter 3 meet it in three instructively different ways. The *split-Bernstein kernel*, a piecewise-polynomial construction originating in approximation theory and adapted to chance constraints by Keil et al. (2021), satisfies the domination condition exactly for appropriate parameter choices: its compact, one-sided structure is designed around Eq. (37) rather than retrofitted to it, and its integrated form is smooth and available in closed form. It is the literature reference whenever an exact pointwise envelope is required; it is not implemented in the local runner. The *Epanechnikov kernel with bias* starts from the AMISE-optimal kernel of Chapter 3 and exploits its compact support: with $b(h) = h$, the shifted surrogate vanishes identically on the violating half-line, so Eq. (37) holds

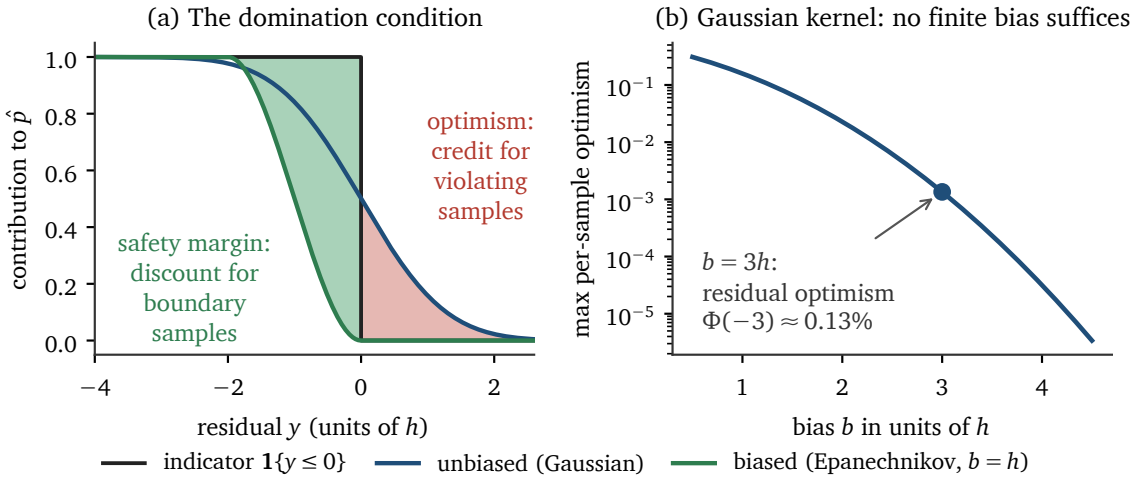


Figure 16: The biasing construction. (a) Per-sample contributions to the probability estimate, in units of the bandwidth. The unbiased Gaussian surrogate crosses the indicator and credits violating residuals (shaded right); the biased Epanechnikov surrogate with $b = h$ lies entirely beneath it, so every per-sample error points to the safe side (shaded left). (b) The Gaussian kernel cannot be made exactly safe: its maximal per-sample optimism $\Phi(-b/h)$ is positive for every finite bias; $b = 3h$ leaves 0.13% per sample.

exactly, and in the lunar landing experiments of Keil et al. (2021) this kernel achieves the lowest average optimal cost among the biased family, the statistical efficiency of Chapter 3 carrying over into the optimization.

The *Gaussian kernel* is the cautionary member, and its limitation here is structural rather than incidental: because its tails are strictly positive everywhere, no finite bias can make the surrogate vanish for $t > 0$, so the Gaussian cannot satisfy Eq. (37) exactly (Keil et al., 2021). Figure 16(b) quantifies the gap. The maximal per-sample optimism after biasing is $\Phi(-b/h)$, positive for every finite b; the practical compromise $b \approx 3h$ caps it at $\Phi(-3) \approx 0.13\%$, covering 99.87% of the tail and rendering the kernel *approximately* conservative. A truncated or compactly supported Gaussian-like kernel could remove the positive tail and close this exact-safety gap, but truncation changes normalization and boundary smoothness, so its one-sided inequality and optimization behavior would require a separate analysis; such a kernel is left for future work. The Gaussian's compensating virtues are real, infinite smoothness and the fastest solver runtimes in the family, and the resulting division of labor is the one this thesis adopts in its continuation stages: Gaussian kernels where speed and smoothness dominate, the shifted Epanechnikov kernel where the fixed-decision envelope matters, and warm-started stages (Section 5.7) to move between them.

5.4 Stochastic Optimization Reformulated via Biased KDE

The full conservative pipeline assembles the constructions of Chapter 4 and of this chapter into five steps, stated as the procedure the experimental chapters implement. First, generate N samples $\xi_1, \ldots, \xi_N$ by simulation, Markov chain Monte Carlo, or historical data. Second, for each candidate decision x, form the residuals $y_j(x) = g(x, \xi_j)$ of Chapter 4. Third, evaluate the biased constraint function $\hat{p}_h^b(x)$ of Eq. (38) with a kernel and bias from Section 5.3. Fourth, enforce $\hat{p}_h^b(x) \geq 1 - \varepsilon$. Fifth, solve the resulting deterministic nonlinear program with the analytic derivatives that the construction provides. The program retains every structural property of Chapter 4: one smooth inequality whose row count is independent of N, with evaluation and gradient cost proportional to N.

The bias changes the balance between safety and cost, and Figure 17 quantifies it on a standalone lognormal scalar diagnostic where the truth is computable. The comparison follows the question posed at the end of Chapter 4: the unbiased reformulation is smooth and less conservative, while the biased reformulation is more conservative and has the fixed-decision pointwise envelope described above. The numbers make the asymmetry explicit. Across sample sizes, the unbiased certificate fails in roughly 34–43% of trials over the tested range, with no monotone improvement across sample sizes; the biased certificate fails in 14% of trials at $N = 50$, falls to 5% by $N = 200$, and is at most 1% from $N = 500$. These observed failures are compatible with sampling error, but the experiment does not provide a selection-aware finite-sample guarantee. The premium paid for this is 2.6–4% of objective value, shrinking as N grows. The results quantify a bounded objective premium in this experiment and a lower observed failure rate for the biased surrogate. They do not establish that the ranking holds for other distributions or data-dependent decisions.

5.5 Joint Chance Constraints: Three Approaches

Everything so far concerns a scalar residual. The joint constraint of Chapters 1–3, $\mathbb{P}(g_r(x, \xi) \leq 0,\ r = 1, \ldots, m) \geq 1 - \varepsilon$, asks about m requirements at once, and the biased framework extends to it along three routes whose trade-offs this section fixes, because the choice among them is a design decision the experimental chapters revisit.

Approach A: Boole's inequality with risk allocation. The union bound $\mathbb{P}(\bigcup_r \{g_r > 0\}) \leq \sum_r \mathbb{P}(g_r > 0)$ yields a sufficient condition: enforce each component at risk

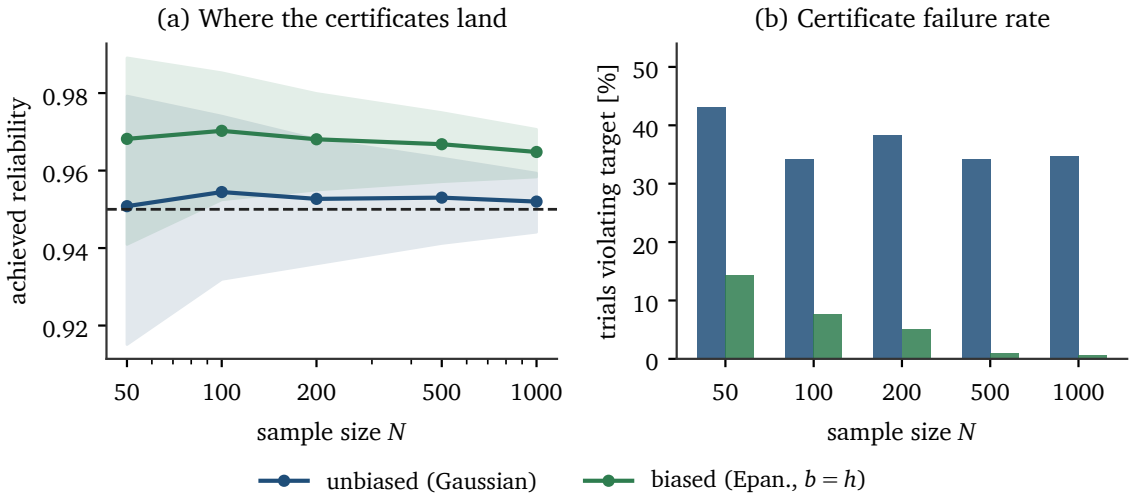


Figure 17: Cost and benefit of conservatism on a standalone lognormal scalar diagnostic (risk level $\varepsilon = 0.05$, 800 trials per sample size; bands are 10th-to-90th percentiles). (a) The unbiased certificate falls below the target on a large fraction of trials at every N; the biased certificate (Epanechnikov, $b = h$) lies above it by a margin that shrinks as the estimator sharpens. (b) Certificate failure rates: the unbiased rates show no monotone improvement, whereas the biased rate falls to at most 1% from $N = 500$ (an empirical trend, not a finite-sample guarantee). The average objective premium is 2.6–4%, decreasing in N.

level ε_r with $\sum_r \varepsilon_r \leq \varepsilon$, applying the one-dimensional biased KDE of Section 5.4 to each residual separately (Prékopa, 1995; Keil et al., 2021). The route is simple, fully tractable, and conservative twice over, once through the bias and once through the union bound; its conservatism grows with m and is most costly exactly when the component violations are strongly correlated, the case in which the union bound is least tight.

Approach B: Multivariate KDE. Estimate the joint density of the residual vector $(g_1(x,\xi),\ldots,g_m(x,\xi))$ with a multivariate kernel and bandwidth matrix H, and integrate over the safe orthant $(-\infty,0]^m$ (Wu et al., 2021; Wu and Kargarian, 2023). The route is exact in the sense that no decomposition is imposed, but it reintroduces precisely the difficulty the residual-space construction was designed to avoid: the curse of dimensionality of Chapter 3 now acts in the constraint dimension m, and the sample sizes it demands grow exponentially.

Approach C: The scalar violation statistic. The recommended extension of this thesis collapses the joint constraint exactly. For positive weights w_r, define

$$s(x,\xi) \;=\; \max_{r=1,\dots,m} w_r\, g_r(x,\xi). \tag{39}$$

The reduction is an identity, not a bound: all components are nonpositive precisely when their weighted maximum is, so

$$\mathbb{P}\big(g_r(x,\xi)\le 0,\; r=1,\dots,m\big) \;=\; \mathbb{P}\big(s(x,\xi)\le 0\big), \tag{40}$$

and the joint chance constraint is a scalar chance constraint on s. A one-dimensional biased KDE suffices; no Boole decomposition, no multivariate estimation, no curse of dimensionality. The constraint map itself has once again performed the dimension reduction, now in m as well as in d. The single difficulty is the one Chapter 3 anticipates: the max in Eq. (39) is nonsmooth, and a nonsmooth residual would re-break exactly the differentiability that Chapter 4 works to establish. Removing that catch is the purpose of the next section.

5.6 Log-Sum-Exp Smoothing of the Scalar Violation Statistic

The max is smoothed by the log-sum-exp surrogate with temperature $\tau > 0$,

$$s_\tau(x,\xi) \;=\; \tau \log\!\left(\sum_{r=1}^{m} \exp\!\left(\frac{w_r\, g_r(x,\xi)}{\tau}\right)\right), \tag{41}$$

which satisfies the two-sided bound

$$\max_r w_r\, g_r(x,\xi) \;\le\; s_\tau(x,\xi) \;\le\; \max_r w_r\, g_r(x,\xi) + \tau \log m, \tag{42}$$

(Boyd and Vandenberghe, 2004). The direction of the bound is the key property of the construction, and it parallels Eq. (37): the surrogate dominates the true statistic, so enforcing $\mathbb{P}(s_\tau \le 0) \ge 1-\varepsilon$ *implies* the true joint constraint, and the smoothing error, like the kernel bias, is conservative by construction. The temperature plays for the max exactly the role the bandwidth plays for the indicator: a smoothing parameter whose cost is a controlled, safe-side margin of at most $\tau \log m$. Figure 18(a) shows the geometry on a two-constraint slice, s_τ lying above the kink and collapsing onto it as τ shrinks.

The gradient completes the smoothness argument. Differentiating Eq. (41) gives

$$\nabla_x s_\tau(x,\xi) = \sum_{r=1}^{m} \pi_r(x,\xi)\, w_r\, \nabla_x g_r(x,\xi), \qquad \pi_r = \frac{\exp\big(w_r g_r/\tau\big)}{\sum_q \exp\big(w_q g_q/\tau\big)}, \tag{43}$$

a softmax-weighted blend of the component gradients, smooth in x and analytically computable. Figure 18(b) shows the weights transferring influence from one constraint to the other across the switching point, continuously, where the raw max would switch abruptly and nondifferentiably. Composing the pieces yields the full biased-KDE reformulation for joint constraints,

$$\frac{1}{N}\sum_{j=1}^{N} F_K\left(\frac{-s_\tau(x,\xi_j) - b(h)}{h}\right) \geq 1-\varepsilon, \tag{44}$$

and Eq. (44) is a proposed extension, motivated by what the formulation absorbs: a joint, non-Gaussian, possibly high-dimensional chance constraint has been reduced to a single smooth one-dimensional deterministic inequality, with two stacked approximation layers: the biased kernel supplies a fixed-decision expectation envelope, while log-sum-exp supplies a deterministic upper bound on the maximum. The interaction of these layers, including how conservatism compounds across h and τ, remains unresolved; Chapter 9 reports a joint-event diagnostic rather than a risk-allocation study.

5.7 Bandwidth Continuation and Kernel Switching

The constructions of this chapter create a practical tension that one further idea resolves. Small bandwidths and exactly-safe kernels are where the guarantees live, but they are also where the optimization is hardest: a sharply smoothed constraint approaches the piecewise-constant empirical estimator of Chapter 2, and a solver started cold against it inherits a rugged, nearly nonsmooth problem. The warm-start strategy of Keil and Rao (2021) converts this tension into a schedule. Figure 19 draws it. Begin with a large bandwidth, where the constraint is heavily smoothed, the problem benign, and the Gaussian kernel's speed available; solve, and pass the solution forward as the initial point for a smaller bandwidth; repeat, increasing the sample size as the bandwidth shrinks so that statistical resolution keeps pace with geometric resolution; and near convergence, switch to the locally implemented shifted Epanechnikov surrogate. The split-Bernstein switch shown in the cited workflow is a literature procedure and is not executed by the local runner. Locally, the lunar benchmark switches estimators

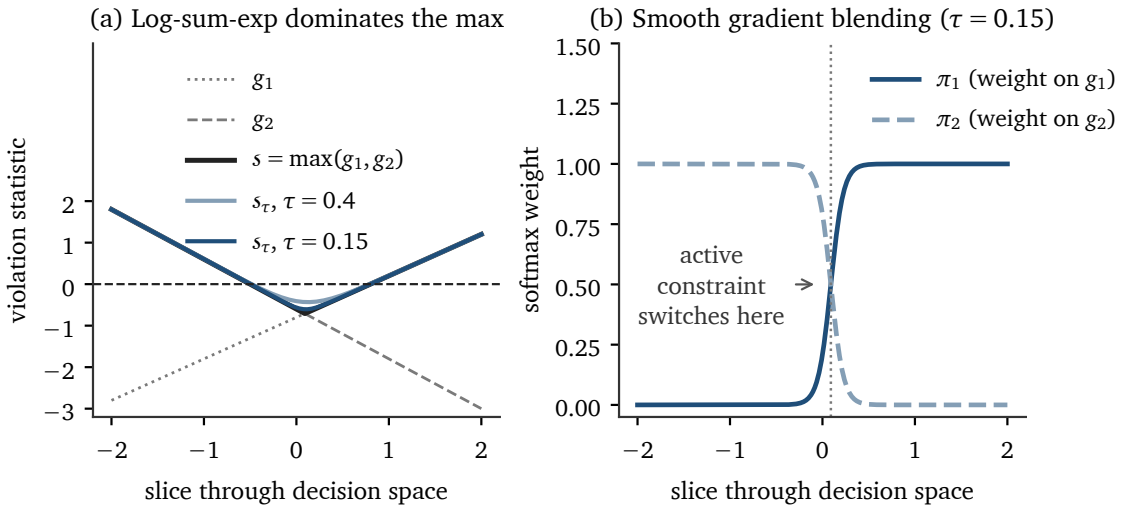


Figure 18: Log-sum-exp smoothing of the scalar violation statistic on a two-constraint slice. (a) The surrogate s_τ dominates $s = \max(g_1, g_2)$ everywhere, with gap at most $\tau \log m$ concentrated at the kink; as τ shrinks the surrogate collapses onto the max, so the smoothing error is a tunable, safe-side margin. (b) The softmax weights of the gradient formula transfer influence from one constraint to the other continuously across the switching point, giving the solver a smooth blend where the raw max would give a nondifferentiable jump.

across warm-started stages at a fixed bandwidth; the joint bandwidth and sample-size sequence is run only for the static and dispatch benchmarks of Chapter 9.

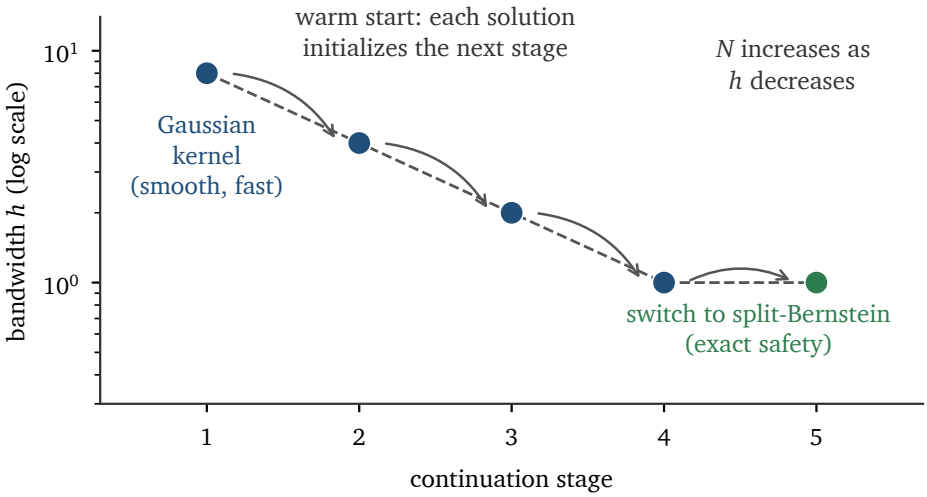


Figure 19: The bandwidth-continuation and kernel-switching schedule of Keil and Rao (2021). Early stages use a large bandwidth and the fast Gaussian kernel, producing a benign problem and a cheap solution; each stage's solution warm-starts the next as the bandwidth shrinks and the sample size grows; the cited final stage switches to the split-Bernstein kernel; that literature step is not executed by the local runner. The homotopy stabilizes the solver and avoids the numerical difficulty of attacking a small-bandwidth, nearly nonsmooth constraint from a cold start.

The schedule is a homotopy in the proper sense: a continuous deformation from an easy problem to the target problem along which the solution is tracked rather than rediscovered, and its effect in the experiments of Keil and Rao (2021) is precisely the one homotopies exist to deliver, stabilized convergence and substantially reduced solve times relative to cold starts at the final bandwidth. With it, the method of Chapters 4 and 5 is complete: a smooth, one-dimensional reformulation with constraint count independent of N for individual and joint chance constraints, equipped with a practical solution strategy. What the method still lacks is a formal guarantee that its answers approximate *the true problem*, in the sense that as data accumulates the reformulated solutions converge to the true ones. The convergence theory of Schuster (2025) supplies that guarantee for the unbiased estimator; whether it extends to the constructions of this chapter is the question the next chapter takes up.

6 Convergence Theory

Chapters 4 and 5 have so far been constructive. Chapter 4 builds a smooth surrogate for the chance constraint, Chapter 5 redesigns it to err only on the safe side, and both chapters illustrate their constructions on standalone diagnostics with known answers. What neither chapter supplies is the statement that elevates a construction to a method: the guarantee that solving the approximated problem is, in the limit of data, the same as solving the true one. That statement exists. The convergence theory of Schuster (2025) proves three theorems which together establish that the KDE-approximated chance-constrained problem is a well-posed approximation of the original: the approximation is exact where the constraint is inactive, the limits of approximate solutions are true solutions, and under a sharp-minimum condition such limits exist. This chapter is the theoretical center of the thesis. It states the gap the theory fills and why density-estimator consistency alone cannot fill it (Section 6.1), fixes the setting and assumptions (Section 6.2), develops each theorem with its statement, proof architecture, and practical meaning (Sections 6.3 to 6.5), draws the boundary of what the theory does and does not cover (Section 6.6), and closes on the structural gap between the two anchor papers of this thesis, which is an open problem in the field (Section 6.7).

6.1 The Gap the Theory Fills

This section explains why the constructions of Chapters 4 and 5 still need an optimizer-level theory. It first separates safety at fixed decisions from correctness of the computed optimizer, and then outlines the chain of three theorems that the rest of the chapter develops.

6.1.1 From estimator consistency to optimizer consistency

The biased construction of Chapter 5 provides a pointwise envelope for fixed decisions: the expected surrogate safe probability is no larger than the corresponding true probability. This does not by itself certify the sample-dependent optimizer returned by the reformulation. The distinction matters at finite N: the envelope concerns individual fixed decisions, whereas convergence concerns the optimizer and the feasible-set geometry. A method could be safe at every N and still be solving, indefinitely, a problem whose minimizer differs substantially from the minimizer of the true problem.

Safety and correctness are different properties, and Chapter 5 has so far addressed only the first, in the form of a fixed-decision envelope.

6.1.2 The three-theorem chain

The deeper question is therefore the one this chapter answers: as the sample size N grows, do solutions of the KDE-approximated problem converge to solutions of the true chance-constrained problem? Schuster (2025) provides the first rigorous answer, and the key insight that organizes the entire theory is stated before any theorem: *density-estimator convergence does not automatically imply optimizer convergence*. That the kernel estimate converges to the true density has been known since Parzen (1962), and Chapter 3 reviews the classical theory in full; but an optimization problem is not a density. Its solution is determined by the interaction of an objective with the *boundary* of a feasible set, and a sequence of converging constraint functions can, in principle, produce feasible sets whose minimizers behave badly: solutions can fail to exist, can wander without converging, or can converge to limits that are infeasible or suboptimal for the true problem. A separate argument is needed to transfer convergence of the constraint approximation to convergence of the solution, and constructing that argument, in three theorems of increasing reach, is exactly what Schuster (2025) does. Figure 20 displays the architecture the rest of this chapter presents: a statistical foundation inherited from classical KDE theory, two analytical engines, and the three theorems they power.

6.2 Setup and Assumptions

The theory is formulated at a level of generality that is itself part of the result. The decision space is not assumed finite-dimensional: Schuster (2025) works in a Banach space, which is precisely the generality required for the optimal control ambitions of this thesis, where decisions are trajectories before they are discretized. The standing assumptions are collected once.

Assumption 1 (Setting of Schuster, 2025)**.** *V is a Banach space and $X \subseteq V$ is closed and nonempty. The objective f is continuous, convex, and monotonically increasing. The random parameter ξ has an absolutely continuous distribution, and, additionally, for every $x \in X$ the pushforward distribution of the residual vector $g(x, \xi) \in \mathbb{R}^m$ is absolutely continuous with density $\rho_{g(x)}$. This excludes degenerate or singular residual maps. Moreover,*

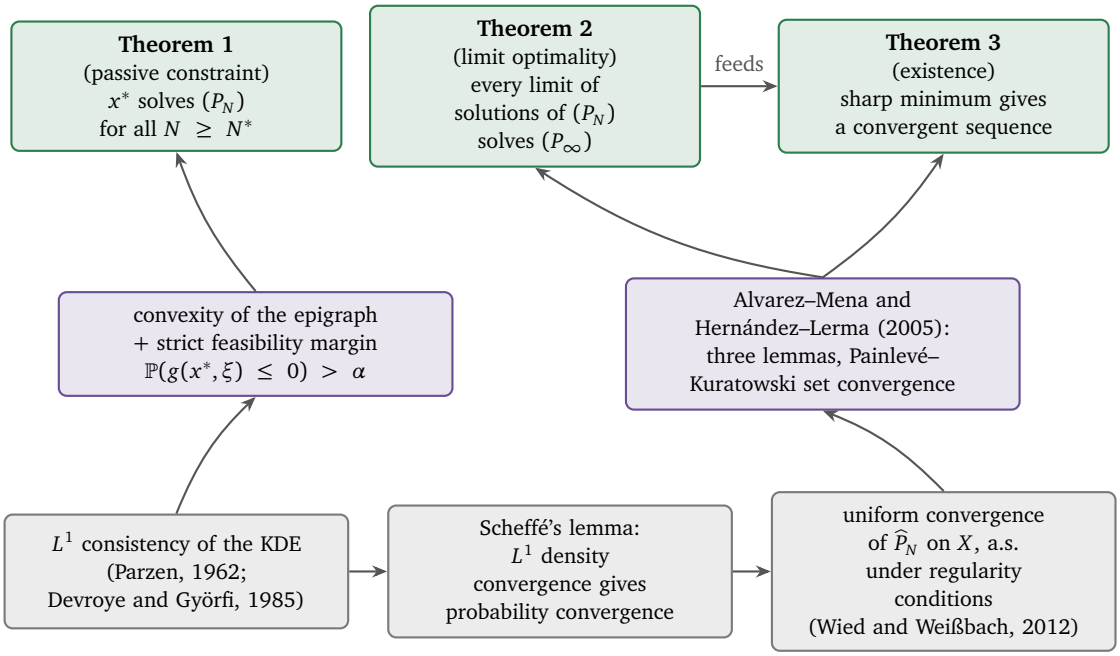


Figure 20: The architecture of the convergence theory of Schuster (2025). The statistical foundation (bottom row) is classical: L^1 consistency of the kernel estimator, upgraded to probability convergence by Scheffé's lemma and to almost-sure uniform convergence on the feasible set under the conditions surveyed by Wied and Weißbach (2012). Two analytical engines (middle row) convert this into optimization statements: a convexity-of-epigraph argument with a strict feasibility margin, and the theorem of Alvarez-Mena and Hernández-Lerma (2005) verified through three lemmas. The three theorems (top row) establish exactness where the constraint is passive, optimality of limits, and existence of convergent solution sequences.

(P_∞) *and the KDE problems considered below are assumed to have solutions; the existence theorem adds its sharp-minimum condition when a convergent solution sequence is required.*

Under Assumption 1 the chance constraint is the statement that the pushforward density places mass at least $\alpha = 1 - \varepsilon$, with risk level ε, on the safe orthant,

$$\mathbb{P}\big(g(x,\xi) \leq 0\big) = \int_{(-\infty,0]^m} \rho_{g(x)}(z)\, dz \geq \alpha, \tag{45}$$

which is the residual-space identity of Chapter 4, here in its general m-dimensional form: the theory covers joint constraints through the multivariate residual, and the scalar case $m = 1$ of Chapters 4 and 5 is the special case used throughout this thesis. The approximation replaces $\rho_{g(x)}$ by its kernel estimate built from the residual samples $y_j(x) = g(x, \xi_j)$, exactly the estimator of Chapter 4, and the pair of problems whose

relationship the theory governs is

$$(P_\infty): \quad \underset{x\in X}{\text{minimize}}\, f(x) \quad \text{subject to} \quad \mathbb{P}\big(g(x,\xi)\le 0\big)\ge \alpha, \tag{46}$$

$$(P_N): \quad \underset{x\in X}{\text{minimize}}\, f(x) \quad \text{subject to} \quad \hat{P}_N\big(g(x,\xi)\le 0\big)\ge \alpha. \tag{47}$$

Here $\hat{P}_N\big(g(x,\xi)\le 0\big)$ is the probability that this kernel estimate, built from N samples with bandwidth h_N, assigns to the safe orthant; for $m=1$ it is the estimate $\hat{p}_{h_N}(x)$ of Chapter 4, written $\hat{p}_N$ for short. Every theorem below is a statement about how the solutions of (P_N) relate to those of (P_∞), almost surely with respect to the sampling of $\xi_1,\xi_2,\ldots$, and the three theorems are numbered here as Theorems 1 to 3, corresponding to Theorems 3.1, 3.2, and 3.3 of Schuster (2025) respectively.

6.3 The Passive Constraint Case

The first theorem settles the regime in which the chance constraint, at the optimum, is not doing any work.

Theorem 1 (Passive constraint; Theorem 3.1 of Schuster, 2025)**.** *Let x^* solve (P_∞) with the chance constraint strictly inactive, $\mathbb{P}(g(x^*,\xi)\le 0)>\alpha$. Assume also the convexity or star-shaped projection and inner-approximation hypotheses used in the cited theorem. Then there exists N^* such that x^* also solves (P_N) for all $N\ge N^*$, almost surely.*

Proof architecture. The argument runs along the bottom-left of Figure 20. The L^1 consistency of the kernel estimator (Parzen, 1962; Devroye and Györfi, 1985) controls the integrated error of the density estimate; Scheffé's lemma (Scheffé, 1947) converts L^1 convergence of densities into convergence of the probabilities they induce, so that $|\hat{P}_N-\mathbb{P}|$ at x^* falls, almost surely, below the strict feasibility margin $\gamma=\mathbb{P}(g(x^*,\xi)\le 0)-\alpha>0$ for all N beyond some N^*. From that point on, x^* remains feasible for every (P_N). The convexity-of-epigraph argument in the cited theorem then transfers the optimality property; it does not rely on the approximate feasible sets being nested.

Practical meaning. When the chance constraint is strictly inactive at the optimum, the theorem shows that the KDE problems eventually recover the same optimizer under the theorem's passive-margin and regularity assumptions; it does not assert zero finite-sample statistical error. Figure 21 draws the mechanism: the margin γ is a fixed

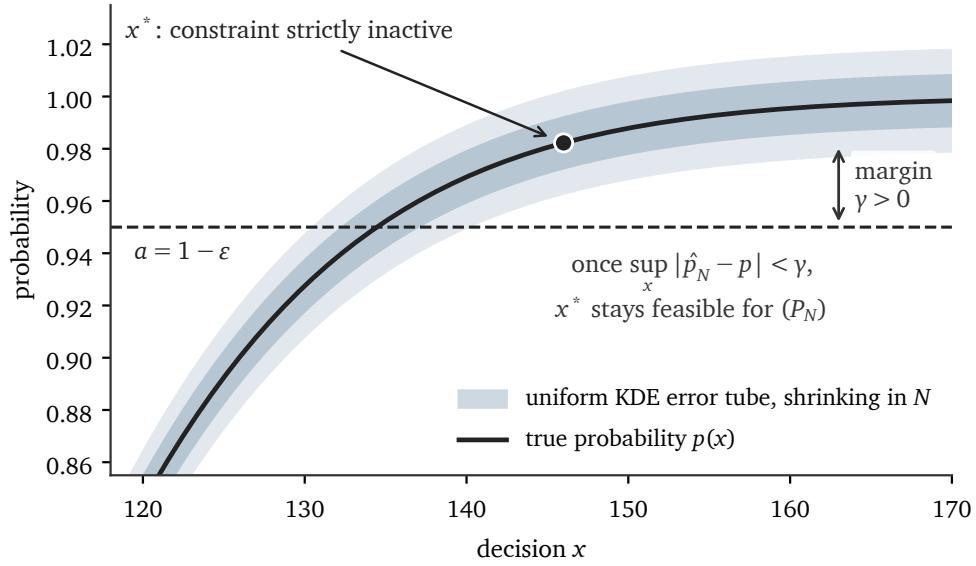


Figure 21: The mechanism of Theorem 1, illustrated on a standalone lognormal scalar probability curve. At a solution x^* where the chance constraint is strictly inactive, the feasibility margin $\gamma = \mathbb{P}(g(x^*, \xi) \leq 0) - \alpha$ is a fixed positive buffer. The KDE error is a tube around the true probability that shrinks almost surely as N grows; once $\sup_x |\hat{p}_N - p| < \gamma$, the point x^* is feasible for every approximated problem and, by the epigraph argument, solves it.

buffer, the uniform estimation error is a tube around the true probability that shrinks with N, and once the tube fits inside the buffer the approximation can no longer disturb the solution. The theorem formalizes why approximation error matters most where the chance constraint is active.

6.4 Limits of Approximate Solutions Are Optimal

The second theorem addresses the regime the first one excludes, the one that matters most in practice: the constraint binds, the approximated solutions move with N, and the question is where they are allowed to go.

Theorem 2 (Limits are optimal; Theorem 3.2 of Schuster, 2025)**.** *Suppose the kernel estimate satisfies*

$$\sup_{x \in X} \left| \hat{P}_N\big(g(x, \xi) \leq 0\big) - \mathbb{P}\big(g(x, \xi) \leq 0\big) \right| \xrightarrow[N \to \infty]{} 0 \quad \textit{almost surely.}$$

If (x_N) is a convergent sequence of solutions of (P_N), then its limit solves (P_∞), almost surely.

Proof architecture. This is the deepest of the three results, and its proof is an exercise in verifying the hypotheses of a general optimization-convergence theorem due to Alvarez-Mena and Hernández-Lerma (2005), which gives conditions under which minimizers of a sequence of problems converge to minimizers of a limit problem. Schuster (2025) establishes the three required conditions through three lemmas. The first upgrades the statistical foundation: uniform convergence of the kernel estimate yields uniform convergence of the constraint function $\hat{P}_N$ to $\mathbb{P}$ on X. The second translates constraint convergence into geometry: the admissible sets of (P_N) converge to the admissible set of (P_∞) in the Painlevé–Kuratowski sense, the notion of set convergence under which limits of feasible points are feasible. The third closes the loop from the other side: every feasible point of the limit problem can be approached by a sequence of points feasible for the approximating problems, so the approximations do not silently exclude parts of the true feasible set. With the three conditions in place, the theorem of Alvarez-Mena and Hernández-Lerma (2005) delivers the conclusion.

Practical meaning. Under the stated uniform-convergence and optimization hypotheses, any convergent sequence of approximated solutions has a true-problem limit. The plots in Chapter 9 are diagnostics; they do not verify those hypotheses and therefore do not establish a limiting optimizer. What the theorem deliberately does not assert is that a convergent sequence exists; it is a statement about limits, conditional on their existence. Closing that gap is the task of the third theorem, and the division of labor between Theorems 2 and 3 is itself instructive: optimality of limits and existence of limits are separate mathematical events, requiring separate hypotheses.

6.5 Existence of a Convergent Solution Sequence

The third theorem supplies the missing existence statement: it gives conditions under which a convergent sequence of approximate solutions exists at all.

Theorem 3 (Existence; Theorem 3.3 of Schuster, 2025)**.** *Suppose x^* is the unique solution of (P_∞) and satisfies a sharp-minimum condition: there exist $\kappa, \delta > 0$ such that $f(x) > f(x^*) + \kappa$ for every feasible x outside the ball $B_\delta(x^*)$. Assume in addition the existence, projection, and inner-approximation hypotheses required by the cited theorem. Then a convergent sequence of solutions of the approximated problems exists, almost surely.*

Proof architecture. The sharp-minimum condition is exactly what forbids the failure mode that Theorem 2 could not exclude. If near-optimal objective values were available far from x^*, the approximated solutions could achieve them and wander indefinitely; the growth condition makes every such excursion expensive, and combined with the set-convergence machinery of Section 6.4 it forces the solution sequence to be Cauchy, hence convergent. Figure 22 contrasts the two geometries: under a sharp minimum the approximate solutions are trapped in a shrinking neighborhood of x^*; in a shallow valley, nothing prevents them from drifting between distant near-optimal points.

Practical meaning. The conditions under which the full theory operates are explicit regularity conditions, and Schuster (2025) makes those conditions concrete. On the geometry side, strict convexity of the objective on a convex feasible set, together with existence, implies uniqueness; a quantitative growth condition (for example, strong convexity on a finite-dimensional compact sublevel set) is still needed to supply the sharp minimum, so the hypothesis of Theorem 3 is met by a recognizable problem class. On the statistical side, the uniform convergence that Theorems 2 and 3 require is not merely an abstract assumption: a Gaussian product kernel with a diagonal bandwidth matrix, under the classical bandwidth conditions and uniform continuity of the true density, is identified as a concrete realization that guarantees it, drawing on the consistency conditions surveyed by Wied and Weißbach (2012) and the L^1 theory of Devroye and Györfi (1985). The uniform-convergence assumption can therefore be discharged by the Gaussian kernel used in this thesis; the objective assumptions hold at most for the dispatch adapter (Section 6.6).

6.6 What the Theory Does and Does Not Cover

Taken together, the three theorems establish two properties for sequences that satisfy their hypotheses. First, the KDE approach is asymptotically consistent at the level of the optimization problem, rather than only at the estimator level. The second is precision about conditions: the theory identifies exactly what convergence costs, namely uniform convergence of the estimate on the feasible set, plus either a passive constraint or a sharp minimum, and it exhibits concrete, standard objects that pay the cost. A reader of Chapters 4 and 5 now knows not only how to build the reformulation but when its assumptions apply.

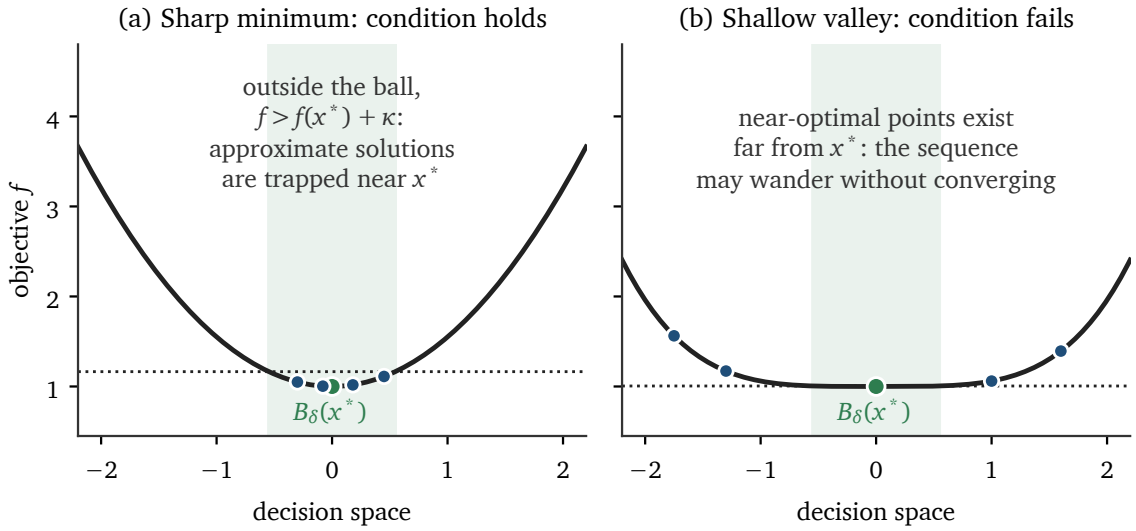


Figure 22: Why Theorem 3 needs the sharp-minimum condition. (a) When the objective grows by at least κ outside the ball $B_\delta(x^*)$ (shaded), every approximate solution (blue) with near-optimal value is forced into the ball, and the set-convergence machinery makes the sequence Cauchy. (b) In a shallow valley the condition fails: near-optimal points exist far from x^*, and approximate solutions may alternate between them without converging, even though, by Theorem 2, any limit they did have would be optimal.

Four limitations remain outside the theory. There are, first, *no convergence rates*: the theory guarantees that the optimization error vanishes but not how fast, and no theorem links the kernel's bias–variance behavior to the decay of $|f(x_N) - f(x^*)|$ or $\|x_N - x^*\|$; Schuster (2025) explicitly flags this as open. Figure 23 illustrates the gap: on a standalone lognormal scalar diagnostic the optimization error decays along an approximately linear trend on logarithmic axes, with slope roughly -0.46, a regular behavior that no available theorem currently predicts. Second, the theorems cover the *unbiased* estimator of Chapter 4; whether and how they extend to the biased construction of Chapter 5 is open, a point developed in Section 6.7. Third, the results rely on convexity of the objective, while the lunar trajectory problem is nonconvex. The dispatch adapter is affine with a convex increasing objective and is the closest local analogue to the theorem's geometry; comparable guarantees for the lunar case do not follow. Fourth, the theory is asymptotic through and through: unlike the scenario approach of Chapter 2, it offers no N-dependent probability that a computed solution is feasible for the true problem. These limitations define the evidence that Chapters 7–9 must supply beyond the mathematical results.

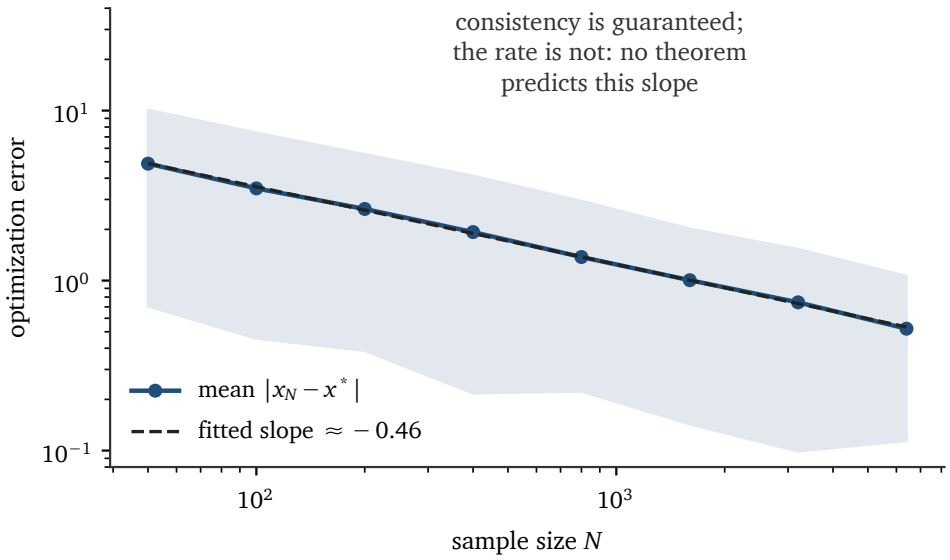


Figure 23: Consistency without rates. Mean optimization error $|x_N - x^*|$ of the unbiased KDE solution on a standalone lognormal scalar diagnostic (500 trials per sample size, band is 10th-to-90th percentiles), on logarithmic axes. The error follows an approximately linear trend of slope ≈ -0.46, but the convergence theory guarantees only that the error vanishes; no theorem predicts the slope.

6.7 The Gap Between the Two Anchor Papers

One observation closes the theoretical development of Chapters 4–6. The two anchor papers of this thesis work with the same high-level idea from opposite directions, and their results do not yet meet. Keil et al. (2021) builds a *biased* integrated kernel that dominates the indicator for each residual, yielding a fixed-decision expectation bound, but proves no convergence of the resulting optimizers. Schuster (2025) proves convergence of the optimizers, but for the *unbiased* kernel density estimate, with no safety guarantee at finite N. Neither paper's theorems cover the other's construction: the engineering line has a fixed-decision envelope without proven optimizer convergence, and the theoretical line has optimizer convergence without a built-in finite-sample envelope.

The resulting open problem is a unified convergence theory for the biased-KDE reformulation, establishing that the conservative construction of Chapter 5 inherits the asymptotic correctness of this chapter, ideally with rates. Its resolution would fuse a finite-sample, selection-aware safety result with correctness in the limit into a single certified method. This thesis does not claim to close that problem. What it does provide, in Chapters 7–9, is empirical groundwork for such a closure: single-seed diagnostics in which the biased and unbiased reformulations are run side by side at

three nested sample sizes with their bandwidth rules, including problems with exact probability references. These diagnostics do not measure convergence; the theory of this chapter determines how they can be interpreted, and the gap between the anchors motivates the comparison. The next chapter instantiates these distinctions in a discretized optimal-control formulation. It records which assumptions are represented by the local solver and which remain outside the numerical evidence.

7 Stochastic Optimal Control: Formulation and Audit

The optimal-control extension is the point at which statistical approximation, numerical transcription, and physical modeling become inseparable. A chance constraint is evaluated on a residual produced by a trajectory, but the trajectory itself is generated by a discretized differential equation. Consequently, a small test violation can coexist with a failed nonlinear program, and a reported objective can depend on the quadrature convention rather than on the physical control effort. This chapter states the formulation used by the local implementation, identifies the differences from the published lunar benchmark, and derives an audit identity that limits the interpretation of the local costs.

7.1 Problem Formulation and Residual Constraints

Let $y(t) \in \mathbb{R}^{n_y}$ and $u(t) \in \mathbb{R}^{n_u}$ denote state and control trajectories, and let ξ denote uncertain parameters. A generic finite-horizon problem is

$$\begin{aligned} \underset{y,u}{\text{minimize}} \quad & J(y,u) = \int_0^{t_f} L(y(t),u(t))\,dt + \Psi(y(t_f)), \\ \text{subject to} \quad & \dot{y}(t) = F(y(t),u(t),\xi), \\ & y(0) \in \mathscr{Y}_0, \quad u(t) \in \mathscr{U}, \\ & \mathbb{P}(g_\ell(y,u,\xi) \le 0) \ge 1-\varepsilon_\ell, \quad \ell = 1,\dots,m. \end{aligned} \tag{48}$$

The residual g_ℓ may be a terminal event, a path event evaluated at a time t, or a vector event reduced to a scalar statistic. The distinction matters: a terminal probability is not a pathwise probability over a continuum of times. In the implementation, path constraints are evaluated at mesh nodes. They are therefore nodewise surrogate constraints unless an additional union or supremum construction is imposed.

For training samples $\{\xi_i\}_{i=1}^N$ and a decision vector z containing the nodal states and controls, the residual observations are

$$r_{\ell i}(z) = g_\ell(y(z),u(z),\xi_i). \tag{49}$$

As in Chapters 4–6, the empirical indicator constraint is replaced by an integrated-kernel average, here written in violation form,

$$\hat{v}_{\ell,h}(z) = \frac{1}{N}\sum_{i=1}^{N} \overline{K}\left(\frac{r_{\ell i}(z)}{h}\right), \qquad \hat{v}_{\ell,h}(z) \le \varepsilon_\ell, \tag{50}$$

where K has CDF F_K and $\overline{K}(t) = 1 - F_K(-t)$ is the integrated violation kernel. Hence $\hat{v}_{\ell,h} = 1 - \hat{p}_h$ for the residual g_ℓ, in the notation of Chapter 4. Thus positive residuals contribute near one and negative residuals contribute near zero. Eq. (50) is a deterministic NLP constraint after the samples and bandwidth have been fixed. It does not by itself provide a confidence level for the unknown probability. The biased construction discussed in Chapter 5 gives a fixed-decision pointwise envelope; the local biased KDE is recorded as an implementation surrogate and is not identified with the Split–Bernstein construction of Keil et al. (2020, 2021).

7.2 Discretization and Continuation

The local fallback solver uses n_t uniformly spaced nodes $0 = t_0 < \cdots < t_{n_t-1} = t_f$ and forward Euler defects. With $\Delta t_j = t_{j+1} - t_j$,

$$y_{j+1} - y_j - \Delta t_j F(y_j, u_j, \xi_0) = 0, \qquad j = 0, \ldots, n_t - 2, \tag{51}$$

where ξ_0 is absent from the deterministic dynamics in the lunar proxy. The collocation vector includes all nodal controls, although the last control does not enter the forward Euler defects. Equality constraints enforce the initial state, the defects, and terminal velocity. Terminal position remains a chance-constrained quantity. Inequality constraints enforce control bounds and the KDE approximations of the terminal and path residuals.

The implementation separates four operations. The problem module defines dynamics and residuals; the collocation module maps a vector z to nodal states and controls; the KDE module evaluates integrated-kernel statistics; and the solver module supplies the sequential least-squares quadratic programming (SLSQP) solver with equality and inequality callables. This separation makes it possible to inspect a statistical estimator without silently changing the dynamics. It also exposes a numerical limitation: the same nodal control can contribute to a path residual and to the objective while not contributing to the state defects when it is the final control. That final control is

therefore an explicit transcription defect, not evidence about physical fuel use. The limitation is quantified below rather than hidden in a single objective number.

The continuation sequence is nominal, worst-case, scenario, unbiased KDE, and biased KDE. A stage is warm-started with the last successful vector. Warm starts are consistent with the continuation strategy described by Keil and Rao (2021), but the local implementation does not use Legendre–Gauss–Radau (LGR) nodes, adaptive mesh refinement, IPOPT, or GPOPS-II. A failed SLSQP stage is retained in the result ledger with its status and constraint margin; it is not promoted to a feasible solution because its held-out violation happens to be below the target.

7.3 Local Lunar Benchmark

The local benchmark has altitude y_1, vertical velocity y_2, and nonnegative thrust u:

$$\dot{y}_1 = y_2, \qquad \dot{y}_2 = -g_{\mathrm{M}} + u, \qquad g_{\mathrm{M}} = 1.622, \qquad 0 \le u \le 3. \tag{52}$$

The local horizon is fixed at $t_f = 20$, the initial state is $(y_1(0), y_2(0)) = (100, 0)$, and only terminal velocity is an equality. Terminal altitude is subject to

$$\mathbb{P}\big(\big|y_1(t_f) - \xi_1\big| - \eta > 0\big) \le \varepsilon_a, \quad \xi_1 \sim \mathcal{N}(0, 0.1^2), \quad \eta = 0.25, \quad \varepsilon_a = 0.1. \tag{53}$$

The nodewise path event is

$$\mathbb{P}\big(u(t_j) + \xi_2 - 3 > 0\big) \le \varepsilon_b, \qquad \varepsilon_b = 0.01, \tag{54}$$

with the locally documented mixture $\xi_2 \sim 0.6\mathcal{N}(-0.25, 0.10^2) + 0.4\mathcal{N}(0.35, 0.15^2)$. The mixture is a synthetic stress test for an asymmetric bimodal disturbance; it is not a claim that the local data reproduce the distribution used in the published study.

The comparison with Keil et al. must be made at the level of formulation. Their lunar study uses $y_1(0) = 10$, $y_2(0) = -2$, free final time, free terminal altitude, terminal velocity zero, and $J = \int u\,dt$; it solves the resulting problem with LGR/GPOPS-II/SNOPT and reports costs around 9.09 for the chance-constrained methods (Keil et al., 2020, 2021). The local problem fixes a different initial altitude, initial velocity, horizon, mesh, and solver. It also uses an 11-node forward-Euler transcription solved by SLSQP, declared synthetic normal and two-mode mixture laws in place of the published uncertainty model, and a trapezoidal nodal objective rather than the physical

integral $\int u\,dt$ (Section 7.4). The two objective values therefore cannot be compared numerically. The local experiment is a pipeline and audit experiment, not a replication.

7.4 Conservation and Quadrature Audit

The local transcription provides a stronger diagnostic than a solver status. Summing the velocity defects over the $n_t - 1$ intervals gives

$$y_{2,n_t-1} - y_{2,0} = \sum_{j=0}^{n_t-2} \Delta t_j(-g_{\mathrm{M}} + u_j). \tag{55}$$

With uniform $\Delta t = 2$, $y_{2,0} = y_{2,n_t-1} = 0$, and $g_{\mathrm{M}} = 1.622$, every feasible trajectory satisfies

$$\sum_{j=0}^{n_t-2} \Delta t\, u_j = g_{\mathrm{M}} t_f = 32.44. \tag{56}$$

Thus, in this fixed-horizon model, the physically consistent piecewise-constant fuel integral is invariant over the feasible set. Trajectory shape still matters because terminal altitude imposes the weighted moment

$$\int_0^{t_f} (t_f - t)u(t)\,dt = \frac{g_{\mathrm{M}} t_f^2}{2} - y_1(0) = 224.4, \tag{57}$$

when terminal altitude is zero. The stochastic terminal event changes the admissible timing of thrust, not the invariant integral implied by terminal velocity.

The code records a trapezoidal integral over all n_t controls, whereas the Euler defects use only $u_0, \ldots, u_{n_t-2}$. On a uniform mesh,

$$J_{\mathrm{trap}} = \Delta t \left(\frac{u_0}{2} + \sum_{j=1}^{n_t-2} u_j + \frac{u_{n_t-1}}{2} \right) = 32.44 + \frac{\Delta t}{2}(u_{n_t-1} - u_0). \tag{58}$$

The nominal record $J = 29.8052$ is explained by the endpoint term: the stored solution has approximately $u_0 = 2.6348$ and $u_{n_t-1} = 0$. The later records at 32.44 are not evidence that robust control requires more physical fuel. They reveal that the endpoint contribution becomes different when the optimizer is warm-started under another constraint. This audit is a substantive result of the local implementation and is the reason Chapter 9 reports the lunar objective only as a transcription diagnostic.

7.5 Interpretation and Transfer Limits

This section interprets the lunar stage ledger and states what the local experiment can and cannot transfer from the anchor papers. It first applies the solver-feasibility and validation gates to the continuation stages, then states the transfer hypotheses that connect the local run to the anchor papers, and finally delimits the claims that the formulation supports.

7.5.1 Stage feasibility gates

The stage ledger is interpreted by two gates: solver feasibility and independent validation. The nominal stage succeeds but has terminal test violation 0.51425. The worst-case, scenario, unbiased KDE, and biased KDE stages report SLSQP failure; their controls remain diagnostic records, not feasible solutions. Their negative chance margins (approximately -0.738, -0.280, -0.0209, and -0.0867) cannot be repaired by a low terminal frequency, even where it lies below 0.1 (scenario and biased KDE stages), especially because no pathwise held-out validation was performed. The local result is therefore evidence about numerical difficulty and residual statistics, not a successful robust landing.

Warm starts can reduce discontinuities between continuation stages, but they cannot create feasibility when the fixed horizon, transcription, and risk margins are incompatible. The continuation of Keil and Rao (2021) couples kernel changes to increasing samples and mesh refinement; the local sequence changes the estimator at a fixed coarse mesh. It is thus a controlled implementation analogue, not a replication of the published convergence workflow.

7.5.2 Keil and Schuster transfer hypotheses

The optimal-control experiment is most informative when stated as three hypotheses rather than as a search for a single best number. First, the Keil hypothesis is that the one-sided Epanechnikov construction with $b(h) = h$ should not underestimate the empirical violation of the residual samples used in the NLP. This is a pointwise property of the integrated kernel. It does not imply that the returned trajectory is feasible when the NLP fails, and it does not provide a finite-sample statement about the unknown distribution. Second, the Schuster hypothesis is that a Gaussian residual KDE with a shrinking bandwidth and increasing sample size should approach the true probabil-

ity functional when the required convergence assumptions hold. This is a sequence statement, not a claim about the $N = 250$ run. Third, the physical hypothesis is that a consistent discretization should make the reported control objective agree with the integral implied by the dynamics. The fixed-horizon identity above provides a direct falsification test for this hypothesis.

The local model satisfies only fragments of these transfer conditions. The biased KDE residual is scalar and uses the same $b(h) = h$ translation as the Keil construction, so the kernel inequality can be checked numerically. The samples are nevertheless IID draws from declared synthetic laws, not the MCMC chains used in the optimal-control paper. The local solver has no mesh-error estimate and no adaptive LGR refinement. The Schuster setting is also only partially represented: the residual KDE is scalar, the sample size is finite, and the static chance boundary is active. Uniform convergence over the decision set is not measured. In addition, the static objective is not globally monotone on its box and the chance-feasible geometry is nonlinear, so the objective and regularity assumptions in the convergence theory cannot be silently assumed.

Adding the Keil warm-start idea to the three hypotheses above gives four transfer claims. The biased-kernel upper-bound claim is represented locally by $b(h) = h$ and pointwise scalar checks; a lower biased KDE violation is evidence of the deliberate one-sided shift, not reproduction of Keil's MCMC/LGR method, and the claim still requires the stated sampling conditions and a feasible nonlinear-program solution. The Schuster convergence claim is represented only in the static and dispatch paper-aligned profiles of Chapter 9, by nested N and shrinking h_N diagnostics; a stable paper-aligned sequence there is numerical evidence for those models, not a uniform-convergence proof, and the claim still requires uniform convergence and a limiting optimizer. The Keil warm-start idea is represented in the lunar run by warm-started stages that change the estimator at a fixed bandwidth, and bandwidth continuation appears only in the same paper-aligned profiles; the published MCMC schedule, mesh error control, and LGR refinement remain absent. The physical-fuel comparison is represented by the telescoping defect audit, in which a changing lunar objective with invariant interval fuel identifies a transcription defect; consistent quadrature and model equivalence are still required before the objectives can be compared. The experiment therefore diagnoses the estimator–trajectory chain without collapsing statistical and physical errors into one metric.

7.5.3 Scope of valid claims

The formulation is adequate for a controlled comparison of residual estimators in low dimension. It is not adequate for a claim about continuous-time path safety, because nodewise constraints do not bound the inter-node maximum. It is not adequate for a physical fuel comparison until the quadrature is made consistent with the defect control parameterization. Finally, the finite-dimensional KDE constraint inherits the statistical qualifications developed earlier: convergence of a KDE sequence under assumptions, as in Schuster (2025), does not give a finite-sample confidence statement for the particular 250-sample local run. These limits determine how the lunar records are interpreted in Chapters 9 and 10.

8 Implementation and Validation Protocol

The implementation is an evidence chain rather than a collection of independent scripts. A numerical value is admissible for interpretation only when its uncertainty law, residual convention, estimator, bandwidth, sample split, solver status, and validation scope are known. This requirement is central for the present thesis because the theory and the two anchor papers make different claims. The results of Schuster (2025) concern sequences of KDE-approximated optimization problems under convergence assumptions. Keil et al. (2020, 2021) construct a biased kernel whose integrated function dominates the violation indicator, and combine it with MCMC sampling and optimal-control continuation. The local code is a small, fully local analogue. Its purpose is to expose which parts of those arguments survive a transparent implementation and which parts still require a different numerical experiment.

8.1 Pipeline and Claim Contract

The pipeline is

$$\begin{gathered}\text{training samples} \to \text{residual map} \to \text{KDE or baseline constraint} \to \\ \text{NLP or collocation} \to \text{independent validation} \to \text{saved evidence.}\end{gathered}$$

Each arrow is a possible failure point. A sampling error changes the empirical residual law. A sign error changes safe probability into violation probability. A bandwidth error changes both the statistical approximation and the geometry seen by the optimizer. A solver can return a finite vector with a failure status. Finally, an independent test frequency can be calculated correctly for an invalid or failed decision. The implementation therefore reports these layers separately rather than collapsing them into one safety label.

The result schema records benchmark, method, distribution, seed, training and test sizes, target risk level, bandwidth, objective, training violation, held-out violation, estimated violation, gap, runtime, solver message, and success status. The paper-aligned runs additionally record the bandwidth rule, bias ratio $b(h)/h$, sample-size schedule, continuation attempts, and whether samples are IID synthetic or MCMC. These fields make it possible to distinguish a paper-aligned unbiased KDE experiment from a paper-aligned biased KDE experiment without changing the residual code.

For a decision $\hat{x}$ and independent test observations ξ_k^{test}, the main validation statistic is

$$\hat{v}_{\text{test}}(\hat{x}) = \frac{1}{N_{\text{test}}} \sum_{k=1}^{N_{\text{test}}} \mathbf{1}\{g(\hat{x}, \xi_k^{\text{test}}) > 0\}. \tag{59}$$

The reported gap is

$$\text{gap} = \varepsilon - \hat{v}_{\text{test}}(\hat{x}). \tag{60}$$

A positive gap is an observation on one independent test sample. It is not a confidence statement about the unknown probability and does not certify a failed optimization stage. For the paper-aligned runs, the test sample is fixed across the nested training sizes, so changes in $\hat{v}_{\text{test}}$ can be interpreted as changes in the fitted decision rather than changes in the test population.

8.2 Data Generation and Residual Interfaces

The uncertainty module creates seeded Gaussian, bimodal, skewed, heavy-tailed, and renewable-error samples. The residual is evaluated only after a candidate decision is supplied, so the KDE is an estimator of the residual distribution induced by that decision, not of the uncertainty in isolation. Scalar residuals use the integrated Gaussian or Epanechnikov kernel. Vector residuals are reduced by an explicitly named joint statistic; the current joint comparison uses Boolean maximum events and log-sum-exp, not a multivariate negative-orthant KDE.

Experiment-specific sample sizes, bandwidth rules, and continuation schedules are defined in Chapter 9. This chapter fixes the interfaces: the uncertainty module produces declared samples, the residual module maps a candidate decision and sample to a scalar or named joint statistic, and the estimator module returns the quantity consumed by the optimizer.

A failed solve remains in the ledger. The test frequency is never used to relabel it as successful. The local lunar solver validates terminal position only; it does not provide an independent all-time path test. The implementation therefore records terminal frequency as terminal evidence and does not call it a path-safety result.

8.3 Validation Layers and Reproducibility Limits

The first layer consists of algebraic and derivative checks. Gaussian and Epanechnikov kernels integrate to one, their integrated limits are correct, and the $b(h) = h$

Epanechnikov contribution is no larger than the safe indicator for positive residuals. Finite-difference checks then test the smooth constraint interface used by SLSQP. These checks catch normalization, sign, support, and gradient errors before statistical interpretation begins.

The second layer uses independent probability calculations. The Gaussian static row admits a noncentral-χ^2 reference, and the dispatch mixture has a closed-form CDF at the balance threshold. Agreement with held-out Monte Carlo answers a distributional check; it says nothing about the skewed or heavy-tailed rows or optimizer convergence. The third layer is independent validation: training samples enter the NLP, while a fixed-seed test set is used only after the decision is returned. Binomial intervals quantify test-sample noise conditional on that decision, not adaptive selection or a scenario guarantee.

The final layer is numerical feasibility. Solver status, objective, iterations, chance margins, and equality residuals are inspected together. The lunar audit shows why this matters: Euler defects fix the interval fuel integral, while the trapezoidal objective includes a control that does not enter the dynamics. The stored nominal objective is therefore a quadrature diagnostic that a held-out terminal frequency cannot repair.

Fixed seeds make the local computation deterministic: the same configuration regenerates the same samples, decisions, and records. Determinism is weaker than statistical reproducibility. Schuster's convergence theory needs a sequence of bandwidths and sample sizes with uniform or L^1 convergence conditions; the local nonlinear static problem has an active KDE boundary and one run cannot establish that convergence. The dispatch objective is closer to the convex setting, but it still has one stochastic residual and no repeated-seed study.

9 Experimental Evaluation

This chapter reports the local evidence rather than treating every output field as a theorem. The analytical formulation question (RQ1) is established in Chapters 4–6. The experiments address RQ2 by comparing nominal, scenario, unbiased KDE, and biased KDE (shifted Epanechnikov kernel, $b(h) = h$) constraints across residual laws and a balance-constrained dispatch adapter; RQ3 by varying bandwidth and nested sample size; and RQ4 by comparing scalar joint-event statistics. The lunar continuation is a separate audit objective that establishes what the local transcription can support after its numerical checks. All experiments are synthetic, seeded, and run locally. The resulting measurements are empirical frequencies or estimator diagnostics; they are not out-of-sample certificates.

9.1 Design, Estimands, and Provenance

The static model has two decision variables and a nonlinear residual whose uncertainty enters through a scalar residual map. Four residual laws are used: Gaussian, bimodal, skewed, and heavy-tailed. For each law, 250 training observations define the optimization constraint and 4000 independent test observations define the held-out frequency of Eq. (59). The target is $\varepsilon = 0.1$. The method order is nominal, scenario, unbiased KDE, and biased KDE. The KDE rows use the bandwidth of Silverman's robust rule, Eq. (25), computed once before optimization from the first uncertainty coordinate of the 250 training observations in the static rows and from the 250 renewable-error training samples in the dispatch rows; the stored values are $h \approx 0.295$, 0.376, 0.242, and 0.277 for the Gaussian, bimodal, skewed, and heavy-tailed laws and $h \approx 0.093$ for dispatch. The exact seed mapping is 20260906 for the Gaussian static run, 20260907 for dispatch and the bimodal row, 20260908 for the joint comparison and the skewed row, 20260909 for sensitivity and the heavy-tailed row, and 20260910 for the lunar continuation. The mapping is recorded explicitly because a statement such as "the five benchmark seeds" is otherwise not reproducible.

The result ledger contains objective, training violation, held-out violation, estimated violation, solver status, and gap fields. The original runner had a reporting defect: it stored a Gaussian diagnostic under `estimated_violation` for the biased KDE. The corrected runner now reports the same biased KDE that appears in the NLP; its successful rows are active at approximately 0.1. The older records remain preserved as an audit artifact, while the corrected and paper-aligned records are used for the com-

Table 2: Static Gaussian benchmark. The test frequency uses 4000 independent observations and $\varepsilon = 0.1$.

Method	Objective	Train	Test	Est. diagnostic	Gap
Nominal	0.0234	0.5320	0.5180	0.5320	−0.4180
Scenario	0.6134	0.0040	0.0015	0.0040	0.0985
Unbiased KDE	0.1981	0.0520	0.0620	0.1000	0.0380
Biased KDE	0.3071	0.0320	0.0193	0.1000	0.0808

parisons below. This correction changes the interpretation of the estimator field but does not change the stored decisions or held-out frequencies.

Each experiment changes one declared part of the method and has one primary estimand and one validation gate. The static comparison changes the estimator and residual law and measures objective together with held-out violation; solver status and an independent Gaussian check are required. The paper-aligned sequence changes N and h_N and follows the decision path over nested prefixes against a fixed independent test set. The dispatch transfer changes the model adapter and estimator and measures objective and balance violation against the exact mixture CDF. The joint experiment changes only the scalar event map and measures the test union frequency after the component and event definitions are checked. The lunar continuation changes the continuation stage and records feasibility and the audit invariant, with solver status (including the Karush–Kuhn–Tucker conditions) and path scope as gates. A result is not promoted to a method claim when its gate fails.

9.2 Static Nonlinear Benchmark

The Gaussian case provides the clearest baseline because its true radial probability can be evaluated independently. Table 2 reports the local results.

The nominal decision is not a marginally unsafe solution: its held-out frequency is 0.5180, five times the target. The scenario solution is strongly conservative on this split, while the two KDE solutions retain more objective value. The unbiased KDE is closer to the Gaussian reference than the biased KDE in objective value, but the biased KDE has the lower test frequency. This ordering is evidence about one fitted sample, not a universal ranking. The analytical Gaussian calculation gives an exact test probability near 0.5137 for the nominal decision and near 0.0572 for the unbiased KDE decision. It also gives a Gaussian chance-constrained reference objective near

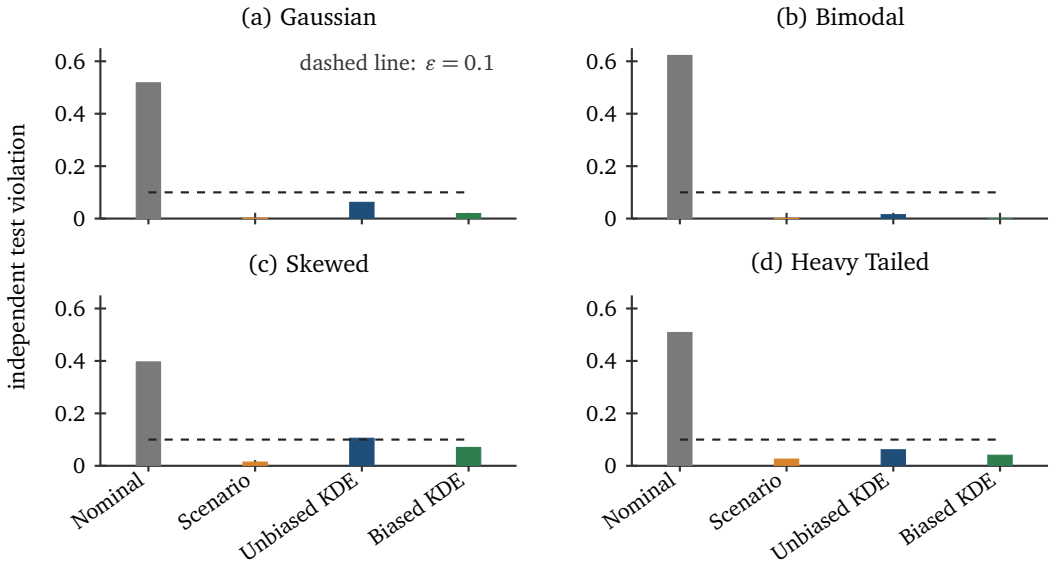


Figure 24: Independent test violation for the static nonlinear benchmark ($N_{\text{train}} = 250$, $N_{\text{test}} = 4000$, $\varepsilon = 0.1$). The dashed line is the design target.

0.1392, below the KDE objective 0.1981. The gap is consistent with smoothing and finite training data imposing extra slack.

Across the non-Gaussian laws, the pairs $(\hat{v}_{\text{test}}^{\text{unbiased}}, \hat{v}_{\text{test}}^{\text{biased}})$ are $(0.0150, 0.0003)$ for the bimodal law, $(0.1053, 0.0700)$ for the skewed law, and $(0.0613, 0.0405)$ for the heavy-tailed law. The skewed unbiased row exceeds the target. Its 95% test interval is approximately $[0.0959, 0.1152]$, so the data do not establish a clear separation from 0.1, but they also do not support calling the method safe. The scenario method fails its nonlinear program on the skewed and heavy-tailed rows despite low recorded test frequencies in the finite sample. This is a useful warning: a low test count from a failed run is not evidence of a feasible optimizer.

The four panels in Figure 24 display these frequencies. The dashed line is only the design target. Figure 25 shows the objective–violation trade-off. In both figures, successful and failed solver rows must be read together with the status field. The figures do not encode statistical confidence, repeated seeds, or a correction for trying four methods across four distributions.

The qualitative pattern is compatible with the motivation for KDE in process systems and data-driven chance constraints (Calfa et al., 2015): a residual-space estimator can represent multimodality without committing to a single Gaussian family. It does not follow that KDE dominates scenario optimization in every regime. Scenario optimization has a different guarantee architecture, with explicit sample-complexity conditions

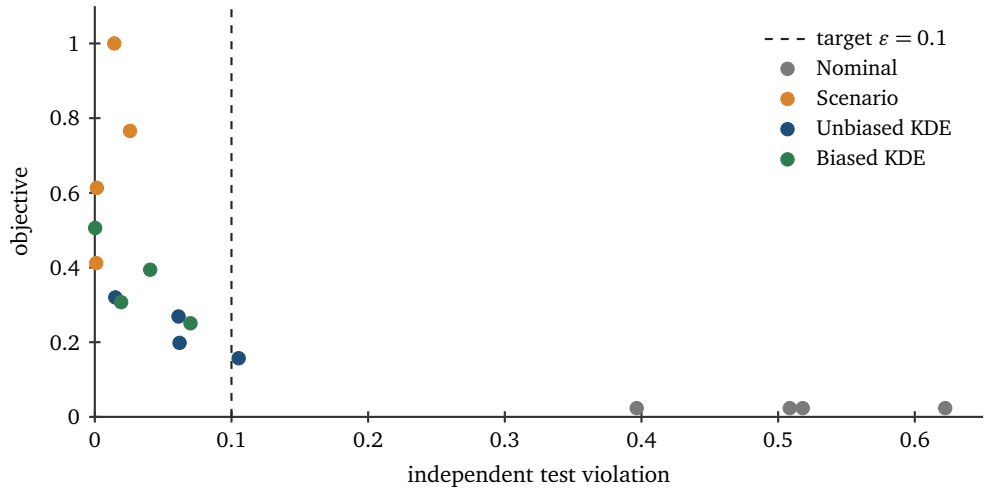


Figure 25: Objective versus independent test violation for the four static residual laws. Solver status and estimator identity are reported in the accompanying ledger.

for convex programs (Calafiore and Campi, 2006; Campi and Garatti, 2008). The local nonlinear problem is outside the setting in which those results can simply be quoted.

9.3 Paper-Aligned Hyperparameter Experiment

The first question is whether the qualitative conclusions survive a bandwidth and sample-size regime that is recognizably related to the literature. The convergence work of Schuster (2025) does not prescribe a finite numerical bandwidth; it requires a shrinking bandwidth with increasing sample size and gives a Gaussian product kernel with diagonal bandwidth as a concrete route. The related numerical work uses the normal-reference scaling $h_N = 1.06\hat{\sigma}_N N^{-1/5}$ in one dimension and large independent Monte Carlo samples (Schuster et al., 2022), where $\hat{\sigma}_N$ is the sample standard deviation of the N training samples to which the bandwidth is applied. Keil et al. (2020, 2021); Keil and Rao (2021) specify a different object: the one-sided Epanechnikov bias $b(h) = h$, risk levels $\varepsilon_a = 0.1$ and $\varepsilon_b = 0.01$, and continuation in bandwidth and sample size inside an LGR optimal-control calculation.

The local profile implements only the portable parts, as a paper-aligned unbiased KDE and a paper-aligned biased KDE. It uses nested IID prefixes of $N \in \{250, 1000, 5000\}$, a fixed independent test set of 10,000 observations, the scalar Schuster bandwidth scaling, with $d = 1$ the dimension of the estimated variable (the scalar residual) and $\hat{\sigma}_N$ computed from the first uncertainty coordinate of the training prefix in the static

rows and from the renewable-error prefix in the dispatch rows,

$$h_N = \hat{\sigma}_N \left(\frac{4}{(d+2)N} \right)^{1/(d+4)}, \qquad d = 1, \tag{61}$$

and the one-sided Epanechnikov translation $b(h) = h$ of Keil et al. The paper-aligned biased KDE tries bandwidth multipliers $(0.25, 0.5, 1, 2)$ and warm-starts each attempt. It does not use Hamiltonian Monte Carlo (HMC), the 50,000-sample Keil chain, LGR nodes, SNOPT, or mesh-error triggers. The output is consequently a scale-normalized local comparison, not a reproduction of either paper. The paper-aligned biased KDE rows (Table 3, Figures 26 and 28) and the biased KDE rows of the main runs (Tables 2, 4, and 6) use the same shifted kernel with $b(h) = h$ and differ in the bandwidth rule and continuation, which are Schuster scaling with the multiplier continuation in this profile and a single Silverman bandwidth without bandwidth continuation in the main rows.

The question is whether increasing N makes the two estimators approach the same fitted decision. In the Gaussian static row, the paper-aligned unbiased KDE objective changes from 0.2270 at $N = 250$ to 0.2016 and 0.1599 at $N = 1000$ and $N = 5000$, with test violations 0.0464, 0.0586, and 0.0834. The paper-aligned biased KDE objective is 0.3623, 0.3077, and 0.4100, with test violations 0.0138, 0.0228, and 0.0091. The observation is not monotone convergence: the Gaussian decisions move toward the empirical target as smoothing decreases, whereas the biased decisions remain more conservative and change bandwidth branch at $N = 5000$. The mechanism is compatible with the one-sided support shift: $b(h) = h$ eliminates optimistic contributions from positive residuals, while the Gaussian estimator allows smooth mass on both sides of the threshold. Because the biased arm also changes the selected bandwidth and continuation branch, this comparison does not identify the causal effect of kernel bias alone.

The distribution-shape comparison gives the same analytical distinction. At $N = 5000$, paper-aligned unbiased KDE test violations are 0.0548 for bimodal, 0.0910 for skewed, and 0.0794 for heavy-tailed residuals. The corresponding paper-aligned biased KDE values are 0, 0.0383, and 0.0275, with objectives 0.8223, 0.4677, and 0.7202. Figure 26 shows the held-out violations of both sequences for all four laws. The paper-aligned biased KDE is safer on this test split but more conservative. This is consistent with the pointwise upper-bound construction of Keil et al., but it does not validate Schuster's optimizer convergence theorem: no uniform residual-grid error was com-

Table 3: Paper-aligned Gaussian static sequence. The test set contains 10,000 fixed independent observations.

	Unbiased KDE			Biased KDE		
N	h	f	v	h	f	v
250	0.3476	0.2270	0.0464	0.3476	0.3623	0.0138
1000	0.2740	0.2016	0.0586	0.2740	0.3077	0.0228
5000	0.1925	0.1599	0.0834	0.3851	0.4100	0.0091

Table 4: Two-generator dispatch benchmark. The uncertain balance residual is tested on an independent sample.

Method	Objective	Train	Test	Solver
Nominal	928.5686	0.5880	0.5788	success
Scenario	935.3871	0.0040	0.0060	success
Unbiased KDE	933.9233	0.0520	0.0398	success
Biased KDE	934.8865	0.0120	0.0110	success

puted, the feasible decision was not shown to converge, and the static objective does not satisfy all of Schuster's assumptions.

Table 3 answers a narrower question than Schuster's theorem. It shows how the chosen finite-dimensional solver behaves along one nested sample path. It does not show almost-sure convergence, because three values of N and one seed cannot establish a limit. Conversely, the lower violations of the biased rows are not a contradiction of Schuster; they result from changing the estimator and deliberately imposing the Keil safety shift. This distinction is the central interpretive rule for the remaining experiments.

9.4 Energy Dispatch and Exact Distributional Check

The dispatch proxy has two generators and a balance residual. Capacity residuals are deterministic in this adapter, so the stochastic problem tests only renewable forecast error in the balance equation. The error is the mixture $0.6\mathcal{N}(-0.25, 0.10^2) + 0.4\mathcal{N}(0.35, 0.15^2)$. Table 4 reports the objective values and held-out frequencies.

Because the mixture is known, its CDF supplies an independent reference. The exact mixture probability at the nominal decision is approximately 0.5951, while the Monte Carlo test frequency is 0.5788. The corresponding probabilities for scenario, unbiased KDE, and biased KDE decisions are approximately 0.0043, 0.0450, and 0.0107. The

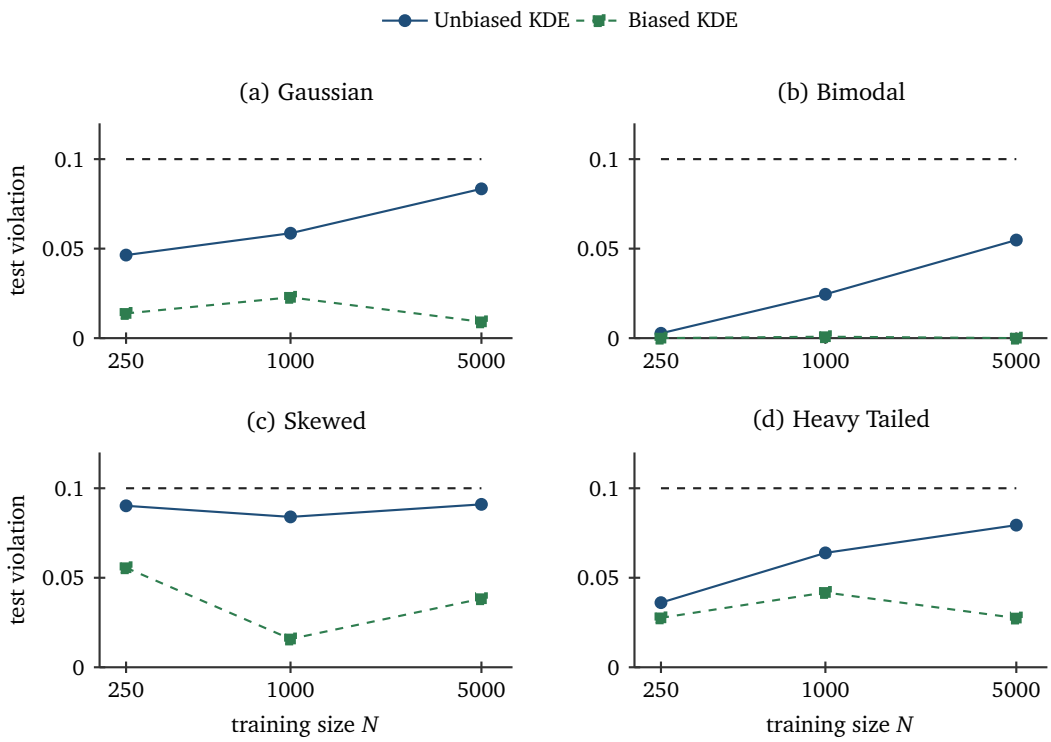


Figure 26: Paper-aligned static sequence. Lines show held-out violation for the paper-aligned unbiased KDE and the paper-aligned biased KDE over nested training sizes. The dashed line is $\varepsilon = 0.1$; it is a target, not a confidence bound.

training values in the table are computed on the optimization sample and the test values on an independent sample; they must not be conflated. The sample frequencies are therefore plausible finite-sample measurements, and the objective ranking is not an artifact of an unknown test distribution. The scenario solution is most conservative; the biased KDE is closer to it than the unbiased KDE in violation while retaining slightly lower objective than scenario. This is a single synthetic balance experiment and says nothing about network congestion, unit commitment, temporal correlation, or capacity uncertainty.

9.5 Bandwidth, Sample Size, and Joint Statistics

This section collects three smaller diagnostics: the sensitivity of a fixed-residual estimate to bandwidth and sample size, the effect of the joint-event statistic, and the paper-aligned sample-size sequence for the dispatch adapter.

Table 5: Fixed-residual sensitivity values. The upper row scales the bandwidth at the baseline training size; the lower row varies the training size, with the bandwidth recomputed by Silverman's rule for each size.

Bandwidth factor	0.25	0.5	1	2	4	Test
Estimated violation	0.3765	0.3808	0.3894	0.4018	0.4234	0.3950
Training size	50	100	250	500	1000	2000
Estimated violation	0.3972	0.4284	0.3894	0.3754	0.3829	0.3809

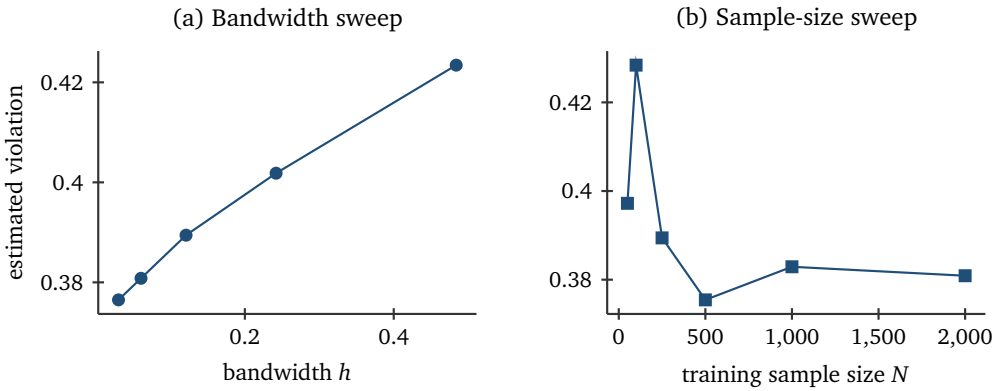


Figure 27: Sensitivity of a fixed-residual violation estimate to bandwidth and training sample size. The experiment does not reoptimize the decision.

Bandwidth sensitivity. The sensitivity experiment holds a scalar residual split fixed and changes only the estimator input. It is deliberately not an end-to-end reoptimization. The baseline bandwidth is $h_0 = 0.120976$ and the fixed test frequency is 0.3950. Table 5 lists the values. The curves in Figure 27 show a smooth bandwidth response but no monotone sample-size law. That is expected for a plug-in estimator evaluated at a fixed decision and a finite random sample; it is not evidence against consistency.

Joint-event statistics. The joint experiment compares statistics rather than optimized risk allocations. Equal-Boole allocation and the exact maximum statistic use the same Boolean event in the current runner, so their held-out rates coincide at 0.5530. The log-sum-exp statistic with $\tau = 0.1$ gives 0.5745. These values are computed on the held-out half of a 4000-sample pool; a separate ledger field averages over the full pool and must not be confused with the test rate. Component rates are approximately 0.3328 and 0.3255. The joint experiment therefore demonstrates how a smooth upper-envelope statistic changes the measured event, but it does not demonstrate a Boole allocation theorem or a joint chance-constrained optimizer. An actual allocation study

would impose separate component constraints and report the union event on an independent test set.

Dispatch sample-size sequence. The dispatch sequence provides a second check because its mixture CDF is known exactly. For the paper-aligned unbiased KDE, the objective decreases from 934.1317 at $N = 250$ to 933.6669 and 933.4633 at $N = 1000$ and $N = 5000$; test violations are 0.0347, 0.0646, and 0.0798. For the paper-aligned biased KDE, the objective is 937.3410, 936.1048, and 935.2208, with test violations 0, 0.0009, and 0.0069. Figure 28 plots both sequences. The trend is more stable than in the static problem because the balance residual is affine and the dispatch objective is convex and increasing on the nonnegative box. This is the local problem closest to Schuster's structural assumptions, although the adapter still has only one stochastic balance residual and no repeated-seed convergence study. The higher objective and lower violation of the paper-aligned biased KDE are the expected price of the pointwise upper envelope, not evidence that the Gaussian estimator is inconsistent.

The dispatch results also answer a practical question: does a paper-aligned bandwidth rule merely alter the estimator, or does it move the decision between the regimes of Table 4? The held-out violations of the paper-aligned unbiased KDE sequence, 0.0347–0.0798, lie between the nominal and scenario test frequencies of 0.5788 and 0.0060, while those of the paper-aligned biased KDE sequence, 0–0.0069, stay close to the scenario level. Exact mixture probabilities were evaluated only for the four decisions of Table 4, so this comparison rests on held-out frequencies. The conclusion is therefore about a trade-off between smoothness and one-sided protection. It is not a claim that either method dominates under temporal renewable uncertainty, since the adapter contains no time correlation or network constraints.

9.6 Lunar Continuation Record

The lunar records are included to make the experimental chain auditable. Table 6 lists the stages.

The table should be read with the formulation audit in Chapter 7. The nominal result is unsafe. The robust rows are failed NLP outputs, and their terminal frequencies do not validate the path event. The terminal-position test uses the Gaussian disturbance only; no saved control trajectory was passed through an independent all-time path test. The stored nominal value below 32.44 is explained by the mismatch in Eq. (58).

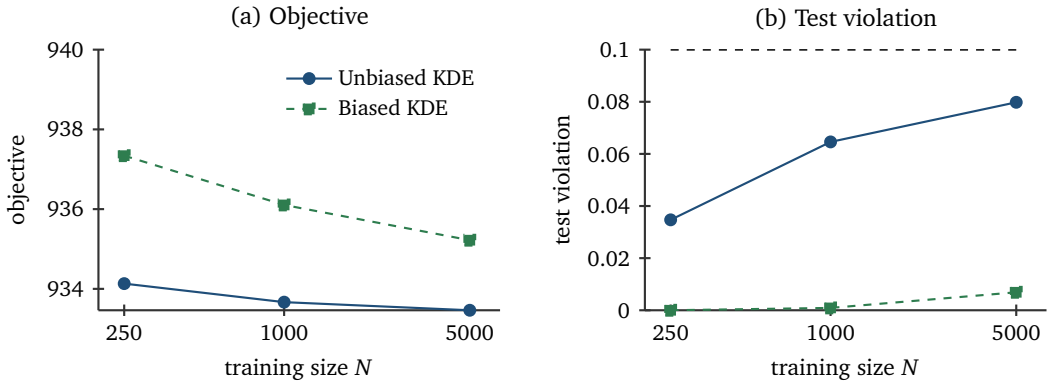


Figure 28: Dispatch sequence for the paper-aligned unbiased KDE and the paper-aligned biased KDE. (a) Objective and (b) held-out balance violation for nested training sizes.

Table 6: Local lunar continuation. The objective is the stored trapezoidal transcription value; robust stages have failed SLSQP status. Validation is terminal-position frequency only.

Stage	Stored f	Train terminal	Test terminal	Status
Nominal	29.8052	0.5080	0.5143	success
Worst-case	29.8052	0.5080	0.5143	failure
Scenario	31.3685	0.0400	0.0255	failure
Unbiased KDE	32.4400	0.1080	0.1128	failure
Biased KDE	32.4400	0.0520	0.0548	failure

These observations rule out a cost claim and a continuous-time safety claim, while preserving the value of the run as a reproducibility and failure-analysis record.

The published costs of around 9.09 (Keil et al., 2020, 2021) come from a different formulation (Section 7.3), so the local rows cannot confirm or refute them. They show instead that a locally assembled pipeline must expose transcription invariants and solver status before numerical comparisons are meaningful.

9.7 Evidence Summary

The observations in this chapter are conditional on the declared synthetic laws, seeded sample splits, and solver statuses. The claim contract and validation layers that govern how these observations may be read are defined in Sections 8.1 and 8.3; Chapter 10 interprets them.

10 Discussion: Evidence and Method Selection

This chapter separates what the experiments establish from what remains a methodological hypothesis. The appropriate choice depends on the residual dimension, estimator, bandwidth and bias, solver status, and independent validation scope. A low held-out frequency is not a safety certificate when the optimization gate has failed or when the fitted decision was selected from the same data without a selection-aware bound.

10.1 Answers to the Research Questions

The four research questions stated in Section 1.5.2 are answered in turn, each with the evidence that supports it and the qualification under which the answer holds.

RQ1: Formulation. Chapters 4–6 establish the scalar residual reformulation and its analytic gradient. For an individual event, estimating the pushforward residual CDF avoids direct KDE in the uncertainty dimension. The claim is conditional on the regularity and convergence assumptions stated in Chapter 6; it is not a finite-sample population guarantee for the optimizer returned by the NLP.

RQ2: Numerical comparison. The static and dispatch experiments show a local objective–violation trade-off among nominal, scenario, unbiased KDE, and biased KDE formulations. For the Gaussian static row, the analytical Gaussian chance-constrained reference objective (0.1392) lies below the unbiased KDE objective (0.1981), consistent with smoothing and finite training data adding slack. The dispatch adapter is the strongest transfer check because its mixture CDF is known independently. The corrected training/test split in Table 4 must be read together with solver status. These are seeded synthetic comparisons, not universal rankings. Scenario guarantees require convexity, IID sampling, support-rank and confidence-level conditions that are not calculated by the local runner.

RQ3: Bandwidth and smoothing. The paper-aligned records show how finite choices of N and h move the returned decision. The biased rows have lower held-out violation and higher objective in these runs, but the implementation also changes the selected bandwidth and continuation branch. The observation therefore supports a

local safety–cost pattern, not a causal estimate of the bias alone and not a convergence result.

RQ4: Joint extension. The maximum statistic represents the joint event exactly. Equal-Boole and maximum rows coincide in the current runner because they evaluate the same Boolean event; this does not test componentwise risk allocation. The log-sum-exp statistic satisfies $s_\tau \geq s$ and therefore changes the event being measured. Its observed rate is a diagnostic of that changed statistic, not evidence for a joint optimizer or a probability margin.

10.2 Relation to the Anchor Literature

The paper-aligned unbiased KDE sequence probes the direction of the Schuster convergence hypothesis. It does not compute uniform residual-grid error, verify the theorem's geometric assumptions, or establish a limiting optimizer. The dispatch adapter is the closest local analogue to the convex setting; the lunar transcription is not.

The biased KDE arm tests the one-sided kernel property with IID samples and finite-dimensional SLSQP. The lunar adapter additionally uses an Euler mesh, whereas the cited Keil workflow uses MCMC, LGR, and continuation. The biased KDE arm is therefore an empirical analogue, not a replication of that workflow.

10.3 Mechanistic Synthesis

The observations of Chapter 9 are traced below to four mechanisms: statistical, optimization, transfer and solver, and joint-event.

Statistical mechanism. Symmetric smoothing can credit positive residuals near the boundary. The $b(h) = h$ Epanechnikov shift removes this per-sample optimism for its compact support, but the resulting inequality is an expectation statement for a fixed, sample-independent decision. Calibration or a selection-aware concentration bound is still needed after optimization.

Optimization mechanism. The active estimated constraint and the objective determine the returned decision together. The observed biased premium is therefore a

property of the full estimator–bandwidth–continuation pipeline. A controlled ablation must hold h and the continuation schedule fixed before attributing an objective or violation change to the kernel bias.

Transfer and solver mechanism. The exact dispatch CDF provides a distributional oracle for the affine balance residual. In contrast, the robust lunar stages have failed solver status and terminal-only validation; the Euler/trapezoidal mismatch also changes the stored objective. Those rows are valuable audit records but cannot support a physical cost or continuous-time safety claim.

Joint-event mechanism. The log-sum-exp upper bound smooths the max while introducing a different scalar event. A genuine allocation study must retain component residuals, optimize separate component constraints, and validate the union event independently.

10.4 Method-Selection Boundaries

Scenario optimization is appropriate when its convexity, IID, support-rank, and confidence assumptions are checked. A parametric reformulation is appropriate when diagnostics support the assumed law and a deterministic equivalent is available.

Unbiased KDE is defensible for low-dimensional empirical residuals when smooth derivatives are valuable and bandwidth, solver margins, and independent test performance are reported. One-sided compact kernels are defensible when the fixed-decision envelope and its objective premium justify the additional conservatism. High-dimensional, rare-event, dependent, or shifted settings require distributionally robust, importance-sampling, or tail-specific methods before direct KDE is used.

10.5 Future Work

The results identify a focused doctoral research program. Each item below is a research target rather than a result established by this thesis.

Selection-aware finite-sample reliability. Chapter 5 gives an expectation envelope for a fixed decision. The principal next step is to make a corresponding statement for

the decision returned after the same data have been used to fit the KDE and solve the nonlinear program. An initial, conservative route is to separate training, calibration, and test data: the optimizer uses the training sample, while an independent calibration sample sets a lower confidence bound on the achieved safe probability. A sharper route would prove a uniform bound over the decision class and a prespecified grid of bandwidths, temperatures, and component weights. The bound should separate sampling error, one-sided kernel bias, and the log-sum-exp approximation, and it should state explicitly how its complexity grows with the number m of joint constraints. Such a result would turn the current fixed-decision envelope into a post-selection safety statement with confidence level $1-\delta$.

The proposed target can be stated using the scalar statistic s and its log-sum-exp approximation s_τ from Chapter 5. For a training sample $\xi_1, \dots, \xi_N$, write the corresponding biased estimator as

$$\hat{p}^{\,b}_{N,h,\tau}(x) = \frac{1}{N}\sum_{j=1}^{N} F_K\left(\frac{-s_\tau(x,\xi_j) - b(h)}{h}\right), \qquad p(x) = \mathbb{P}(s(x,\xi) \le 0). \tag{62}$$

The theorem to seek is a post-selection implication, uniform over x and over a prespecified tuning set $\mathscr{H} \times \mathscr{T}$:

$$\hat{p}^{\,b}_{N,h,\tau}(\hat{x}) \ge 1-\varepsilon+\Delta_{N,h,\tau}(\delta) \implies p(\hat{x}) \ge 1-\varepsilon \quad \text{with probability at least } 1-\delta. \tag{63}$$

Here the required margin is a research object, not a quantity established in this thesis. A possible decomposition is only a conjectural template: the $\tau \log m$ term is a residual-statistic margin and becomes a probability margin only under an anti-concentration condition. The constants, exponent, and complexity term must be derived rather than assumed:

$$\Delta_{N,h,\tau}(\delta) = \underbrace{C_{\mathrm{stat}}\sqrt{\frac{\mathscr{C}(\mathscr{H},\mathscr{T},X)+\log(1/\delta)}{N}}}_{\text{sampling and selection}} + \underbrace{C_{\mathrm{kde}}h^q}_{\text{kernel smoothing}} + \underbrace{C_{\mathrm{lse}}\tau\log m}_{\text{joint smoothing}}, \tag{64}$$

where the constants and the complexity term would have to be derived from explicit residual regularity, anti-concentration, and function-class assumptions. An independent calibration theorem is a rigorous first milestone; a same-sample result with a non-vacuous complexity term is the stronger doctoral target.

Conservatism and objective cost. Safety is useful only if its objective cost is quantified. Under a density regularity or anti-concentration condition near the residual boundary, future work should bound the probability mass introduced by bandwidth h and temperature τ, then propagate that bound through an objective error bound or sharp-minimum condition. This would provide a principled rule for choosing (h, τ) and would establish when the biased construction approaches the true optimum as N increases. The claim must remain conditional: uniqueness alone is insufficient without the compactness, coercivity, or error-bound assumptions used in the optimization argument.

With an appropriate objective error bound, the desired conclusion would take the form

$$0 \leq f(\hat{x}_{N,h,\tau}) - f(x^*) \leq C_{\text{opt}}\, \Delta_{N,h,\tau}(\delta), \tag{65}$$

for a solution x^* of the exact problem. Establishing this inequality, rather than merely observing a higher objective in a finite experiment, would quantify the price of one-sided protection.

Continuous-time chance constraints. The lunar experiment currently evaluates a discretized path. A stronger control result would repair the transcription and quadrature first, then derive a mesh-to-trajectory residual bound from temporal regularity and integrator error. The statistical safety margin could subsequently be combined with that discretization margin to control the probability of a violation at any time in the horizon. This requires stating the dependence structure of the disturbance trajectory and validating the complete path on a refined mesh; nodewise test frequencies cannot establish this claim.

Validation and transfer. The theoretical targets should be evaluated with independent repetitions rather than a single seed. Static and dispatch problems with exact probability oracles should report coverage, excess violation, objective regret, solver status, and runtime over a factorial grid of N, h, τ, and m. The joint experiment should optimize the componentwise event itself, rather than only compare scalar statistics. Finally, dependence, distribution shift, and a public or semi-real benchmark are needed to determine whether the observed conservatism survives outside the present synthetic setting.

11 Conclusion

This thesis studies chance-constrained optimization when the distribution of the constraint residual is learned from data. The central question is not whether KDE can produce a smooth curve, but whether that curve can be used inside an optimizer without confusing smoothness, safety, and convergence. The work follows the construction from indicator and scenario constraints, through unbiased integrated KDE, to the one-sided biased kernels associated with Keil et al. (2020, 2021), and then connects the numerical method to the convergence results of Schuster (2025).

11.1 Contributions and Answers to the Research Questions

The thesis makes six connected contributions. It gives a residual-space KDE reformulation that turns a scalar probability constraint into a deterministic average of integrated kernels, making gradients or smooth numerical evaluations available to the optimizer. It separates uncertainty generation, residual evaluation, estimator selection, nonlinear optimization, and independent validation into auditable records. It checks the pointwise one-sided property of the biased KDE surrogate with $b(h) = h$. It implements a paper-aligned profile with nested N, Schuster bandwidth scaling, Keil bias, and continuation metadata. It measures KDE trade-offs against nominal and scenario baselines on seeded synthetic nonlinear and dispatch problems. Finally, it proves a fixed-horizon fuel invariant and identifies the quadrature mismatch in the optimal-control audit. The evidential status of each contribution is given by its derivation, kernel check, local records, seeded runs, reproducibility checks, or model-level proof as appropriate.

The research-question answers are conditional. For RQ1, Chapters 4–6 establish the residual-space reformulation and its analytic gradient under the regularity and convergence assumptions of Chapter 6, not as a finite-sample population guarantee for the optimizer returned by the NLP. For RQ2, the seeded synthetic experiments show a local objective–violation trade-off among nominal, scenario, unbiased KDE, and biased KDE formulations, not a universal ranking. For RQ3, finite choices of bandwidth and sample size move the returned decisions, but three sample sizes do not constitute a convergence proof; the biased rows are more conservative, although the implementation also changes the selected bandwidth and continuation branch. For RQ4, the maximum statistic represents the joint event exactly, whereas the log-sum-exp statistic changes the event being measured, so its observed rate is a diagnostic of that changed statistic only.

The principal observations are equally conditional. Nominal test violations lie between about 0.40 and 0.62, showing that mean or deterministic surrogates miss tail geometry in these synthetic models. The paper-aligned unbiased KDE sequence shows a finite-sample smoothing trade-off as the target is approached, but it does not verify uniform convergence. The paper-aligned biased KDE sequence has lower violation and higher objective in the local runs, which is evidence of one-sided conservatism rather than an MCMC/LGR replication. The joint experiment measures only how the log-sum-exp statistic shifts the event, so no componentwise risk-allocation result follows. The lunar robust stages fail SLSQP; solver feasibility therefore precedes any test-frequency interpretation, and no landing-safety claim is made. Chapter 10 develops these interpretations and their boundaries in detail.

The comparison with the literature is consequently complementary. Schuster supplies the optimization-level convergence question: an approximate solution sequence is meaningful only when the KDE converges in the required sense and the limiting optimization problem has the necessary regularity. Keil et al. supply a constructive safety-oriented kernel and a continuation strategy for a different optimal-control transcription. The local results do not contradict either paper. They show where the hypotheses become operational: an active constraint, a finite sample, a failed solver, an absent mesh-error trigger, or a mismatched quadrature rule changes the claim that can be made.

11.2 Limitations

The local evidence is synthetic, low-dimensional, and based on one primary training seed per reported optimization row (Section 9). A held-out frequency is a conditional Monte Carlo measurement for the returned decision; its point estimate alone is not a finite-sample certificate. The biased domination property gives the training-sample envelope

$$\hat{p}_h^b(\hat{x}) \leq \frac{1}{N} \sum_{j=1}^{N} \mathbf{1}\{g(\hat{x}, \xi_j) \leq 0\}, \tag{66}$$

but it does not give a population guarantee for the data-dependent optimizer. Chapter 6 also leaves convergence rates, biased-KDE optimizer convergence, nonconvex guarantees, and finite-sample feasibility outside its theorems (Sections 6.6 and 6.7). The reported rows therefore support empirical comparisons under the declared protocol, not universal estimator rankings.

The joint experiment evaluates scalar event statistics; its equal-Boole and maximum rows coincide and do not establish componentwise risk allocation (Sections 5.5 and 9.5). The dispatch adapter has one stochastic balance residual and no temporal or network dependence. The lunar rows are failed robust NLP stages and validate terminal position only (Table 6); nodewise frequencies do not imply an all-time path guarantee. Finally, the Euler defects imply a fixed-horizon fuel invariant, whereas the stored trapezoidal objective contains an endpoint term (Eqs. (56) and (58)). Correcting the quadrature removes that transcription artifact, but the fixed-horizon model still cannot rank physical fuel use; the stored costs remain diagnostics until the objective or horizon is reformulated.

11.3 Future Work

The technical roadmap is specified in Chapter 10 (Section 10.5).

Taken together, the results place residual-space KDE in a specific methodological gap: empirical or non-Gaussian uncertainty with a low-dimensional residual and a solver that benefits from smooth constraints. Its use is credible only when estimator, bandwidth, sampling, solver status, and validation scope are reported together; stronger safety or convergence claims remain open questions identified in Chapters 6 and 10.

References

Jorge Alvarez-Mena and Onésimo Hernández-Lerma. Convergence and approximation of optimization problems. *SIAM Journal on Optimization*, 15(2):527–539, 2005. doi: 10.1137/S1052623402413938.

Charles Audet and Warren Hare. *Derivative-Free and Blackbox Optimization*. Springer, Cham, 2017. doi: 10.1007/978-3-319-68913-5.

Aharon Ben-Tal, Laurent El Ghaoui, and Arkadi Nemirovski. *Robust Optimization*. Princeton Series in Applied Mathematics. Princeton University Press, Princeton, NJ, 2009.

Daniel Bienstock, Michael Chertkov, and Sean Harnett. Chance-constrained optimal power flow: risk-aware network control under uncertainty. *SIAM Review*, 56(3): 461–495, 2014. doi: 10.1137/130910312.

John R. Birge and François Louveaux. *Introduction to Stochastic Programming*. Springer, New York, second edition, 2011. doi: 10.1007/978-1-4614-0237-4.

Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, Cambridge, 2004. doi: 10.1017/CBO9780511804441.

Jean-Baptiste Caillau, Max Cerf, Achille Sassi, Emmanuel Trélat, and Hasnaa Zidani. Solving chance constrained optimal control problems in aerospace via kernel density estimation. *Optimal Control Applications and Methods*, 39(5):1833–1858, 2018. doi: 10.1002/oca.2445.

Giuseppe C. Calafiore and Marco C. Campi. The scenario approach to robust control design. *IEEE Transactions on Automatic Control*, 51(5):742–753, 2006. doi: 10.1109/TAC.2006.875041.

Bruno A. Calfa, Ignacio E. Grossmann, Anshul Agarwal, Scott J. Bury, and John M. Wassick. Data-driven individual and joint chance-constrained optimization via kernel smoothing. *Computers & Chemical Engineering*, 78:51–69, 2015. doi: 10.1016/j.compchemeng.2015.04.012.

Marco C. Campi and Simone Garatti. The exact feasibility of randomized solutions of uncertain convex programs. *SIAM Journal on Optimization*, 19(3):1211–1230, 2008. doi: 10.1137/07069821X.

Abraham Charnes and William W. Cooper. Chance-constrained programming. *Management Science*, 6(1):73–79, 1959. doi: 10.1287/mnsc.6.1.73.

Abraham Charnes and William W. Cooper. Deterministic equivalents for optimizing and satisficing under chance constraints. *Operations Research*, 11(1):18–39, 1963.

doi: 10.1287/opre.11.1.18.

Abraham Charnes, William W. Cooper, and Gifford H. Symonds. Cost horizons and certainty equivalents: an approach to stochastic programming of heating oil. *Management Science*, 4(3):235–263, 1958. doi: 10.1287/mnsc.4.3.235.

Zhi Chen, Daniel Kuhn, and Wolfram Wiesemann. Data-driven chance constrained programs over Wasserstein balls. *Operations Research*, 72(1):410–424, 2024. doi: 10.1287/opre.2022.2330.

Okan Ciftci. Microgrid energy management with flexibility constraints: A data-driven solution method. Master's thesis, Louisiana State University, 2017.

Okan Ciftci, Mahdi Mehrtash, and Amin Kargarian. Data-driven nonparametric chance-constrained optimization for microgrid energy management. *IEEE Transactions on Industrial Informatics*, 16(4):2447–2457, 2020. doi: 10.1109/TII.2019.2932078.

Rama Cont. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001. doi: 10.1080/713665670.

George B. Dantzig. Linear programming under uncertainty. *Management Science*, 1 (3-4):197–206, 1955. doi: 10.1287/mnsc.1.3-4.197.

Luc Devroye and László Györfi. *Nonparametric Density Estimation: The L_1 View*. Wiley, New York, 1985.

V. A. Epanechnikov. Non-parametric estimation of a multivariate probability density. *Theory of Probability and its Applications*, 14(1):153–158, 1969. doi: 10.1137/1114019.

Bharath Gopalakrishnan, Arun Kumar Singh, K. Madhava Krishna, and Dinesh Manocha. Solving chance constrained optimization under nonparametric uncertainty through Hilbert space embedding. arXiv preprint, 2018. arXiv:1811.09311.

Martin Gugat and Michael Schuster. Stationary gas networks with compressor control and random loads: optimization with probabilistic constraints. *Mathematical Problems in Engineering*, 2018:7984079, 2018. doi: 10.1155/2018/7984079.

Rachel E. Keil and Anil V. Rao. Warm start method for solving chance constrained optimal control problems using biased kernel density estimators. *Journal of Dynamic Systems, Measurement, and Control*, 143(12):124502, 2021. doi: 10.1115/1.4052173.

Rachel E. Keil, Alexander T. Miller, Mrinal Kumar, and Anil V. Rao. Method for chance constrained optimal control using biased kernel density estimators. arXiv preprint, 2020. arXiv:2003.08010.

Rachel E. Keil, Alexander T. Miller, Mrinal Kumar, and Anil V. Rao. Method for solving chance constrained optimal control problems using biased kernel density estimators. *Optimal Control Applications and Methods*, 42(1):330–354, 2021. doi: 10.1002/oca.2675.

Wei Liu, Li Yang, and Bo Yu. Kernel density estimation based distributionally robust mean-CVaR portfolio optimization. *Journal of Global Optimization*, 84:1053–1077, 2022. doi: 10.1007/s10898-022-01177-5.

James Luedtke and Shabbir Ahmed. A sample approximation approach for optimization with probabilistic constraints. *SIAM Journal on Optimization*, 19(2):674–699, 2008. doi: 10.1137/070702928.

Bruce L. Miller and Harvey M. Wagner. Chance constrained programming with joint constraints. *Operations Research*, 13(6):930–945, 1965. doi: 10.1287/opre.13.6.930.

John A. Nelder and Roger Mead. A simplex method for function minimization. *The Computer Journal*, 7(4):308–313, 1965. doi: 10.1093/comjnl/7.4.308.

Arkadi Nemirovski and Alexander Shapiro. Convex approximations of chance constrained programs. *SIAM Journal on Optimization*, 17(4):969–996, 2006. doi: 10.1137/050622328.

Bernardo K. Pagnoncelli, Shabbir Ahmed, and Alexander Shapiro. Sample average approximation method for chance constrained programming: theory and applications. *Journal of Optimization Theory and Applications*, 142(2):399–416, 2009. doi: 10.1007/s10957-009-9523-6.

Emanuel Parzen. On estimation of a probability density function and mode. *Annals of Mathematical Statistics*, 33(3):1065–1076, 1962. doi: 10.1214/aoms/1177704472.

Alejandra Peña-Ordieres, James R. Luedtke, and Andreas Wächter. Solving chance-constrained problems via a smooth sample-based nonlinear approximation. *SIAM Journal on Optimization*, 30(3):2221–2250, 2020. doi: 10.1137/19M1261985.

András Prékopa. Logarithmic concave measures with application to stochastic programming. *Acta Scientiarum Mathematicarum*, 32:301–316, 1971.

András Prékopa. *Stochastic Programming*. Kluwer Academic Publishers, Dordrecht, 1995.

András Prékopa. Probabilistic programming. In Andrzej Ruszczyński and Alexander Shapiro, editors, *Stochastic Programming*, volume 10 of *Handbooks in Operations Research and Management Science*, pages 267–351. Elsevier, 2003. doi: 10.1016/S0927-0507(03)10005-9.

Xiaotong Qi, Wei Liu, Bo Yu, and Li Yang. Distributionally robust chance-constrained optimal load shedding model for active distribution networks based on KDE. Optimization Online preprint, 2026. URL https://optimization-online.org/2026/04/distributionally-robust-chance-constrained-optimal-load-shedding-model-for-active-distribution-networks-based-on-kde/. Optimization Online ID 34324.

R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000. doi: 10.21314/JOR.2000.038.

Murray Rosenblatt. Remarks on some nonparametric estimates of a density function. *Annals of Mathematical Statistics*, 27(3):832–837, 1956. doi: 10.1214/aoms/1177728190.

Sam L. Savage. *The Flaw of Averages: Why We Underestimate Risk in the Face of Uncertainty*. Wiley, Hoboken, NJ, 2009.

Henry Scheffé. A useful convergence theorem for probability distributions. *Annals of Mathematical Statistics*, 18(3):434–438, 1947. doi: 10.1214/aoms/1177730390.

Michael Schuster. On the convergence of optimization problems with kernel density estimated probabilistic constraints. *Applied Set-Valued Analysis and Optimization*, 7 (2):209–221, 2025. doi: 10.23952/asvao.7.2025.2.05.

Michael Schuster, Elisa Strauch, Martin Gugat, and Jens Lang. Probabilistic constrained optimization on flow networks. *Optimization and Engineering*, 23:1–50, 2022. doi: 10.1007/s11081-021-09619-x.

David W. Scott. *Multivariate Density Estimation: Theory, Practice, and Visualization*. Wiley, New York, 1992. doi: 10.1002/9780470316849.

Alexander Shapiro, Darinka Dentcheva, and Andrzej Ruszczyński. *Lectures on Stochastic Programming: Modeling and Theory*. SIAM, Philadelphia, PA, second edition, 2014. doi: 10.1137/1.9781611973433.

Bernard W. Silverman. *Density Estimation for Statistics and Data Analysis*. Chapman & Hall, London, 1986.

Allen L. Soyster. Convex programming with set-inclusive constraints and applications to inexact linear programming. *Operations Research*, 21(5):1154–1157, 1973. doi: 10.1287/opre.21.5.1154.

Wim van Ackooij and René Henrion. Gradient formulae for nonlinear probabilistic constraints with Gaussian and Gaussian-like distributions. *SIAM Journal on Optimization*, 24(4):1864–1889, 2014. doi: 10.1137/130922689.

Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006. doi: 10.1007/s10107-004-0559-y.

Can Wan, Changfei Zhao, and Yonghua Song. Chance constrained extreme learning machine for nonparametric prediction intervals of wind power generation. *IEEE Transactions on Power Systems*, 35(5):3869–3884, 2020. doi: 10.1109/TPWRS.2020.2986282.

Matt P. Wand and M. Chris Jones. *Kernel Smoothing*. Chapman & Hall, London, 1995. doi: 10.1007/978-1-4899-4493-1.

Jingyao Wang, Cheng Wang, Yile Liang, Tao Bi, Miadreza Shafie-khah, and Joao P. S. Catalão. Data-driven chance-constrained optimal gas-power flow calculation: a Bayesian nonparametric approach. *IEEE Transactions on Power Systems*, 36(5): 4683–4698, 2021. doi: 10.1109/TPWRS.2021.3065465.

Dominik Wied and Rafael Weißbach. Consistency of the kernel density estimator: a survey. *Statistical Papers*, 53(1):1–21, 2012. doi: 10.1007/s00362-010-0338-1.

Chutian Wu and Amin Kargarian. Computationally efficient data-driven joint chance constraints for power systems scheduling. *IEEE Transactions on Power Systems*, 38 (3):2858–2869, 2023. doi: 10.1109/TPWRS.2022.3195127.

Chutian Wu, Amin Kargarian, and Hyun Woo Jeon. Data-driven nonparametric joint chance constraints for economic dispatch with renewable generation. *IEEE Transactions on Industry Applications*, 57(6):6537–6546, 2021. doi: 10.1109/TIA.2021.3105364.

A Core Implementation

This appendix lists the core implementation behind the methods and results of the thesis: the kernel and bandwidth functions, the KDE chance-constraint estimators, the residual maps of the three benchmark models, and the optimization layer that turns these estimators into constraints of a nonlinear program. Input validation, file input and output, result bookkeeping, and the code that produces the figures are omitted. Throughout, a residual $r = g(x, \xi) \leq 0$ is safe and $r > 0$ is a violation, as in Eq. (26). The listings are verbatim excerpts. Their line numbers are those of the source module named in each caption, so a gap in the numbering marks omitted lines.

A.1 KDE Estimators

Listing A.1 evaluates the integrated kernels F_K at a standardized argument. The Gaussian case uses $F_K = \Phi$, as in Eq. (32). The biased Epanechnikov kernel shifts the integrated Epanechnikov kernel by one bandwidth, which is the bias $b(h) = h$ in Eq. (37). Its support moves to $[0, 2]$ in standardized coordinates, so every positive residual contributes zero safe mass.

Listing A.1: Integrated Gaussian, Epanechnikov, and biased Epanechnikov kernels (`kde_cco/kde/kernels.py`)

```
54 def gaussian_cdf(u: np.ndarray | float, h: float | None = None) -> np.ndarray:
55     """Evaluate the integrated Gaussian kernel ``K_h(u)``."""
56     x = _finite_array(u, name="u")
57     if h is not None:
58         x = x / _bandwidth(h)
59     return ndtr(x)
60
61
62 def epanechnikov_cdf(u: np.ndarray | float, h: float | None = None) -> np.ndarray:
63     """Evaluate the integrated Epanechnikov kernel ``K_h(u)``."""
64     x = _finite_array(u, name="u")
65     if h is not None:
66         x = x / _bandwidth(h)
67     out = np.empty_like(x, dtype=float)
68     low = x <= -1.0
69     high = x >= 1.0
70     middle = ~(low | high)
71     out[low] = 0.0
72     out[high] = 1.0
73     z = x[middle]
74     out[middle] = 0.5 + 0.75 * (z - z**3 / 3.0)
75     return out
76
77
78 def biased_epanechnikov_cdf(u: np.ndarray | float, h: float | None = None) -> np.ndarray:
79     """Evaluate the one-sided conservative Epanechnikov integrated kernel.
```

```
80
81      The translate has support ``[0, 2]`` in standardized coordinates.  Hence
82      ``K_B(-r/h) <= 1{r <= 0}`` pointwise, so one minus its sample mean is an
83      upper surrogate for the violation probability.
84      """
85      x = _finite_array(u, name="u")
86      if h is not None:
87          x = x / _bandwidth(h)
88      return epanechnikov_cdf(x - 1.0)
```

Listing A.2 implements the two bandwidth rules. `silverman_bandwidth` is the robust rule Eq. (25) used in the main experiment rows, and `schuster_bandwidth` is the scalar Schuster scaling Eq. (61) used in the paper-aligned profile. Both keep a positive floor, so the kernel never becomes singular inside the optimizer.

Listing A.2: Silverman and Schuster bandwidth rules (`kde_cco/kde/bandwidth.py`)

```
8  def silverman_bandwidth(samples: np.ndarray, *, min_bandwidth: float = 1e-12) -> float:
9      """Return Silverman's robust rule-of-thumb bandwidth.
10
11     The baseline is ``0.9 * min(s, IQR / 1.34) * n**(-1/5)``.  If the
12     robust scale vanishes, the available positive scale is used and a unit
13     scale is used only for a constant sample.  ``min_bandwidth`` prevents a
14     singular kernel in downstream optimization.
15     """
16     x = np.asarray(samples, dtype=float)
17     if x.ndim != 1 or x.size == 0:
18         raise ValueError("samples must be a nonempty one-dimensional array")
19     if not np.all(np.isfinite(x)):
20         raise ValueError("samples must contain only finite values")
21     floor = float(min_bandwidth)
22     if not np.isfinite(floor) or floor <= 0.0:
23         raise ValueError("min_bandwidth must be positive and finite")
24     std = float(np.std(x, ddof=1)) if x.size > 1 else 0.0
25     q25, q75 = np.percentile(x, [25.0, 75.0])
26     iqr_scale = float((q75 - q25) / 1.34)
27     if std > 0.0 and iqr_scale > 0.0:
28         scale = min(std, iqr_scale)
29     else:
30         scale = max(std, iqr_scale)
31     if not np.isfinite(scale) or scale <= 0.0:
32         scale = 1.0
33     return max(floor, float(0.9 * scale * x.size ** (-0.2)))
34
35
36 def schuster_bandwidth(samples: np.ndarray, *, dimension: int = 1,
37                        min_bandwidth: float = 1e-12) -> float:
38     """Return the scalar bandwidth implied by Schuster's KDE scaling.
39
40     Schuster uses a Gaussian product kernel with a diagonal bandwidth matrix
41     and the factor ``(4 / ((d + 2) N))**(1 / (d + 4))``.  For the scalar
42     residual KDE used by this implementation, ``dimension=1`` and the matrix
43     reduces to ``h**2 * s**2``.  The function exposes the rule explicitly so
44     that paper-aligned runs cannot silently use Silverman's robust constant.
45     It is a scaling rule, not a finite-sample guarantee.
46     """
47     x = np.asarray(samples, dtype=float)
```

```
    if x.ndim != 1 or x.size == 0:
        raise ValueError("samples must be a nonempty one-dimensional array")
    if not np.all(np.isfinite(x)):
        raise ValueError("samples must contain only finite values")
    d = int(dimension)
    if d < 1:
        raise ValueError("dimension must be a positive integer")
    floor = float(min_bandwidth)
    if not np.isfinite(floor) or floor <= 0.0:
        raise ValueError("min_bandwidth must be positive and finite")
    scale = float(np.std(x, ddof=1)) if x.size > 1 else 0.0
    if not np.isfinite(scale) or scale <= 0.0:
        scale = 1.0
    factor = (4.0 / ((d + 2.0) * x.size)) ** (1.0 / (d + 4.0))
    return max(floor, float(scale * factor))
```

Listing A.3 turns residual samples $r_j = g(x, \xi_j)$ into the quantities the optimizer consumes. `kde_safe_probability` is $\hat{p}_h(x)$ of Eq. (31), imposed as Eq. (33). `kde_violation_upper` with `bias=True` returns $1 - \hat{p}_h^b(x)$ of Eq. (38), which is never smaller than the empirical violation frequency. `kde_safe_probability_gradient` is the analytic gradient Eq. (34) for the Gaussian kernel.

Listing A.3: Integrated-KDE chance-constraint estimators and gradient (`kde_cco/constraints/scalar.py`)

```
def kde_safe_probability(residual_values: ArrayLike, h: float,
                         kernel: str | Callable[[np.ndarray], np.ndarray] = "gaussian") ->
    ↪ float | np.ndarray:
    """Estimate ``P(residual <= 0)`` with an integrated KDE."""
    r, was_vector = _validate_residuals(residual_values)
    h = _validate_bandwidth(h)
    return _reduce_sample_axis(_cdf(kernel, -r / h), was_vector)


def kde_violation_upper(residual_values: ArrayLike, h: float,
                        kernel: str | Callable[[np.ndarray], np.ndarray] = "epanechnikov",
                        bias: bool = False) -> float | np.ndarray:
    """Estimate violation probability, optionally using the conservative bias.

    ``bias=True`` uses the *local shifted Epanechnikov surrogate* implemented
    in this package.  Its safe contribution is zero for every positive
    residual, so ``1 - mean(K_B(-r_i/h))`` is pointwise no smaller than the
    empirical strict-violation probability.  This establishes dominance for
    this surrogate; it is not a claim that this translation reproduces Keil
    et al.'s Split-Bernstein construction, and the bound does not apply to an
    ordinary KDE.
    """
    r, was_vector = _validate_residuals(residual_values)
    h = _validate_bandwidth(h)
    if bias:
        if not isinstance(kernel, str) or kernel.lower() != "epanechnikov":
            raise ValueError("the conservative biased surrogate requires the Epanechnikov
    ↪ kernel")
        safe = np.mean(biased_epanechnikov_cdf(-r / h))
    else:
        safe = np.mean(_cdf(kernel, -r / h), axis=0)
```

```
    out = 1.0 - safe
    return float(out) if was_vector else np.asarray(out, dtype=float)


def kde_safe_probability_gradient(residual_values: ArrayLike,
                                  residual_jacobian: ArrayLike,
                                  h: float) -> float | np.ndarray:
    """Analytic chain derivative of Gaussian KDE safe probability.

    The Jacobian has shape ``(n_samples,)`` for one decision variable or
    ``(n_samples, n_variables)`` for several variables.  For
    ``p=N^-1 sum Phi(-r_i/h)``, ``dp/dx=-N^-1 sum phi(r_i/h) dr_i/dx/h``.
    """
    r, was_vector = _validate_residuals(residual_values)
    if not was_vector:
        raise ValueError("analytic gradient requires one-dimensional residual samples")
    jac = _finite_array(residual_jacobian, name="residual_jacobian")
    if jac.ndim == 0:
        jac = jac.reshape(1)
    if jac.ndim == 1:
        if jac.shape[0] != r.shape[0]:
            raise ValueError("residual_jacobian sample dimension does not match residuals"
    ↪ )
        jac = jac[:, None]
        scalar_output = True
    elif jac.ndim == 2:
        if jac.shape[0] != r.shape[0] or jac.shape[1] == 0:
            raise ValueError("residual_jacobian must have shape (n_samples, n_variables)")
        scalar_output = False
    else:
        raise ValueError("residual_jacobian must be one- or two-dimensional")
    h = _validate_bandwidth(h)
    # gaussian_pdf(..., h) is the scaled density and already contains 1 / h.
    weights = gaussian_pdf(-r, h=h)[:, None]
    gradient = -np.mean(weights * jac, axis=0)
    return float(gradient[0]) if scalar_output else np.asarray(gradient, dtype=float)
```

Listing A.4 reduces the residual vector $(g_1, \ldots, g_m)$ to one scalar statistic. `max_violation` is the exact reduction Eq. (40), and `smooth_max` is the log-sum-exp surrogate Eq. (41) with unit weights $w_r = 1$ and temperature τ, evaluated stably through `scipy.special.logsumexp`.

Listing A.4: Exact and smoothed joint statistics (`kde_cco/constraints/joint.py`)

```
def max_violation(residual_matrix: np.ndarray, axis: int | None = None) -> np.ndarray |
    ↪ float:
    """Return the exact maximum residual, optionally along an axis."""
    try:
        return np.max(_matrix(residual_matrix), axis=axis)
    except (TypeError, ValueError, AxisError) as exc:
        raise ValueError("axis is invalid for residual_matrix") from exc


def smooth_max(values: np.ndarray, tau: float, axis: int | None = None) -> np.ndarray |
    ↪ float:
    """Compute ``tau * log(sum(exp(values / tau)))`` stably."""
    if not np.isfinite(tau) or tau <= 0:
```

```
37          raise ValueError("tau must be positive and finite")
38      a = _matrix(values)
39      try:
40          out = tau * logsumexp(a / tau, axis=axis)
41      except (TypeError, ValueError, AxisError) as exc:
42          raise ValueError("axis is invalid for residual_matrix") from exc
43      # The subtraction-free expression is intentionally used instead of a
44      # clipped approximation: for finite values log-sum-exp is a smooth upper
45      # envelope of max and is useful as a differentiable joint surrogate.
46      return out
```

A.2 Benchmark Models

Listing A.5 defines the static nonlinear benchmark of Section 9 (Table 2). The objective is a shifted quadratic with a small coupling term, and the deterministic constraints keep the decision in a compact nonnegative region. The uncertain residual is the circular safety condition $(x_1 + 0.25\,\xi_1)^2 + (x_2 + 0.25\,\xi_2)^2 - 1 \le 0$, evaluated for all samples at once.

Listing A.5: Static nonlinear benchmark (`kde_cco/problems/static_problem.py`)

```
40      def objective(x):
41          x = np.asarray(x, dtype=float)
42          if x.shape != (2,) or not np.all(np.isfinite(x)):
43              raise ValueError("decision must be a finite vector with shape (2,)")
44          return float((x[0] - 0.8) ** 2 + (x[1] - 0.6) ** 2 + 0.05 * x[0] * x[1])
45
46      def deterministic_constraints(x):
47          x = np.asarray(x, dtype=float)
48          if x.shape != (2,) or not np.all(np.isfinite(x)):
49              raise ValueError("decision must be a finite vector with shape (2,)")
50          # Keep decisions in a compact, nonnegative design region.
51          return np.array([x[0], x[1], 2.0 - x[0], 2.0 - x[1], 1.8 - x[0] - x[1]])
52
53      def uncertain_residuals(x, xi):
54          x = np.asarray(x, dtype=float)
55          z = np.asarray(xi, dtype=float)
56          if x.shape != (2,) or not np.all(np.isfinite(x)):
57              raise ValueError("decision must be a finite vector with shape (2,)")
58          if z.ndim == 1:
59              z = z.reshape(1, -1)
60          if z.ndim != 2 or z.shape[1] != 2 or z.shape[0] == 0 or not np.all(np.isfinite(z))
    ↪ :
61              raise ValueError("xi must be a nonempty finite array with shape (n, 2)")
62          # Safety event: residual <= 0.  Broadcast over samples.
63          return (x[0] + sigma * z[:, 0]) ** 2 + (x[1] + sigma * z[:, 1]) ** 2 - radius**2
```

Listing A.6 defines the two-generator dispatch benchmark (Table 4). The cost is quadratic in the output of each generator. The first residual component is the balance deficit $d - r_0 - \xi - (x_1 + x_2)$, where d is the demand, r_0 the nominal renewable output,

and ξ the renewable forecast error. The two capacity components do not depend on ξ, so the stochastic part of the problem is the balance equation alone.

Listing A.6: Dispatch benchmark (`kde_cco/problems/energy_dispatch.py`)

```
    def objective(x):
        x = np.asarray(x, dtype=float)
        return float(0.018 * x[0] ** 2 + 12.0 * x[0] + 0.028 * x[1] ** 2 + 16.0 * x[1])

    def deterministic_constraints(x):
        x = np.asarray(x, dtype=float)
        # Generation nonnegative, capacity limits, and optional spinning reserve.
        return np.array([x[0], x[1], caps[0] - x[0], caps[1] - x[1],
                         x.sum() - (d - r + reserve)])

    def uncertain_residuals(x, xi):
        x = np.asarray(x, dtype=float)
        z = np.asarray(xi, dtype=float)
        if x.shape != (2,) or not np.all(np.isfinite(x)):
            raise ValueError("dispatch must be a finite vector with shape (2,)")
        if z.ndim == 0:
            z = z.reshape(1)
        elif z.ndim == 2 and z.shape[1] == 1:
            z = z[:, 0]
        elif z.ndim != 1:
            raise ValueError("xi must be a scalar, (n,), or (n, 1) renewable-error array")
        if z.size == 0 or not np.all(np.isfinite(z)):
            raise ValueError("xi must be nonempty and finite")
        balance = d - r - z - np.sum(x)
        # Include capacity residuals to support joint chance constraints.
        return np.column_stack((np.broadcast_to(balance, z.shape),
                                np.broadcast_to(np.asarray(x)[0] - caps[0], z.shape),
                                np.broadcast_to(np.asarray(x)[1] - caps[1], z.shape)))
```

Listing A.7 gives the lunar model of Section 7.3. `dynamics` is Eq. (52), and `terminal_residual` and `path_residual_nodes` are the residuals inside the terminal and path events Eqs. (53) and (54). When all n_t node controls are passed, `objective` evaluates the trapezoidal fuel integral whose mismatch with the Euler defects is Eq. (58).

Listing A.7: Lunar landing dynamics, objective, and residuals (`kde_cco/problems/lunar_landing.py`)

```
    def dynamics(self, t, state, control, disturbance=0.0) -> np.ndarray:
        del t
        state_arr = np.asarray(state, dtype=float)
        control_arr = np.asarray(control, dtype=float)
        disturbance_arr = np.asarray(disturbance, dtype=float)
        if state_arr.shape[-1:] != (2,) or not np.all(np.isfinite(state_arr)):
            raise ValueError("state must have a finite trailing dimension of size 2")
        if not np.all(np.isfinite(control_arr)) or not np.all(np.isfinite(disturbance_arr)
    ↪ ):
            raise ValueError("control and disturbance must be finite")
        out = np.empty(np.broadcast(state_arr[..., 0], state_arr[..., 1], control_arr,
    ↪ disturbance_arr).shape + (2,))
```

```
        out[..., 0] = state_arr[..., 1]
        out[..., 1] = -self.gravity + control_arr + disturbance_arr
        return out

    def objective(self, states, controls, time=None) -> float:
        del states
        u = np.asarray(controls, dtype=float).reshape(-1)
        if u.size == 0 or not np.all(np.isfinite(u)):
            raise ValueError("controls must be nonempty and finite")
        if time is None:
            return float(np.mean(u))
        t = np.asarray(time, dtype=float).reshape(-1)
        if t.size not in (u.size, u.size + 1) or t.size < 2 or not np.all(np.isfinite(t)):
            raise ValueError("time must have one value per control node or interval")
        if np.any(np.diff(t) <= 0):
            raise ValueError("time must be strictly increasing")
        if u.size == t.size - 1:
            return float(np.sum(u * np.diff(t)))
        trap = np.trapezoid if hasattr(np, "trapezoid") else np.trapz
        return float(trap(u, t))
```

```
    def terminal_residual(self, state, xi) -> np.ndarray:
        """Terminal residual ``abs(y(tf)-xi_1)-delta``; nonpositive is safe."""
        s = np.asarray(state, dtype=float)
        if s.shape != (2,) or not np.all(np.isfinite(s)):
            raise ValueError("state must be a finite vector with shape (2,)")
        z = _error_vector(xi, name="xi")
        return np.abs(s[0] - z) - self.delta
```

```
    def path_residual_nodes(self, controls, xi) -> np.ndarray:
        """Node-by-sample path residuals with shape ``(n_nodes, n_samples)``."""
        U = np.asarray(controls, dtype=float).reshape(-1)
        if U.size == 0 or not np.all(np.isfinite(U)):
            raise ValueError("controls must be nonempty and finite")
        z = _error_vector(xi, name="xi")
        return U[:, None] + z[None, :] - self.u_max
```

A.3 Optimization

Listing A.8 connects a chance constraint to SciPy's SLSQP. A chance constraint is passed as a function that is nonpositive when it holds, for example $(1-\varepsilon)-\hat{p}_h(x)$. SciPy instead expects inequality functions $c(x) \geq 0$, so `_constraint_from_chance` negates the function, and `_solve` passes it together with the deterministic constraints and the box bounds to `scipy.optimize.minimize`.

Listing A.8: Chance-constraint adapter for SLSQP (`kde_cco/solvers/static.py`)

```
def _constraint_from_chance(chance_constraint: Any) -> Callable[[np.ndarray], np.ndarray]:
    candidate = chance_constraint
    if isinstance(candidate, Mapping):
        for key in ("constraint", "constraints", "function", "fun"):
            if key in candidate:
```

```
                candidate = candidate[key]
                break
        else:
            raise TypeError("chance_constraint mapping must contain a callable constraint"
    ↪ )
    elif hasattr(candidate, "constraint"):
        candidate = candidate.constraint
    if not callable(candidate):
        raise TypeError("chance_constraint must be callable or contain a callable '
    ↪ constraint'")

    residual = _vector_constraint(candidate, name="chance_constraint")

    def feasible(x: np.ndarray) -> np.ndarray:
        # The package convention is residual <= 0.  Flip sign for SciPy.
        return -residual(x)

    return feasible
```

```
    constraints: list[dict[str, Any]] = []
    deterministic = getattr(problem, "deterministic_constraints", None)
    if deterministic is not None:
        constraints.append({"type": "ineq", "fun": _vector_constraint(deterministic, name=
    ↪ "deterministic_constraints")})
    if chance_constraint is not None:
        constraints.append({"type": "ineq", "fun": _constraint_from_chance(
    ↪ chance_constraint)})

    scipy_options = {"ftol": float(tolerance), "maxiter": int(maxiter)}
    if options:
        scipy_options.update(dict(options))
    started = perf_counter()
    result = minimize(objective_scalar, x0, method=method, bounds=bounds,
                      constraints=constraints, options=scipy_options)
```

Listing A.9 is the optimal-control transcription of Section 7. `transcribe_euler` returns the forward Euler defects Eq. (51) and the boundary residuals. The inner function `values` of `_chance_constraints` computes the terminal and node-wise path margins of one continuation stage. The nominal stage uses the sample mean, the worst-case stage the largest residual, and the scenario stage one inequality per sample. The unbiased stage uses the Gaussian KDE and the biased stage the shifted Epanechnikov kernel, both in the safe-probability form equivalent to Eq. (50). The core of `solve_collocation` stacks states and controls into one decision vector and passes the defects as equalities and the chance margins as inequalities to SLSQP.

Listing A.9: Euler transcription and KDE chance stages (`kde_cco/solvers/collocation.py`)

```
def transcribe_euler(problem, time, states, controls):
    """Return the Euler transcription and its defect and boundary residuals."""
    t = np.asarray(time, dtype=float).reshape(-1)
    X = np.asarray(states, dtype=float)
    U = np.asarray(controls, dtype=float).reshape(-1)
```

```
    if t.size < 2 or not np.all(np.isfinite(t)) or np.any(np.diff(t) <= 0):
        raise ValueError("time must be a finite strictly increasing mesh")
    if X.shape != (t.size, 2) or U.size not in {t.size, t.size - 1}:
        raise ValueError("states must be (N,2), controls must contain N or N-1 values")
    interval_u = U[:-1] if U.size == t.size else U
    defects = np.asarray([
        X[k + 1] - X[k] - (t[k + 1] - t[k]) * problem.dynamics(t[k], X[k], interval_u[k])
        for k in range(t.size - 1)
    ])
    initial_residual = X[0] - np.asarray(problem.initial_state, dtype=float)
    terminal_residual = X[-1] - np.asarray(problem.terminal_state, dtype=float)
    return {
        "time": t, "states": X, "controls": U,
        "interval_controls": interval_u, "defects": defects,
        "initial_residual": initial_residual,
        "terminal_residual": terminal_residual,
        "n_state_variables": int(X.size), "n_control_variables": int(U.size),
    }
```

```
    def values(states, controls):
        terminal = problem.terminal_residual(states[-1], terminal_samples)
        path = problem.path_residual_nodes(controls, path_samples)
        if name in {"nominal", "mean"}:
            terminal = problem.terminal_residual(states[-1], np.mean(terminal_samples))
            path = problem.path_residual_nodes(controls, np.mean(path_samples))
        if name in {"nominal", "mean", "worst_case", "worst"}:
            return np.asarray([-np.max(terminal), -np.max(path)], dtype=float)
        if name == "scenario":
            # Keep one inequality per sampled event.  This is deliberately
            # distinct from the worst-case reduction above.
            return np.r_[-terminal.reshape(-1), -path.reshape(-1)]
        path_samples_by_node = path.T
        if name == "unbiased_kde":
            terminal_safe = float(kde_safe_probability(terminal, ht, kernel))
            path_safe = np.asarray(kde_safe_probability(path_samples_by_node, hp, kernel),
    ↪  dtype=float)
        else:
            terminal_safe = 1.0 - float(kde_violation_upper(
                terminal, ht, kernel, bias=True))
            # The biased surrogate currently accepts vector samples only.  Apply
            # it independently at each node so the returned margins retain the
            # node dimension used by the NLP.
            path_safe = 1.0 - np.asarray([
                kde_violation_upper(path_samples_by_node[:, node], hp, kernel, bias=True)
                for node in range(path_samples_by_node.shape[1])
            ], dtype=float)
        return np.r_[terminal_safe - (1.0 - epsilon_a),
                     path_safe - (1.0 - epsilon_b)]
```

```
    ts, ps = _samples(terminal_samples), _samples(path_samples)
    ea = problem.epsilon_a if epsilon_a is None else float(epsilon_a)
    eb = problem.epsilon_b if epsilon_b is None else float(epsilon_b)
    chance, metadata = _chance_constraints(problem, stage, ts, ps, ea, eb, bandwidth)

    def unpack(z):
        return z[:2 * n].reshape(n, 2), z[2 * n:]

    def equality(z):
```

```
        X, U = unpack(z)
        tr = transcribe_euler(problem, t, X, U)
        # Position is governed by its chance event; only terminal velocity is
        # a deterministic boundary condition in the published benchmark.
        return np.r_[tr["defects"].reshape(-1), tr["initial_residual"],
                     tr["terminal_residual"][1]]

    def inequality(z):
        X, U = unpack(z)
        return chance(X, U)

    z0 = np.r_[X0.reshape(-1), U0]
    bounds = [(None, None)] * (2 * n) + [problem.control_bounds] * n
    started = _clock.perf_counter()
    result = minimize(
        lambda z: problem.objective(*unpack(z), t), z0, method=method,
        bounds=bounds,
        constraints=[{"type": "eq", "fun": equality},
                     {"type": "ineq", "fun": inequality}],
        options={"maxiter": int(maxiter), "ftol": float(tolerance)},
    )
```

Listing A.10 is the bandwidth continuation of the paper-aligned profile (Table 3). `_continuation` solves a sequence of problems with bandwidths $c\,h_N$, warm-starts each solve from the previous decision, and stops at the first failure. The paper-aligned unbiased KDE uses one Gaussian solve at the bandwidth h_N of Eq. (61). The paper-aligned biased KDE runs the multipliers $c \in \{0.25, 0.5, 1, 2\}$ with the $b(h) = h$ Epanechnikov surrogate and keeps the last successful stage.

Listing A.10: Bandwidth continuation in the paper-aligned profile (`kde_cco/experiments/paper_aligned.py`)

```
def _continuation(problem, initial, constraint_factory, solve, multipliers):
    """Solve from smooth/larger bandwidth to smaller bandwidth."""
    current = np.asarray(initial, dtype=float)
    records = []
    for multiplier in multipliers:
        result = solve(problem, initial_guess=current,
                       chance_constraint=constraint_factory(float(multiplier)),
                       maxiter=500)
        records.append((float(multiplier), result))
        current = result["decision"]
        if not result["success"]:
            break
    return records
```

```
            h = schuster_bandwidth(scale_samples, dimension=1)

            def residual(x, z):
                return np.asarray(problem.uncertain_residuals(x, z), float).reshape(-1)

            for method in ("schuster_gaussian", "keil_biased_epanechnikov"):
                if method == "schuster_gaussian":
                    multipliers = (1.0,)
```

```
81                  estimator = lambda r, bw: 1.0 - float(kde_safe_probability(r, bw, "
    ↪ gaussian"))
82                  def make_constraint(bw):
83                      return lambda x: float((1.0 - epsilon) - kde_safe_probability(
    ↪ residual(x, train), bw, "gaussian"))
84              else:
85                  # Keil's B(h)=h shift is exact for the Epanechnikov kernel.
86                  # The local analogue begins with a sharp surrogate and
87                  # increases h until SLSQP can resolve the conservative
88                  # constraint.  This is the Keil continuation direction;
89                  # the mesh-error trigger is unavailable in this adapter.
90                  multipliers = (0.25, 0.5, 1.0, 2.0)
91                  estimator = lambda r, bw: float(kde_violation_upper(r, bw, "
    ↪ epanechnikov", True))
92                  def make_constraint(bw):
93                      return lambda x: float(kde_violation_upper(residual(x, train), bw,
    ↪  "epanechnikov", True) - epsilon)
94              started = time.perf_counter()
95              continuation = _continuation(problem, np.array([.5, .5]),
96                                           lambda multiplier: make_constraint(h *
    ↪ multiplier),
97                                           solve_static, multipliers)
98              successful = [(multiplier, result) for multiplier, result in continuation
99                            if result["success"]]
100             selected_multiplier, solved = successful[-1] if successful else
    ↪ continuation[-1]
```

Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, dass alle Stellen der Arbeit, die wörtlich oder sinngemäß aus anderen Quellen übernommen wurden, als solche kenntlich gemacht sind und dass die Arbeit in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt wurde.

Nürnberg, den 4. November 2026

Syed Ansab Waqar Gillani